

Gestion de projets

Plan :

Chapitre 1 : Introduction à la gestion de projet informatique

- Notion de projet
- Spécificités des projets informatiques
- Contraintes fondamentales d'un projet
- Typologies de projets informatiques
- Enjeux de la gestion de projet informatique
- Rôle de la gestion de projet informatique

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

- Introduction
- Les parties prenantes d'un projet informatique
- La Maîtrise d'Ouvrage (MOA)
- La Maîtrise d'Œuvre (MOE)
- L'Assistance à Maîtrise d'Ouvrage (AMOA)
- Le chef de projet informatique
- L'équipe projet
- Gouvernance des projets IT
- Communication et coordination entre acteurs
- Facteurs clés de succès liés aux acteurs

Chapitre 3 : Les étapes projet

- L'avant-projet.
- La conception et la réalisation
- La validation
- La recette et la réception
- La livraison
- Le bilan de fin de projet

Chapitre 4 : Les cycles de vie

- Les normes
- Les modèles séquentiels
- Le cycle de vie « itératif »
- Les méthodes « agiles »
- Les bonnes pratiques

Chapitre 5 : Le suivi du projet

- Les réunions
- La communication
- Les indicateurs et les tableaux de bord
- La gestion du portefeuille projet

Chapitre 6 : Organiser le changement

- L'accompagnement au changement
- La conduite du changement

Plan :

Chapitre 7 : Planification d'un projet

- Le découpage du projet
- L'estimation des charges et des délais
- L'ordonnancement des tâches
- Le diagramme de Gantt
- La planification des ressources
- Le plan de projet (plan de management)
- La gestion des risques dans la planification
- La planification en méthodes agiles
- Les outils de planification

Chapitre 8 - L'estimation de charge

- Le constat.
- Pourquoi sous-estime-t-on ?
- Quel est le processus d'estimation ?
- Quelles sont les méthodes d'estimation ?
- Les facteurs d'ajustement.
- L'utilisation des méthodes selon la typologie des projets

Chapitre 9 - Les risques

- Identifier
- Prioriser
- Prévenir ou réduire les risques
- Suivre les risques

Chapitre 1 : Introduction à la gestion de projet informatique

- Notion de projet
- Spécificités des projets informatiques
- Contraintes fondamentales d'un projet
- Typologies de projets informatiques
- Enjeux de la gestion de projet informatique
- Rôle de la gestion de projet informatique

Chapitre 1: Introduction à la gestion de projet informatique

1.1. Notion de projet

1.1.1. Définition d'un projet

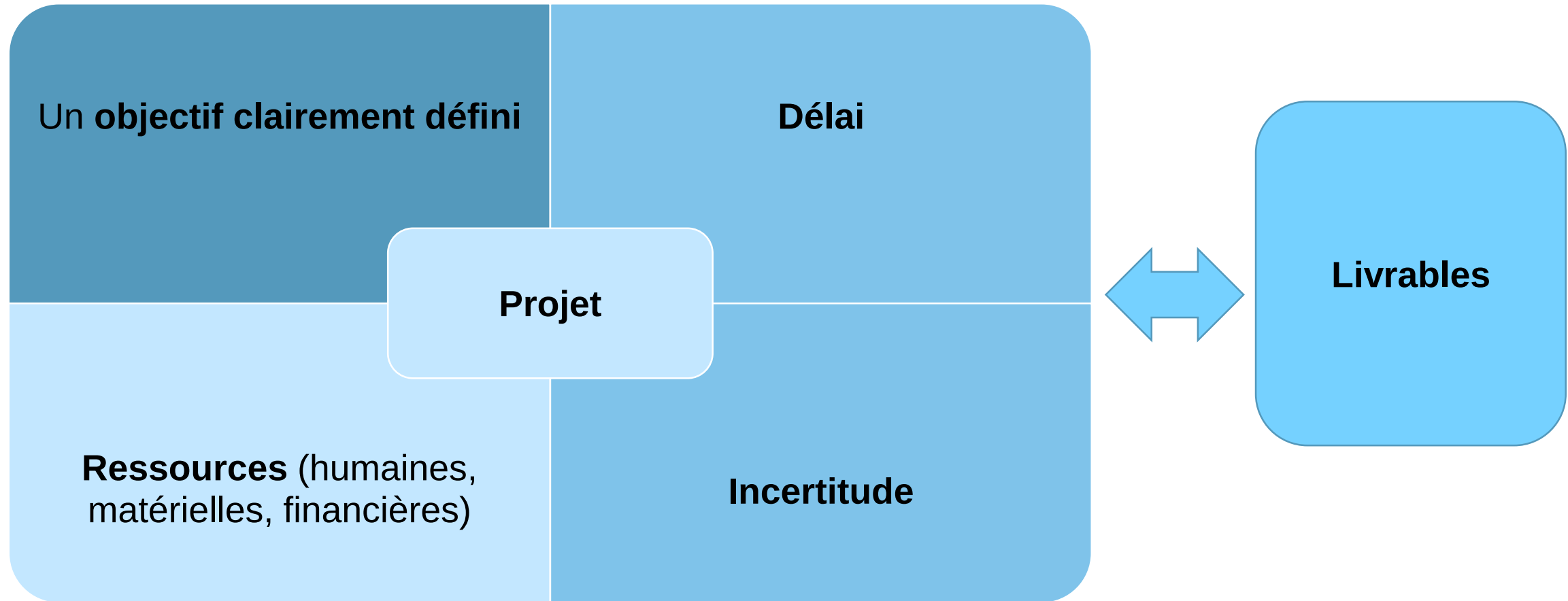
Un **projet** est un ensemble d'activités **temporaires**, **organisées** et **coordonnées**, visant à atteindre un **objectif précis**, par la production d'un **résultat unique** (produit, service ou solution), dans le respect de contraintes prédéfinies.

Un projet se caractérise donc par :

- une **date de début** et une **date de fin**,
- un **objectif clairement défini**,
- un **livrable identifiable**,
- des **ressources limitées** (humaines, matérielles, financières),
- un **niveau d'incertitude** plus ou moins élevé.

Chapitre 1: Introduction à la gestion de projet informatique

1.1. Notion de projet



Chapitre 1: Introduction à la gestion de projet informatique

1.1. Notion de projet

1.1.2. Différence entre projet et activité opérationnelle

Projet	Activité opérationnelle
Temporaire	Permanente
Objectif unique	Processus répétitif
Forte incertitude	Faible incertitude
Innovation fréquente	Routine

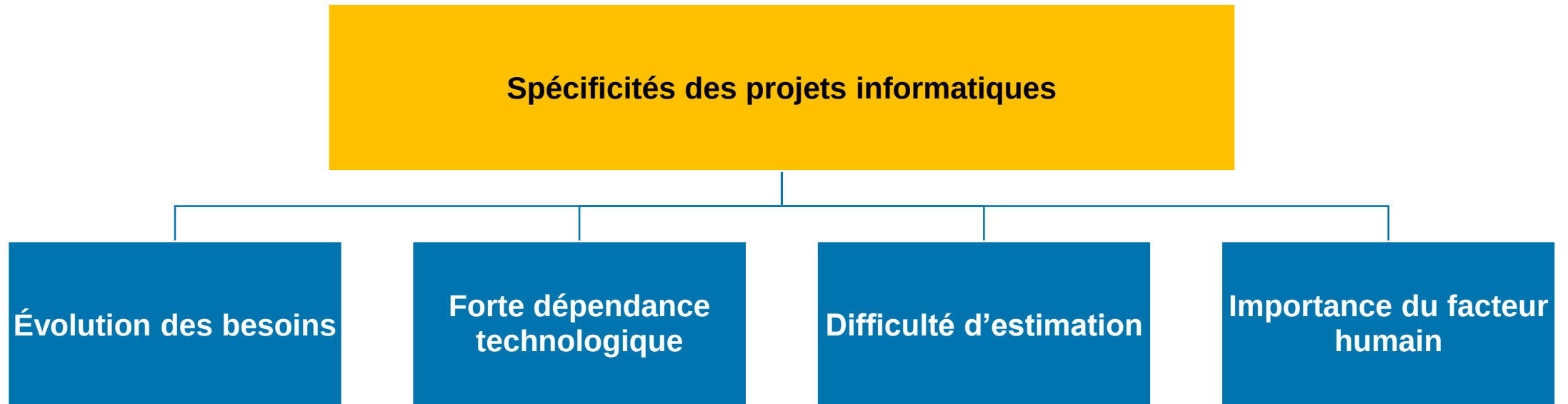
1.1.3. Programme et portefeuille de projets

- **Programme** : ensemble de projets liés entre eux, coordonnés pour atteindre un objectif stratégique commun.
- **Portefeuille de projets** : ensemble de projets et programmes pilotés au niveau stratégique d'une organisation.

Chapitre 1: Introduction à la gestion de projet informatique

1.2. Spécificités des projets informatiques

Les projets informatiques présentent des **particularités** qui les rendent plus complexes que d'autres types de projets.



Chapitre 1: Introduction à la gestion de projet informatique

1.2. Spécificités des projets informatiques

1.2.1. Évolution des besoins

- Les besoins des utilisateurs sont souvent mal définis au départ
- Ils évoluent avec le temps et l'usage
- Risque élevé de changements en cours de projet

1.2.2. Forte dépendance technologique

- Technologies rapidement obsolètes
- Dépendance aux outils, frameworks et plateformes
- Problèmes de compatibilité et d'intégration

1.2.3. Difficulté d'estimation

- Estimation imprécise des charges et des délais
- Dépendance aux compétences de l'équipe
- Incertitude sur les risques techniques

1.2.4. Importance du facteur humain

- Travail collaboratif
- Communication essentielle
- Changements possibles dans l'équipe

Chapitre 1: Introduction à la gestion de projet informatique

1.3. Contraintes fondamentales d'un projet

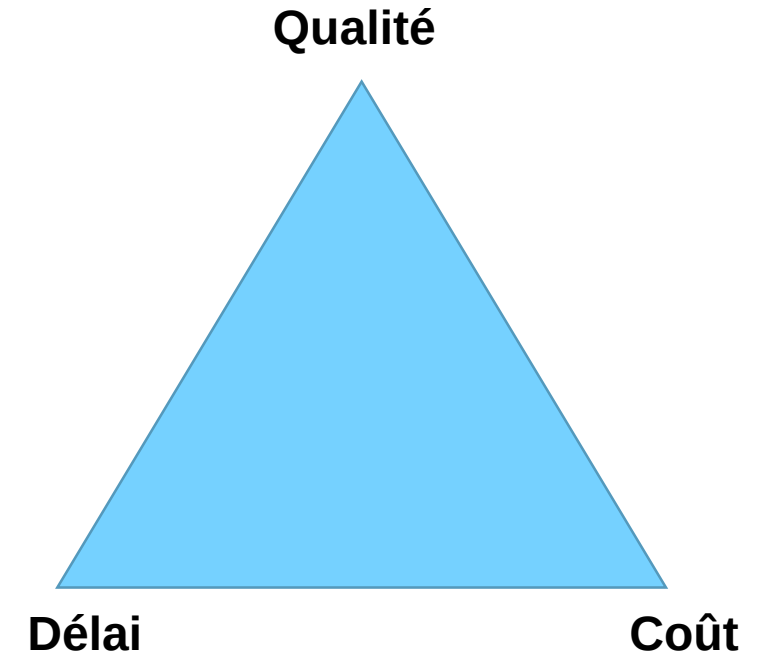
1.3.1. Le triangle de gestion de projet (QCD)

Tout projet est soumis à trois contraintes majeures :

- **Qualité** : conformité aux besoins et exigences
- **Coût** : respect du budget alloué
- **Délai** : respect des échéances

Ces contraintes sont **interdépendantes** :

→ Modifier l'une impacte nécessairement les autres.



Si on veut **réduire les délais** sans toucher au budget, il faut réduire le périmètre ou accepter moins de qualité. Si on veut **ajouter des fonctionnalités**, il faut plus de temps ou plus de budget. Le chef de projet doit négocier ces arbitrages avec le commanditaire.

Chapitre 1: Introduction à la gestion de projet informatique

1.3. Contraintes fondamentales d'un projet

1.3.2. Le périmètre du projet

Le **périmètre** définit :

- ce qui est **inclus** dans le projet,
- ce qui est **exclu**.

Une mauvaise définition du périmètre entraîne :

- dérive des besoins,
- dépassement des délais et des coûts.

Chapitre 1: Introduction à la gestion de projet informatique

1.4. Typologies de projets informatiques

1.4.1. Projets logiciels

- Applications web
- Applications mobiles
- Logiciels métiers

1.4.2. Projets d'infrastructure IT

- Réseaux informatiques
- Serveurs
- Data centers
- Sécurité informatique

1.4.3. Projets data et intelligence artificielle

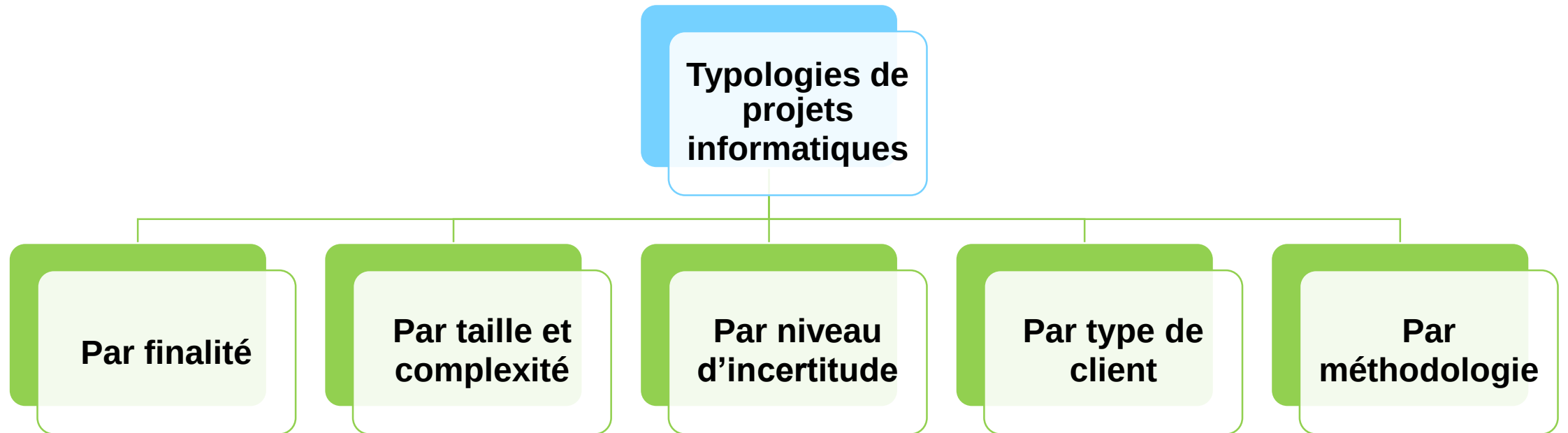
- Big Data
- Data warehouses
- Machine Learning
- Business Intelligence

1.4.4. Projets de transformation digitale

- Dématérialisation des processus
- E-gouvernement
- Digitalisation des services

Chapitre 1: Introduction à la gestion de projet informatique

1.4. Typologies de projets informatiques



Chapitre 1: Introduction à la gestion de projet informatique

1.4. Typologies de projets informatiques

❑ Par finalité

- **Développement applicatif** : nouvelles applis, ERP, mobile, outils métier.
- **Infrastructure** : serveurs, cloud, virtualisation, datacenter.
- **Refonte/modernisation** : migration legacy, amélioration maintenabilité.
- **Sécurité** : RGPD, cybersécurité, durcissement SI.

❑ Par taille et complexité

- **Petits projets** : <6 mois, <5 pers., cycles courts, organisation légère.
- **Moyens projets** : 6 mois–2 ans, 5–20 pers., organisation structurée, suivi formel.
- **Grands projets** : >2 ans, >20 pers., sous-projets, gouvernance complexe.
- **Programmes** : regroupent plusieurs projets interdépendants.

Chapitre 1: Introduction à la gestion de projet informatique

1.4. Typologies de projets informatiques

❑ Par type de client

- Interne : au sein de l'organisation, relations directes.
- Externe : client extérieur, relations contractuelles.
- Mixte/partenarial : plusieurs organisations, gouvernance partagée.

❑ Par niveau d'incertitude

- Routine : besoins stables, faible risque.
- Amélioration : innovation modérée, risque moyen.
- Innovation : expérimentations nouvelles, risque élevé.
- Rupture : transformation radicale, très haut risque.

❑ Par méthodologie

- Cascade (Waterfall) : séquentiel, validation formelle, livraison unique.
- Agile (Scrum, Kanban) : itérations courtes, livraisons fréquentes, adaptation continue.
- Hybride : mix Cascade + Agile, prédictibilité + flexibilité.

Chapitre 1: Introduction à la gestion de projet informatique

1.5. Enjeux de la gestion de projet informatique

La gestion de projet informatique regroupe l'ensemble des méthodes, outils et compétences permettant de **planifier, organiser, piloter et contrôler** des projets de **systèmes d'information** et de développement logiciel. Elle vise à livrer **un produit ou un service conforme aux besoins du client**, dans le respect des contraintes de **coût, délais, qualité et risques**.

Chapitre 1: Introduction à la gestion de projet informatique

1.5. Enjeux de la gestion de projet informatique

Enjeux

Stratégiques

Économiques

Techniques

Humains &
organisationnels

Gouvernance & pilotage

Chapitre 1: Introduction à la gestion de projet informatique

1.5. Enjeux de la gestion de projet informatique

❑ Stratégiques

- Transformation digitale : digitalisation, nouveaux modèles, expérience client.
- Avantage concurrentiel : innovation rapide, optimisation coûts...
- Conformité & risques : RGPD, normes, sécurité.
- Agilité organisationnelle : systèmes évolutifs, adaptabilité.

❑ Économiques

- Maîtrise des coûts et respect du budget.
- Retour sur investissement (ROI) : gains quantifiables et qualitatifs.
- Coût du retard : opportunités perdues, pénalités.
- Optimisation du portefeuille de projets.

Chapitre 1: Introduction à la gestion de projet informatique

1.5. Enjeux de la gestion de projet informatique

❑ Techniques

- Maîtrise de la complexité et architecture modulaire.
- Qualité et robustesse : réduction de la dette technique.
- Intégration et interopérabilité.
- Sécurité : Security by Design.

❑ Humains & organisationnels

- Gestion du changement et adoption utilisateur.
- Mobilisation et montée en compétences des équipes.
- Collaboration transverse entre métiers et IT.
- Gestion du stress et motivation.

❑ Gouvernance & pilotage

- Visibilité et transparence pour sponsors et direction.
- Prise de décision éclairée (technique, périmètre, ressources).
- Alignement stratégique et ajustement en continu.
- Capitalisation et amélioration continue.

Chapitre 1: Introduction à la gestion de projet informatique

1.5. Enjeux de la gestion de projet informatique

Une bonne gestion de projet informatique permet de :

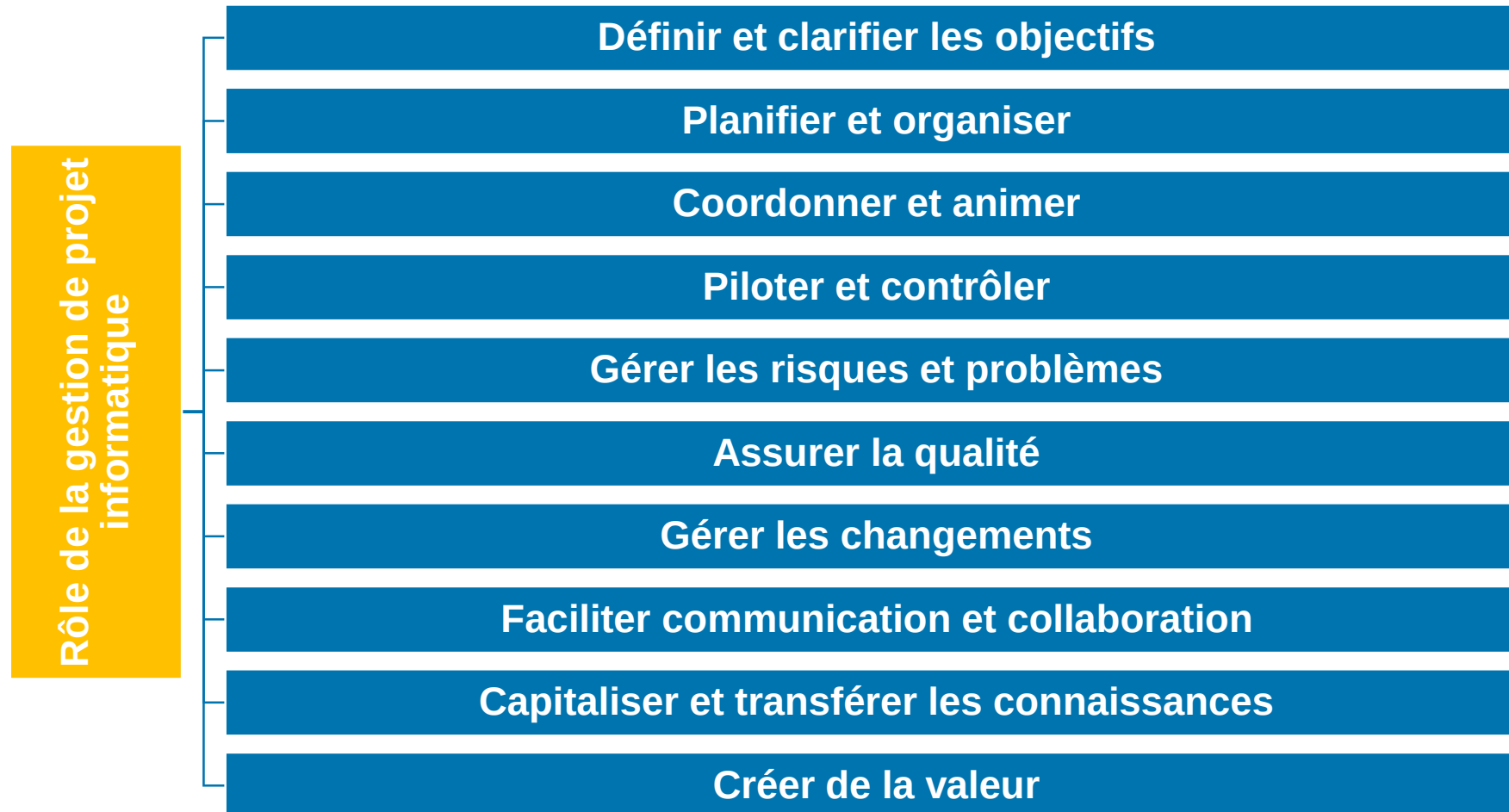
- ✓ maîtriser les coûts,
- ✓ respecter les délais,
- ✓ améliorer la qualité des livrables,
- ✓ réduire les risques,
- ✓ satisfaire les utilisateurs.

À l'inverse, une mauvaise gestion peut entraîner :

- échec du projet,
- dépassement budgétaire,
- rejet par les utilisateurs,
- pertes financières importantes.

Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique



Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique

❑ Définir et clarifier les objectifs

- Traduire les besoins métier en spécifications techniques.
- Délimiter le périmètre et fixer les critères de succès.
- Aligner toutes les parties prenantes.

❑ Planifier et organiser

- Découper le projet en tâches gérables.
- Estimer ressources et budget, séquencer les tâches.
- Définir rôles, responsabilités et allocation des ressources.

❑ Coordonner et animer

- Dynamiser l'équipe et maintenir la motivation.
- Faciliter la communication et la collaboration.
- Assurer la cohérence entre équipes et intervenants.

Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique

❑ **Piloter et contrôler**

- Suivre l'avancement, le budget et la qualité.
- Détecter les écarts et prendre des mesures correctives.
- Reporting transparent et gestion de configuration.

❑ **Gérer les risques et problèmes**

- Identifier, évaluer et prioriser les risques.
- Planifier réponses et gérer crises ou problèmes.

❑ **Assurer la qualité**

- Définir standards, mettre en place processus et contrôles.
- Suivre l'amélioration continue.

Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique

❑ Gérer les changements

- Processus formel de modification et adaptation du projet.
- Protection de l'équipe contre les interruptions fréquentes.

❑ Faciliter communication et collaboration

- Communication vers le haut, horizontal et vers le bas.
- Gestion des parties prenantes et engagement adapté.

❑ Capitaliser et transférer les connaissances

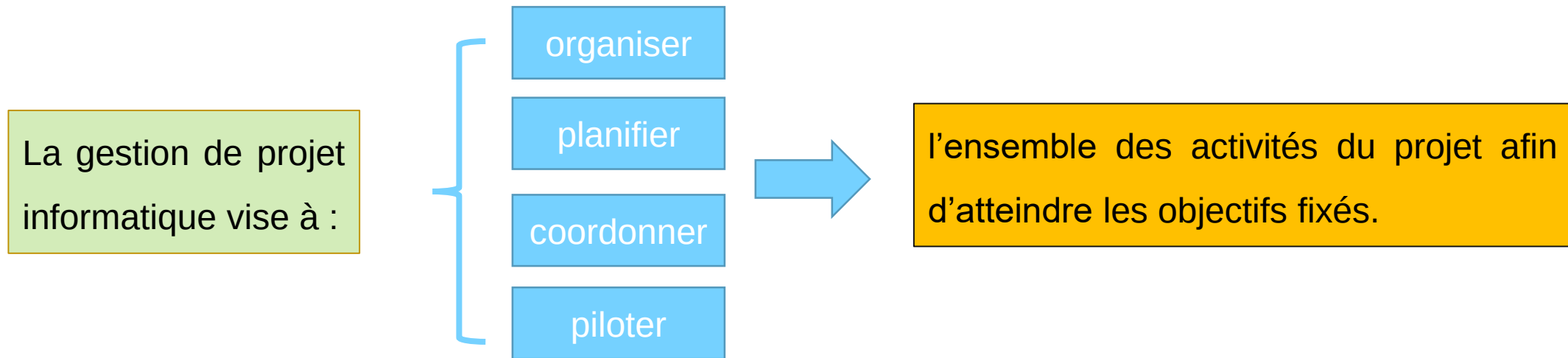
- Documentation, transfert vers exploitation et retour d'expérience.
- Amélioration continue des pratiques.

❑ Créer de la valeur

- Optimiser les ressources et réduire les gaspillages.
- Accélérer le time-to-value et gérer l'incertitude.

Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique



Elle constitue aujourd'hui une **compétence clé** pour les ingénieurs et les responsables IT.

Le projet informatique est une **activité complexe**, marquée par l'incertitude, le changement et la dépendance technologique.

La **gestion de projet informatique** est donc indispensable pour garantir le succès des projets IT.

Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique

❑ Trois types de gestion:

- Gestion de la **productivité**: organisation, direction de travaux et contrôle (suivi)
- Gestion des **ressources humains et matériels**: choix de l'équipe, affectation de personnel, coordination, sélection matériel, gestion du budget et contrôle des coûts
- Gestion du **temps** pour la maîtrise des délais

❑ Ces trois types de gestion sont interdépendants

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

- 2.1. Introduction
- 2.2. Les parties prenantes d'un projet informatique
- 2.3. La Maîtrise d'Ouvrage (MOA)
- 2.4. La Maîtrise d'Œuvre (MOE)
- 2.5. L'Assistance à Maîtrise d'Ouvrage (AMOA)
- 2.6. Le chef de projet informatique
- 2.7. L'équipe projet
- 2.8. Gouvernance des projets IT
- 2.9. Communication et coordination entre acteurs
- 2.10. Facteurs clés de succès liés aux acteurs

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.1. Introduction

Un projet informatique ne repose pas uniquement sur la technologie. Son succès dépend largement de la **coordination entre les acteurs**, de la **clarté des rôles** et de la **qualité de la gouvernance** mise en place.

De nombreux échecs de projets IT sont dus à :

- ☐ une mauvaise définition des responsabilités,
- ☐ des conflits entre acteurs,
- ☐ une absence de mécanismes de décision clairs,
- ☐ une communication inefficace.

☞ Ce chapitre vise à identifier **tous les acteurs d'un projet IT**, leurs **rôles respectifs** et les **structures de gouvernance** permettant de piloter efficacement le projet.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.2. Les parties prenantes d'un projet informatique

2.2.1. Définition des parties prenantes

Les **parties prenantes** sont toutes les personnes, groupes ou organisations :

- ☐ impliqués dans le projet,
- ☐ ou pouvant influencer son déroulement,
- ☐ ou étant impactés par ses résultats.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.2. Les parties prenantes d'un projet informatique

2.2.2. Typologie des parties prenantes

On distingue :

a) Parties prenantes internes

- Direction générale
- Direction des systèmes d'information (DSI)
- Chef de projet
- Équipe projet
- Utilisateurs internes

b) Parties prenantes externes

- Clients
- Fournisseurs
- Prestataires
- Consultants
- Partenaires technologiques

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.2. Les parties prenantes d'un projet informatique

2.2.3. Importance de l'identification des parties prenantes

Une mauvaise identification des parties prenantes peut entraîner :

- des résistances au changement,
- des conflits,
- des retards,
- un rejet du projet.

👉 **La gestion des parties prenantes consiste à :**

- identifier leurs attentes,
- analyser leur pouvoir et leur influence,
- adapter la communication.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.3. La Maîtrise d'Ouvrage (MOA)

2.3.1. Définition de la MOA

La **Maîtrise d'Ouvrage (MOA)** représente le **client** ou le **donneur d'ordre** du projet. Elle est responsable de l'**expression des besoins** et de la **validation des résultats**.

2.3.2. Rôles et responsabilités de la MOA

- ✓ définit les objectifs du projet,
- ✓ exprime les besoins fonctionnels,
- ✓ rédige ou valide le **cahier des charges**,
- ✓ valide les livrables,
- ✓ finance le projet,
- ✓ décide des orientations majeures.

☞ La MOA répond à la question :
« **Que faut-il faire ?** »

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.4. La Maîtrise d'Œuvre (MOE)

2.4.1. Définition de la MOE

La **Maîtrise d'Œuvre (MOE)** est responsable de la **réalisation technique** du projet. Elle conçoit, développe et met en œuvre la solution informatique.

2.4.2. Rôles et responsabilités de la MOE

- ✓ analyse la faisabilité technique,
- ✓ conçoit l'architecture du système,
- ✓ développe la solution,
- ✓ réalise les tests,
- ✓ déploie le système,
- ✓ assure la maintenance technique.

☞ La MOE répond à la question :
« **Comment le faire ?** »

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.5. L'Assistance à Maîtrise d'Ouvrage (AMOA)

2.5.1. Définition de l'AMOA

L'**AMOA** est un acteur **intermédiaire** entre la MOA et la MOE.

Elle accompagne la MOA dans la définition, le pilotage et la réussite du projet.

2.5.2. Rôles de l'AMOA

- ✓ aide à l'expression des besoins,
- ✓ formalise les exigences fonctionnelles,
- ✓ participe au choix des solutions,
- ✓ suit l'avancement du projet,
- ✓ facilite la communication MOA–MOE,
- ✓ accompagne la conduite du changement.

2.5.3. Intérêt de l'AMOA

L'AMOA est particulièrement utile lorsque :

- ✓ la MOA manque de compétences techniques,
- ✓ le projet est complexe,
- ✓ plusieurs prestataires sont impliqués.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

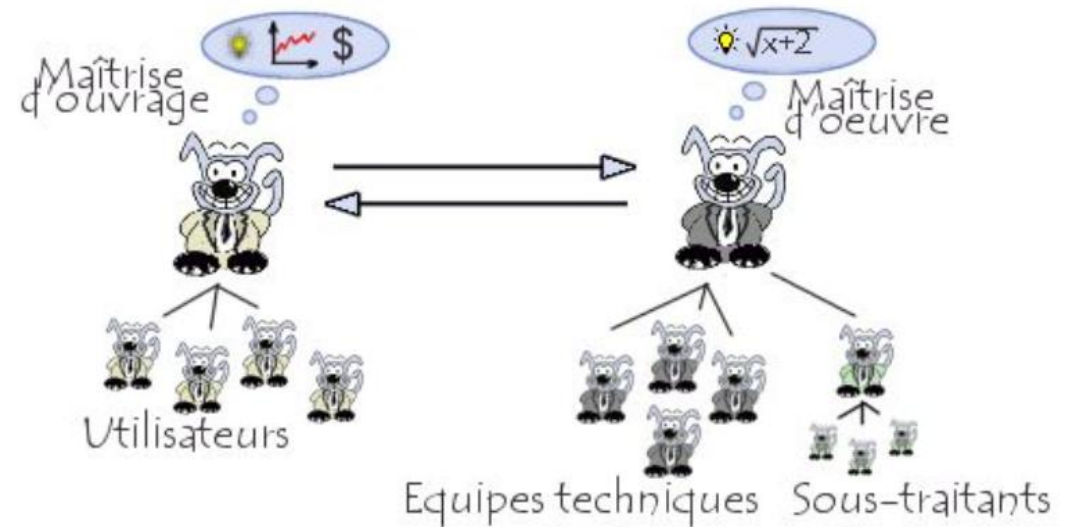
Maître d'œuvre et maître d'ouvrage

Un projet vise à satisfaire un ensemble de **besoins** exprimés par des futurs utilisateurs. Mais il est conduit à terme par des **spécialistes** en développement de SI



Que faut-il faire ?

Comment le faire ?



Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.6. Le chef de projet informatique

2.6.1. Définition

Le **chef de projet informatique** est le responsable opérationnel du projet.

Il coordonne l'ensemble des acteurs afin d'atteindre les objectifs du projet.

2.6.2. Missions principales

- planifie les activités,
- organise les ressources,
- suit l'avancement,
- gère les risques,
- assure la communication,
- rend compte à la gouvernance.

2.6.3. Positionnement du chef de projet

Il se situe :

- entre la MOA et la MOE,
- entre la direction et l'équipe projet.

Il joue un rôle clé de **coordination et de médiation**.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.7. L'équipe projet

2.7.1. Composition de l'équipe projet

Une équipe projet IT peut inclure :

- développeurs,
- architectes,
- testeurs,
- analystes fonctionnels,
- administrateurs systèmes,
- experts métiers.

2.7.2. Fonctionnement de l'équipe projet

- Travail collaboratif
- Répartition claire des rôles
- Responsabilisation
- Communication régulière

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.8. Gouvernance des projets IT

2.8.1. Définition de la gouvernance

La **gouvernance du projet** désigne l'ensemble des **mécanismes de décision**, de **pilotage** et de **contrôle** permettant d'assurer l'alignement du projet avec la stratégie de l'organisation.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.8. Gouvernance des projets IT

2.8.2. Les instances de gouvernance

a) Comité de pilotage

- Instance stratégique
- Prend les décisions majeures
- Valide les orientations et les livrables clés

b) Comité projet

- Instance opérationnelle
- Suit l'avancement
- Gère les problèmes quotidiens

2.8.3. Rôle de la gouvernance

La gouvernance permet de :

- clarifier les responsabilités,
- accélérer la prise de décision,
- maîtriser les risques,
- assurer la cohérence stratégique.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.9. Communication et coordination entre acteurs

2.9.1. Importance de la communication

Une communication défailante est l'une des principales causes d'échec des projets IT.

2.9.2. Outils de communication

- Réunions
- Rapports d'avancement
- Tableaux de bord
- Outils collaboratifs (Teams, Jira, Trello)

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.10. Facteurs clés de succès liés aux acteurs

- ☐ Clarté des rôles
- ☐ Engagement de la MOA
- ☐ Compétence du chef de projet
- ☐ Implication des utilisateurs
- ☐ Gouvernance efficace

Le succès d'un projet informatique repose avant tout sur les **acteurs**, leurs **rôles clairement définis** et une **gouvernance efficace**.

La technologie seule ne suffit pas : la dimension humaine et organisationnelle est déterminante.

Chapitre 3 : Les étapes projet

- L'avant-projet.
- La conception et la réalisation
- La validation
- La recette et la réception
- La livraison
- Le bilan de fin de projet

Chapitre 3 : Les étapes projet

Introduction

- ❑ La **gestion d'un projet informatique** repose sur une démarche structurée composée de plusieurs **phases**, de la conception à la clôture. Chaque phase a des objectifs, des livrables et des critères de validation précis.
- ❑ Selon la **méthodologie utilisée**, ces étapes peuvent s'adapter ou se chevaucher, mais l'essentiel est d'assurer une progression maîtrisée, documentée et validée par l'ensemble des parties prenantes.

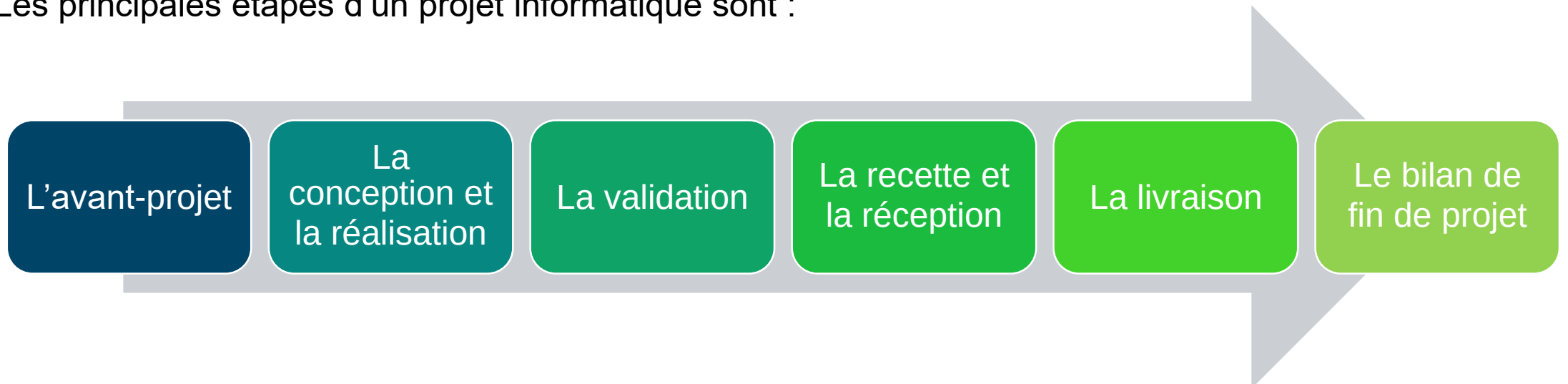
Chapitre 3 : Les étapes projet

Introduction

Chaque étape a des **objectifs précis**, des **acteurs impliqués**, des **activités spécifiques** et des **livrables attendus**. Le respect de ces étapes permet :

- de réduire les risques,
- d'assurer la qualité du produit final,
- de maîtriser les coûts et les délais,
- de garantir la satisfaction du client.

Les principales étapes d'un projet informatique sont :



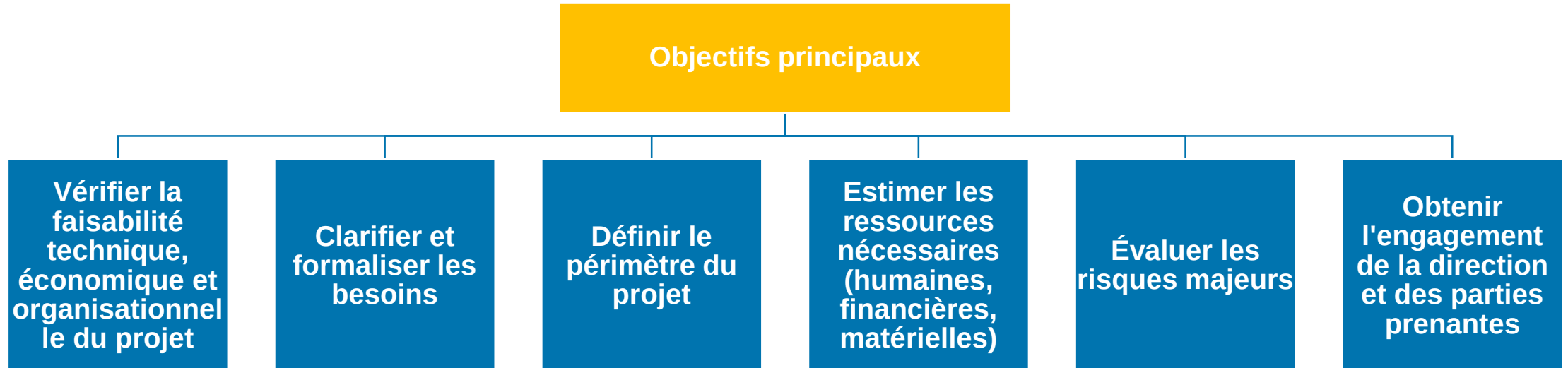
Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.1 Définition et objectifs

L'avant-projet constitue **la phase préparatoire et stratégique du projet**. Elle vise à **transformer une idée ou un besoin exprimé en un projet formalisé et validé** par toutes les parties prenantes. Cette étape est cruciale car elle conditionne la réussite de l'ensemble du projet.



Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.2 Les activités de l'avant-projet

a) L'étude d'opportunité

b) L'étude de faisabilité

c) L'expression des besoins

d) La définition du périmètre

e) L'analyse des risques
préliminaire

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.2 Les activités de l'avant-projet

a) L'étude d'opportunité

Objectif : Décider si le projet doit être lancé.

1. Alignement stratégique

- Cohérence avec la vision et les objectifs de l'organisation
- Contribution à la stratégie globale

2. Analyse de la valeur

- Estimation des bénéfices :
 - Gains de productivité
 - Réduction des coûts
 - Nouveaux revenus
 - Amélioration de l'image
- Comparaison avec les coûts estimés

3. Analyse de l'urgence

- Besoin immédiat ou différable ?

4. Étude des alternatives

- Ne rien faire
- Améliorer l'existant
- Acheter une solution du marché
- Développer en interne

5. Analyse coûts–bénéfices préliminaire

- Coûts : développement, déploiement, formation, maintenance
- Bénéfices : quantitatifs et qualitatifs

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.2 Les activités de l'avant-projet

b) L'étude de faisabilité

Objectif : Vérifier si le projet est réalisable et viable

Faisabilité technique

- Technologies adaptées au besoin
- Disponibilité des compétences (interne, formation, sous-traitance)
- Contraintes majeures : performance, volumétrie, sécurité
- Compatibilité avec l'infrastructure et les systèmes existants

Faisabilité organisationnelle

- Impact sur les processus et les rôles
- Nouveaux besoins en compétences
- Capacité à gérer le changement
- Identification et gestion des résistances

Faisabilité économique

- Estimation détaillée des coûts :
 - Développement (équipe, outils, licences)
 - Déploiement (infrastructure, migration)
 - Formation
 - Maintenance (corrective & évolutive)
- Projection des gains : productivité, réduction des coûts, nouveaux revenus
- Calcul du **ROI** et du **délai de récupération**

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.2 Les activités de l'avant-projet

c) L'expression des besoins

Objectif : Identifier et formaliser les attentes des utilisateurs et parties prenantes.

Recueil des besoins

- Entretiens individuels (utilisateurs clés, experts métier)
- Ateliers collaboratifs
- Observation sur le terrain
- Questionnaires

Hiérarchisation des besoins

- Indispensables
- Importants
- Souhaitables (Nice to have)
- Permet d'arbitrer en cas de contraintes de délais ou de budget

Formalisation des besoins

Besoins fonctionnels

- Fonctionnalités
- Traitements
- Règles de gestion

Besoins non fonctionnels

- Performance (temps de réponse, débit)
- Sécurité (authentification, confidentialité)
- Ergonomie & accessibilité
- Disponibilité
- Maintenabilité

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.2 Les activités de l'avant-projet

d) La définition du périmètre

Objectif : Délimiter clairement le champ du projet.

Éléments inclus

- Fonctionnalités à développer
- Utilisateurs concernés (profils, nombre, localisation)
- Systèmes interfacés
- Limites organisationnelles et géographiques
 - Sites, départements, pays

Éléments exclus

- Fonctionnalités hors projet
- Usages non couverts

Enjeux du périmètre

- Clarifier les attentes
- Éviter les malentendus
- Éviter le dépassements de coûts
- Éviter les retards de planning

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.2 Les activités de l'avant-projet

e) L'analyse des risques préliminaire

Objectif : Identifier et anticiper les menaces pouvant compromettre le projet.

Typologie des risques

Risques techniques

- Complexité élevée
- Technologies nouvelles ou immatures
- Dépendances externes
- Contraintes de performance

Risques organisationnels

- Ressources clés limitées
- Résistance au changement
- Conflits inter-départements
- Instabilité organisationnelle

Risques liés aux parties prenantes

- Faible implication du sponsor ou des utilisateurs
- Attentes irréalistes
- Désaccords sur les priorités

Risques externes

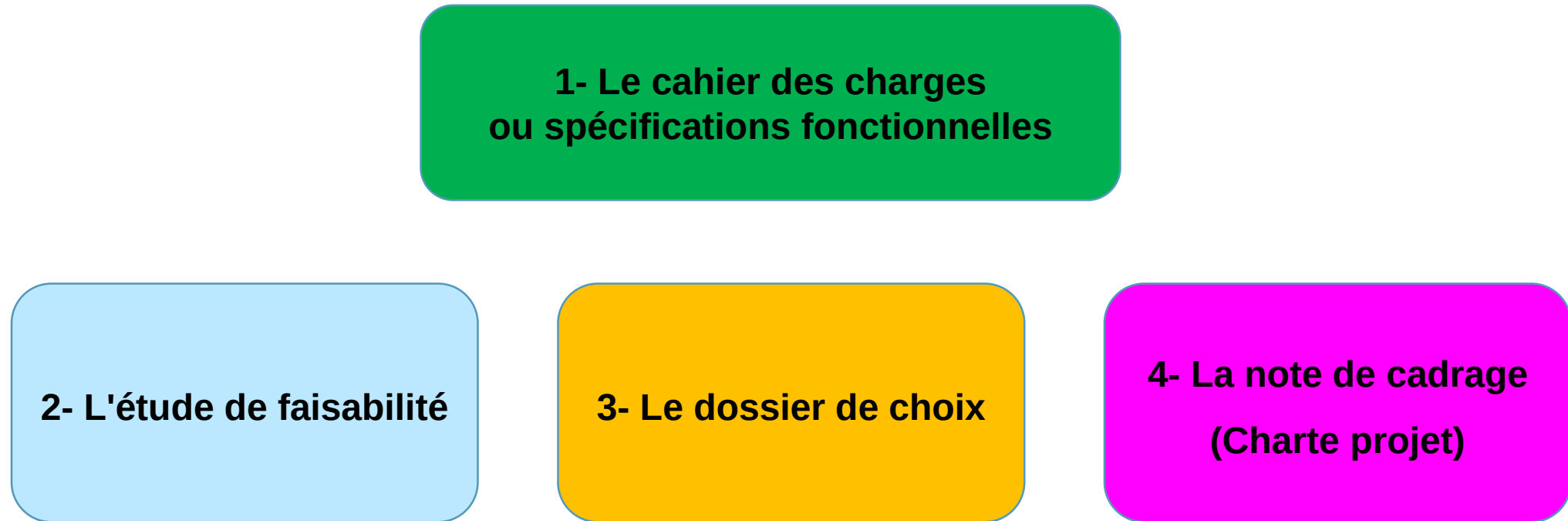
- Évolutions réglementaires
- Dépendance aux fournisseurs
- Contraintes budgétaires ou politiques

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.3 Les livrables de l'avant-projet



Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.3 Les livrables de l'avant-projet

Le cahier des charges ou spécifications fonctionnelles

Document contractuel qui définit les besoins, les objectifs du projet, les contraintes, ainsi que les critères d'acceptation du système à réaliser.

L'étude de faisabilité

Document d'analyse qui évalue les scénarios possibles, leurs coûts, leurs risques, et propose la solution la plus adaptée au projet.

Le dossier de choix

Document de décision qui propose les solutions techniques envisagées, compare les alternatives (développement interne, progiciel, externalisation), justifie le choix retenu, et présente le planning et le budget prévisionnels.

La note de cadrage ou charte projet

Document de cadrage qui définit le cadre du projet : objectifs, périmètre, contraintes, parties prenantes, responsabilités, et organisation du projet.

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.3 Les livrables de l'avant-projet

Cahier des charges / Spécifications fonctionnelles

- Document **contractuel** décrivant les besoins
- **Contexte & objectifs** : origine du projet, objectifs métier, bénéfices attendus
- **Besoins fonctionnels** : cas d'utilisation, scénarios, règles de gestion
- **Besoins non fonctionnels** : performance, sécurité, ergonomie, disponibilité
- **Contraintes** :
 - Techniques (technologies, compatibilité)
 - Organisationnelles (processus, normes internes)
 - Réglementaires (RGPD, normes sectorielles)
 - Délais & budget
- **Interfaces** : systèmes existants, flux de données, protocoles
- **Critères d'acceptation** : mesurables et vérifiables

Étude de faisabilité

- Analyse de **plusieurs scénarios** :
 - Développement interne
 - Progiciel du marché
 - Externalisation / SaaS
 - Approche hybride
- **Évaluation des coûts** :
 - Développement / acquisition
 - Infrastructure
 - Formation
 - Maintenance
 - Conduite du changement
- **Analyse des risques** et plans de mitigation
- **Recommandation** du scénario le plus adapté

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.3 Les livrables de l'avant-projet

Dossier de choix

- Document d'aide à la **décision**
- Présentation des **solutions envisagées**
- **Comparaison multicritères** :
 - Coût
 - Délai
 - Risque
 - Évolutivité
- Justification du **choix retenu**
- **Planning prévisionnel** (phases & jalons)
- **Budget prévisionnel** par poste

Note de cadrage / Charte projet

- Lancement officiel du projet
- **Objectifs et périmètre** (inclus / exclus)
- **Contraintes** : budget, délai, qualité
- **Parties prenantes** et rôles
- Désignation du **chef de projet**
- **Organisation du projet** :
 - Équipe
 - Comités
 - Processus de décision

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.4 Les acteurs impliqués

Maître d'Ouvrage (MOA)

- ❖ Représente les **utilisateurs**
- ❖ Exprime et formalise les **besoins métier**
- ❖ Définit les **objectifs**
- ❖ **Valide** les livrables de l'avant-projet
- ❖ **Finance** le projet
(Direction générale)

Utilisateurs finaux

- ❖ Participent aux **entretiens et ateliers**
- ❖ Apportent la **connaissance métier**
- ❖ **Valident** les besoins et la solution finale

Experts métier

- ❖ Maîtrise approfondie du **domaine fonctionnel**
- ❖ Formalisent les **règles de gestion**
- ❖ Valident la **pertinence fonctionnelle**

Maître d'Œuvre (MOE)

- ❖ Responsable de la **solution technique**
- ❖ Participe à l'**étude de faisabilité**
- ❖ Propose les **solutions techniques**
- ❖ Estime les **charges et coûts**
(DSI ou prestataire externe)

Direction générale

- ❖ Valide l'**opportunité stratégique**
- ❖ Alloue les **budgets**
- ❖ **Arbitre** les priorités et conflits

Experts techniques

- ❖ Architectes, sécurité, DBA, etc.
- ❖ Analysent la **faisabilité technique**
- ❖ Identifient les **contraintes et risques**
- ❖ Proposent des **orientations techniques**

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.5 Validation et décision de lancement

Décisions possibles

❖ Lancer le projet

- Opportunité confirmée
- Faisabilité validée
- Budget alloué

→ Passage en **phase de conception / réalisation**

❖ Différer le projet

- Contexte défavorable (budget, priorités)
- Projet jugé pertinent à moyen terme

❖ Demander des compléments d'étude

- Incertitudes à lever
 - Scénarios à approfondir
- Prolongation de l'avant-projet

❖ Abandonner le projet

- Faisabilité non démontrée
- ROI insuffisant
- Risques trop élevés

Instance de décision

- ❖ **Comité de pilotage** (ou équivalent)
- ❖ Analyse de **l'ensemble des livrables** de l'avant-projet0

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.5 Validation et décision de lancement

Actions en cas de décision positive

- ☐ **Officialisation du lancement** (note de cadrage / lettre de mission)
- ☐ **Nomination du chef de projet**
- ☐ **Allocation du budget**
- ☐ **Engagement des ressources**
- ☐ **Validation du planning macro**
- ☐ **Constitution de l'équipe projet**

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.6. Durée et importance de l'avant-projet

Durée de l'avant-projet

- ☐ **Petit projet** : quelques jours
 - Projet de courte durée (quelques semaines)
 - ☐ **Projet moyen** : 2 à 6 semaines
 - Projet sur plusieurs mois
 - ☐ **Grand projet** : plusieurs mois
 - Projet complexe, pluriannuel
- ☞ La durée dépend de la **taille**, de la **complexité** et des **enjeux** du projet

Importance stratégique

- ☐ Phase **clé** pour la réussite du projet
- ☐ La phase avant-projet → Réduction possible de **50% des risques d'échec**
- ☐ Évite les erreurs majeures :
 - Besoins mal compris
 - Faisabilité surestimée
 - Risques sous-estimés

Valeur ajoutée de l'avant-projet

- ☐ **Alignement** des parties prenantes
- ☐ **Fondations solides** pour la suite du projet
- ☐ Décision basée sur des **données objectives**

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.1. Vue d'ensemble

La phase de conception et réalisation transforme les besoins exprimés en **solution opérationnelle**. Elle se décompose généralement en deux grandes activités : la **conception** qui définit "comment" le système répondra aux besoins, et la **réalisation** qui construit effectivement le système.

Selon la méthodologie adoptée, ces activités peuvent être séquentielles (d'abord toute la conception, puis toute la réalisation) ou imbriquées (conception et réalisation par itérations successives).

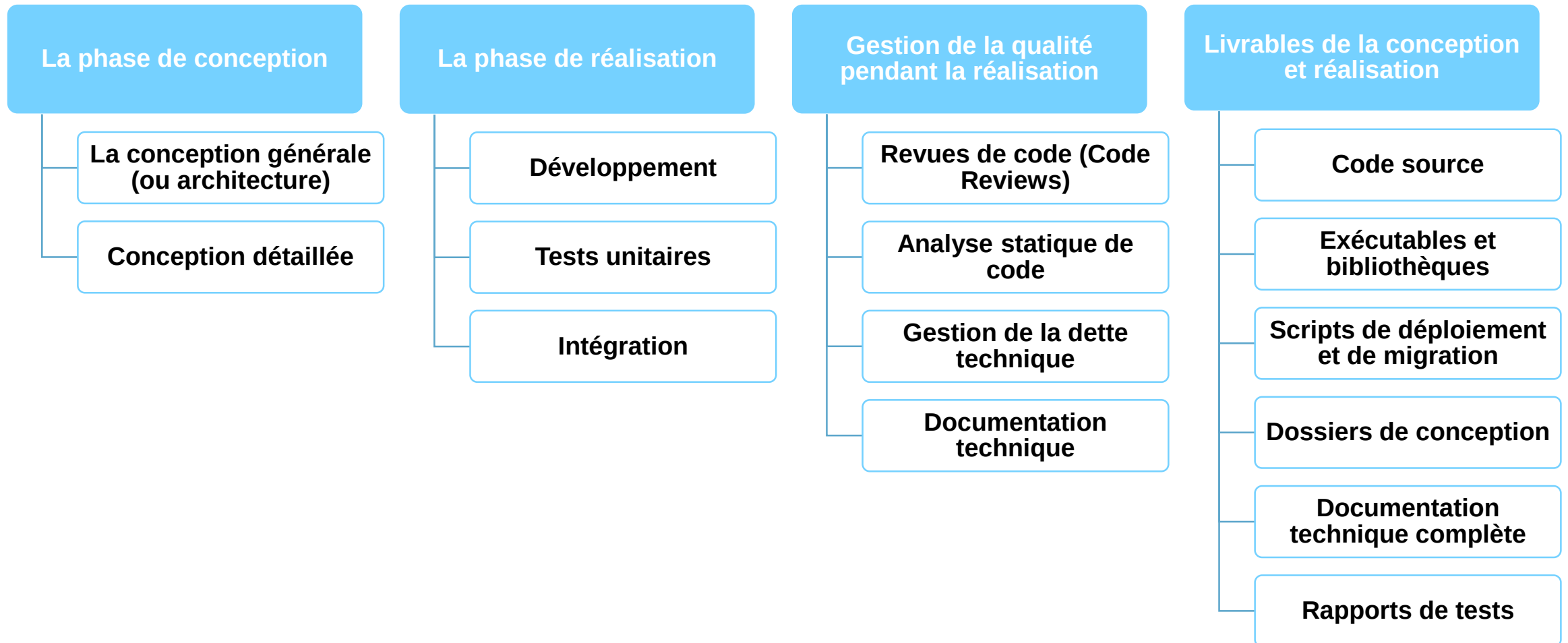
Cette étape correspond au **cœur du projet informatique**.

Elle transforme les besoins exprimés en une **solution informatique concrète**.

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation



Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

Objectif de la conception

- ☐ Traduire les **besoins fonctionnels** → en **spécifications techniques**
- ☐ Préparer une **implémentation maîtrisée**
- ☐ Préparer la phase de réalisation
- ☐ Réduire les risques techniques et fonctionnels

👉 Deux niveaux :

- ☐ **Conception générale**
- ☐ **Conception détaillée**

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

a) La conception générale (ou architecture)

Définition :

C'est la première étape de conception qui définit l'**architecture globale du système**, les grands modules, leurs responsabilités, leurs interactions, l'architecture technique, la sécurité et la disponibilité.

Objectif :

Fournir une **vision globale** du système et orienter les choix techniques et organisationnels, avant de détailler l'implémentation.

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

a) La conception générale (ou architecture)

Exemples de diagrammes utilisés :

- ❖ **Diagramme de composants UML** : montre les modules principaux et leurs interactions
- ❖ **Diagramme de séquence UML (simplifié)** : visualise les échanges de données entre modules pour un scénario principal
- ❖ **Diagramme de couches (layered architecture)** : présente la séparation présentation / métier / persistance / infrastructure
- ❖ **Schéma d'architecture technique / réseau** : topologie réseau, serveurs, flux, zones de sécurité

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

a) Conception générale (Architecture)

Architecture applicative

- Découpage en **modules / composants**
 - Ex. e-commerce : catalogue, panier, commandes, clients, paiement, logistique
- Définition des **interactions entre composants**
 - Échanges de données
 - Protocoles de communication
- **Architecture en couches** :
 - Présentation
 - Métier
 - Persistance
 - Infrastructure
- Modélisation : **diagrammes UML** (composants, séquences)

Architecture technique

- Définition de l'**infrastructure**
- Choix technologiques :
 - Langages & frameworks
 - Bases de données
 - Serveurs & middleware
- **Dimensionnement** :
 - Serveurs, réseau, stockage
 - Cloud vs On-premise
- **Architecture réseau** :
 - Topologie
 - Zones de sécurité (DMZ)
 - Load balancing

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

a) Conception générale (Architecture)

Sécurité (Security by Design)

- **Authentification** : login, SSO, MFA
- **Autorisation** : rôles, permissions
- **Chiffrement** :
 - Données en transit (HTTPS/TLS)
 - Données au repos
- **Traçabilité & logs**
- Mécanismes de protection : Pare-feu, IDS/IPS, Gestion des vulnérabilités

Montée en charge & disponibilité

- **Scalabilité** : Verticale et Horizontale
- Architecture **stateless**
- Mise en cache
- **Haute disponibilité** : Redondance, Failover, Réplication des données

Livrable – Conception générale

- **DCG : Dossier de Conception Générale**
 - Diagrammes d'architecture
 - Choix techniques justifiés
 - Flux & interfaces
 - Principes de sécurité
 - Contraintes de performance

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

b) Conception détaillée

Définition :

C'est l'étape qui **affine chaque composant** identifié en conception générale en spécifiant tous les éléments nécessaires au développement : algorithmes, traitements, structures de données, interfaces, API, règles de gestion et gestion des erreurs.

Objectif :

Fournir aux développeurs toutes les **spécifications précises** permettant l'implémentation correcte du système.

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

b) Conception détaillée

Exemples de diagrammes utilisés :

- ❑ **Diagramme de classes UML** : définit les classes, attributs, méthodes et relations
- ❑ **Diagramme de séquence détaillé**
- ❑ **Modèle conceptuel, logique et physique de données (MCD / MLD / MPD)** : décrit les structures de données et bases de données
- ❑ **Wireframes / maquettes d'interface** : structure des écrans et navigation
- ❑ **Diagramme d'activité UML** : décrit les algorithmes et flux de traitement détaillés

Exemple concret :

Pour le module gestion commandes :

- Classe Commande avec attributs id, client, articles, total et méthodes calculerTotal(), valider()
- Interface utilisateur : écran de validation de commande avec champs et boutons précis

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

b) Conception détaillée

Spécifications des composants

- ❖ Détail des **traitements et algorithmes**
- ❖ Gestion des cas particuliers et erreurs
- ❖ Définition précise des **structures de données**

Interfaces utilisateur (UI/UX)

- ❖ **Wireframes** : structure des écrans
- ❖ **Mockups** : design graphique
- ❖ **Prototypes interactifs**
- ❖ Ergonomie & accessibilité : WCAG, Responsive design

Modélisation des données

- ❖ Bases relationnelles /Bases NoSQL

API et interfaces techniques

- ❖ Spécification des services :
 - Endpoint & méthodes HTTP
 - Paramètres et validations
 - Formats de réponse
 - Gestion des erreurs
- ❖ Protocoles :
 - REST, SOAP
 - Messaging (RabbitMQ, JMS)

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

b) Conception détaillée

Règles de gestion détaillées

- Formalisation **sans ambiguïté**
- Règles conditionnelles claires
- Validations et messages d'erreur

Gestion des erreurs & exceptions

- Erreurs techniques & fonctionnelles
- Timeout & retry
- Mécanismes de fallback
- Journalisation pour le support

Livrable – Conception détaillée

•DCD : Dossier de Conception Détaillée

- Spécifications techniques complètes
- Modèles de données
- Maquettes & ergonomie
- API & protocoles
- Matrice de traçabilité

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

Définition :

La phase de réalisation correspond au **développement effectif de la solution**. Elle inclut l'écriture du code source selon des **standards et bonnes pratiques**, la **gestion des versions**, le **développement itératif et incrémental**, les **tests unitaires** et **l'intégration progressive des composants**. Elle intègre également la gestion de la qualité du code via revues, analyse statique, documentation technique et contrôle de la dette technique.

Objectif :

Produire une **solution fonctionnelle, fiable et maintenable**, respectant les spécifications, prête à être testée et intégrée, tout en assurant la qualité et la traçabilité du code.

La phase de réalisation transforme les spécifications en code fonctionnel et testé, tout en assurant la qualité, la traçabilité et la maintenabilité du système.

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

Activités principales :

1.Développement

- ☐ Organisation du code selon **standards, conventions, modules réutilisables**
- ☐ Respect des **bonnes pratiques** et principes SOLID
- ☐ Utilisation de **design patterns**, documentation du code, et **versioning** (Git, SVN)
- ☐ Développement **itératif et incrémental** avec intégration continue

2.Tests unitaires

- ☐ Vérification de chaque composant isolé : cas normaux et erreurs
- ☐ Automatisation des tests et mesure de la **couverture de code**
- ☐ Frameworks typiques : JUnit, pytest, Jest, NUnit

3.Intégration

- ☐ Assemblage progressif des composants
- ☐ Tests des **interfaces et cohérence des données**
- ☐ Détection et résolution des conflits ou incompatibilités
- ☐ Outils d'intégration continue : Jenkins, GitLab CI, GitHub Actions

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

a) Le développement

Il s'agit de la réalisation du système en respectant des standards de codage, des bonnes pratiques, une organisation modulaire du code et un suivi par version, avec un développement itératif et des livraisons progressives.

b) Les tests unitaires

Les développeurs vérifient chaque composant de façon isolée afin de s'assurer du bon fonctionnement des fonctionnalités, y compris les cas d'erreur, en privilégiant l'automatisation des tests.

c) L'intégration

Les différents composants sont assemblés progressivement pour tester leurs interactions, garantir la cohérence du système et détecter les incompatibilités, souvent à l'aide d'outils d'intégration continue.

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

Objectifs de la réalisation

- ☐ Transformer la **conception** en **code exécutable**
- ☐ Construire la solution informatique
- ☐ Garantir la **qualité**, la **maintenabilité** et la **robustesse**
- ☐ Livrer des incréments **fonctionnels et testés**

Activités de réalisation

- ☐ Développement des composants logiciels
- ☐ Configuration des systèmes
- ☐ Intégration des modules
- ☐ Gestion des versions
- ☐ Documentation technique

Livrables de la réalisation

- ☐ Code source
- ☐ Versions intermédiaires
- ☐ Documentation technique

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

a) Organisation du développement

Standards de développement

- ❑ Conventions de **nommage** (variables, classes, tables...)
- ❑ Règles de **structuration du code**
 - Organisation des dossiers
 - Fonctions/méthodes de taille maîtrisée
 - Séparation des responsabilités
- ❑ **Templates et patterns** pour un code homogène

Bonnes pratiques de développement

- ❑ Application des principes **SOLID**
- ❑ Utilisation de **design patterns** :
 - MVC, Factory, Singleton
 - Observer, Strategy
- ❑ **Documentation du code** :
 - Commentaires utiles
 - JavaDoc / Sphinx
 - README par module

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

a) Organisation du développement

Gestion de versions

- ☐ Outil de versioning : **Git / SVN**
- ☐ Traçabilité complète des changements
- ☐ Stratégie de branches :
 - **Main / Master** : production
 - **Feature branches** : développements
 - **Release branches** : préparation livraison
 - **Hotfix** : corrections urgentes

Développement itératif & incrémental

- ☐ Développement par **itérations courtes**
 - Sprints (2–4 semaines)
- ☐ Livraison progressive des **fonctionnalités prioritaires**
- ☐ Chaque itération produit un **incrément livrable**
- ☐ Avantages :
 - Feedback rapide
 - Adaptation continue
 - Réduction des risques

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

b) Les tests unitaires

Principe

- ☐ Test de chaque **unité de code** isolément
- ☐ Simulation des dépendances (mocks, stubs)

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

b) Les tests unitaires

Cas de test

- ☐ **Cas nominaux** (fonctionnement normal)
- ☐ **Cas limites** (valeurs extrêmes)
- ☐ **Cas d'erreur** (données invalides, exceptions)

Automatisation

- ☐ Frameworks :
 - JUnit (Java)
 - pytest (Python)
 - Jest (JavaScript)
 - NUnit (C#), PHPUnit (PHP)
- ☐ Exécution fréquente (idéalement **à chaque commit**)

Couverture de code

- ☐ Mesure :
 - Lignes exécutées
 - Branches testées
 - Fonctions couvertes
- ☐ Objectif courant : **70–80 %**
- ☐ ⚠ Couverture ≠ qualité → importance des assertions

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

c) L'intégration

Approches d'intégration

- **Intégration Continue (CI) :**
 - Build automatique
 - Exécution des tests
 - Notification des résultats
- Outils : Jenkins, GitLab CI, GitHub Actions, CircleCI
- Intégration progressive :
 - Bottom-up
 - Top-down
 - Par sous-systèmes

Tests d'intégration

- Vérification du fonctionnement **global**
- Cohérence des échanges entre composants
- Tests avec :
 - Bases de données
 - Services externes
 - Systèmes legacy

Gestion des conflits

- Conflits possibles :
 - Modifications concurrentes
 - Dépendances circulaires
 - Versions incompatibles
- Intégration fréquente :
 - Détection rapide
 - Résolution facilitée

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.4. Gestion de la qualité pendant la réalisation

La **gestion de la qualité pendant la réalisation** vise à garantir que le logiciel développé respecte les standards, est fiable, maintenable et conforme aux spécifications. Cette étape accompagne le développement en mettant en place des pratiques de contrôle et d'amélioration continue. Elle inclut la **revue du code, l'analyse statique, la maîtrise de la dette technique et la documentation**, afin de prévenir les erreurs, faciliter la maintenance et assurer une livraison de qualité aux utilisateurs finaux.

Objectifs

- Garantir la **qualité du code**
- Réduire les **bugs** et risques de sécurité
- Assurer la **maintenabilité** et l'évolutivité

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.4. Gestion de la qualité pendant la réalisation

Revue de code (Code Reviews)

- ☐ Examen du code par les **pairs**
- ☐ Vérification :
 - Respect des standards
 - Détection de bugs et failles de sécurité
 - Amélioration : performance, lisibilité, maintenabilité
- ☐ **Partage des connaissances**

Formats

- ☐ Formelles : inspections structurées
- ☐ Informelles : pair programming, pull/merge requests

Outils

- ☐ GitHub Pull Requests
- ☐ GitLab Merge Requests
- ☐ Gerrit

Analyse statique de code

- ☐ Analyse automatique **sans exécution**
- ☐ Détection de :
 - Bugs potentiels
 - Failles de sécurité (SQL injection, XSS)
 - Complexité excessive
 - Code mort ou dupliqué
 - Violations des standards

Outils courants

- ☐ SonarQube (multi-langages)
- ☐ ESLint (JavaScript)
- ☐ Pylint (Python)
- ☐ RuboCop (Ruby)
- ☞ Intégration possible dans le **pipeline CI**
 - Blocage du code de mauvaise qualité

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.4. Gestion de la qualité pendant la réalisation

Gestion de la dette technique

❑ Définition :

- Raccourcis techniques à court terme
- Impact négatif à long terme

❑ Exemples :

- Code non optimal
- Absence de tests
- Documentation insuffisante
- Architecture peu évolutive

Conséquences

- ❑ Maintenance plus difficile
- ❑ Augmentation des bugs
- ❑ Ralentissement des évolutions

Gestion

- ❑ Identifier et mesurer (ex. SonarQube)
- ❑ Documenter (backlog)
- ❑ Prioriser (critique vs acceptable)
- ❑ Réduire progressivement (refactoring)
- ⚠ Objectif : **dette maîtrisée**, pas zéro dette

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.4. Gestion de la qualité pendant la réalisation

Documentation technique

- **DAT** : Dossier d'Architecture Technique
- **Guide d'installation** : prérequis & procédures
- **Documentation API** :
 - Swagger / OpenAPI
- **Guides de configuration**
- **Procédures de déploiement**
- ☞ Essentielle pour :
 - Maintenance
 - Onboarding des développeurs
 - Support et exploitation

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.5. Livrables de la conception et réalisation

Les principaux livrables de cette phase sont le **code source** versionné et documenté dans le repository (Git, SVN), les **exécutables et bibliothèques** compilés et packagés, les **scripts de déploiement** et de migration de base de données, les **dossiers de conception** (DCG, DCD) mis à jour si nécessaire, la **documentation technique** complète et à jour, et les **rapports de tests unitaires** et d'intégration avec les taux de couverture.

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.1 Objectifs de la phase de validation

La phase de validation consiste à **vérifier que le système développé répond aux exigences définies dans le cahier des charges**. C'est la dernière étape avant la mise en production, destinée à s'assurer que le système est prêt à être utilisé sans risque majeur.

Objectif :

- ☐ Vérifier la **conformité** du système au **cahier des charges**
- ☐ Confirmer la **conformité du système** aux spécifications fonctionnelles et techniques.
- ☐ Détecter et corriger les **bugs ou anomalies critiques** avant la mise en production.
- ☐ Vérifier les **performances, la sécurité, la compatibilité et l'ergonomie**
- ☐ S'assurer du **bon fonctionnement** dans toutes les situations prévues
- ☐ Tester le comportement face à des **cas imprévus**
- ☐ Vérifier la **conformité aux normes et réglementations**
- ☐ Garantir que la **documentation et les procédures** sont complètes et fiables.
- ☐ Obtenir l'**approbation formelle** des responsables pour passer à la phase de recette et au déploiement.

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.1 Objectifs de la phase de validation

Pourquoi la validation est critique ?

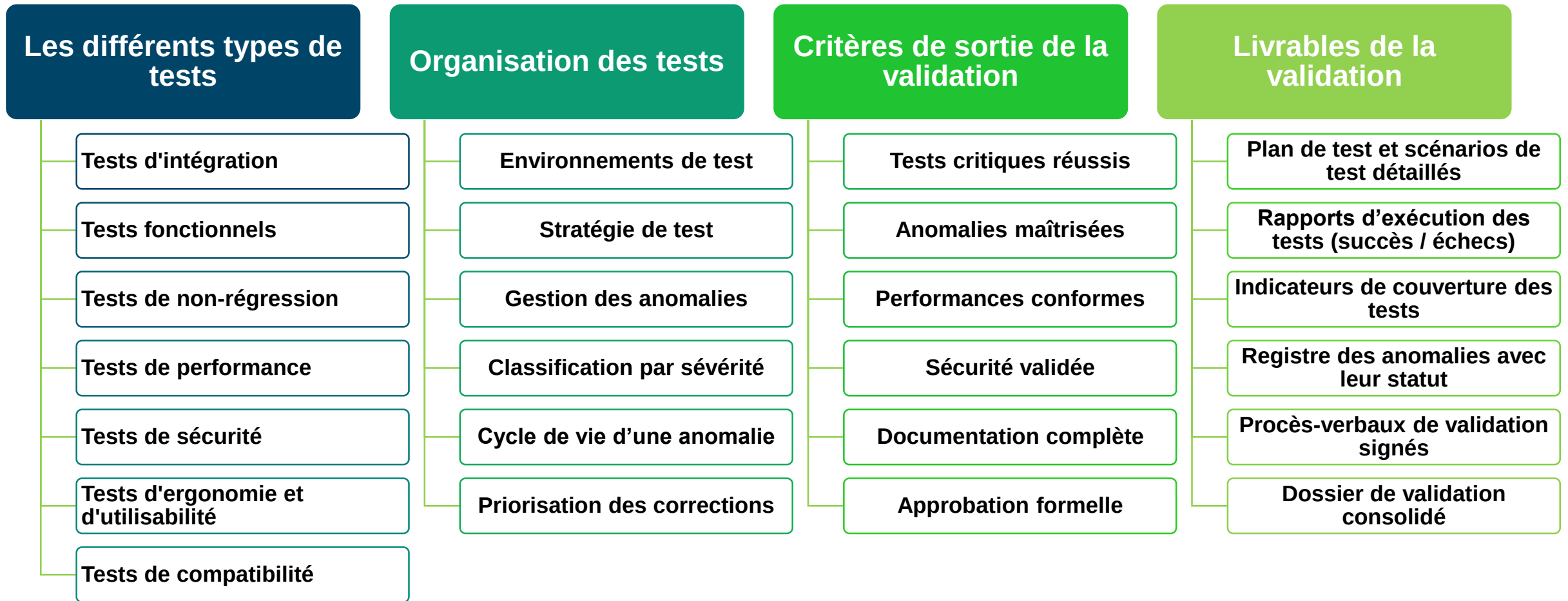
- **Dernière barrière** avant la mise en production
- Les anomalies non détectées :
 - Impactent directement les **utilisateurs**
 - Peuvent générer des **risques métier, financiers ou juridiques**
- Conditionne la **qualité perçue** du système

☞ Un système validé est un système **fiable, conforme et déployable sans risque majeur**

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

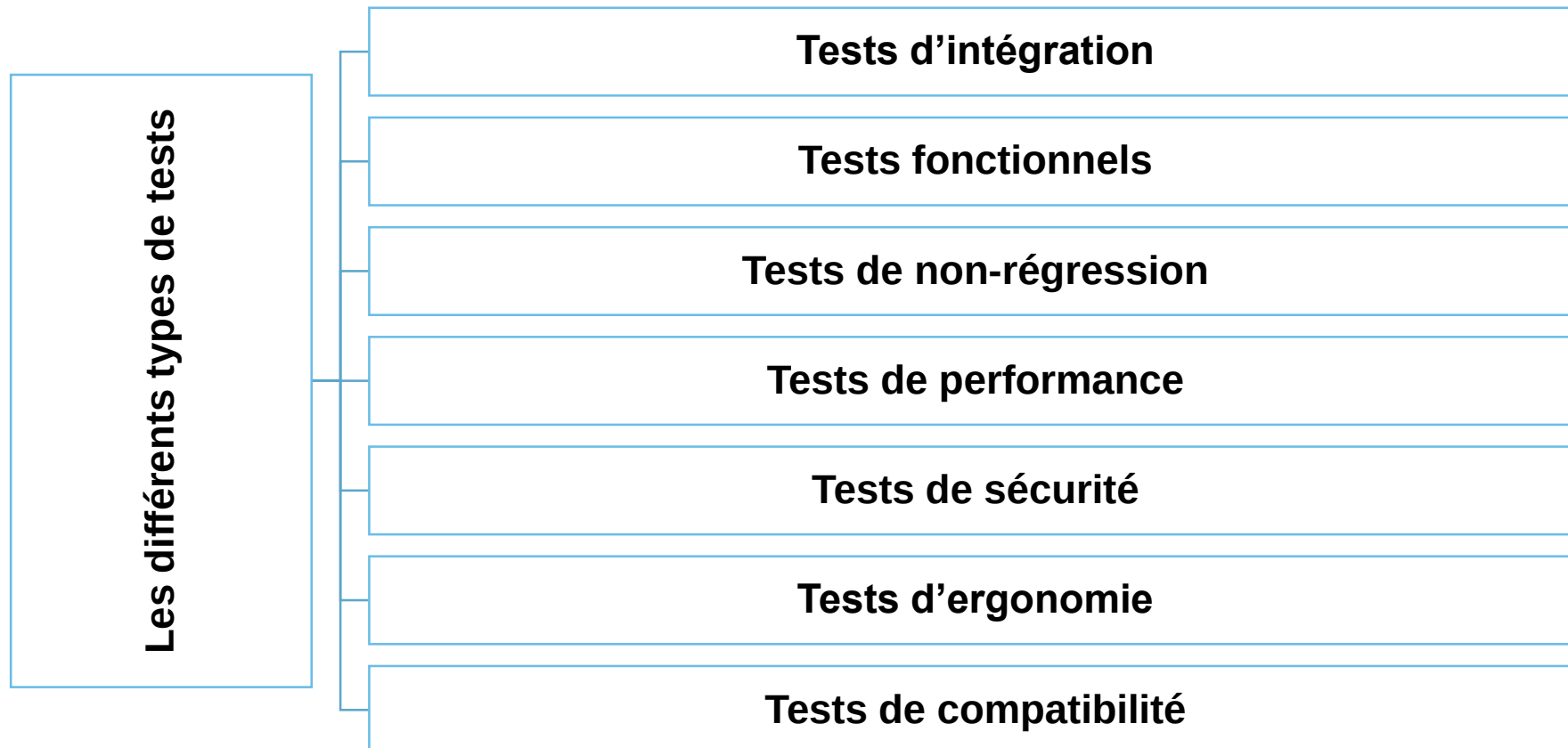


Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.2 Les différents types de tests



Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.2. Les différents types de tests

a) Tests d'intégration

Objectif

Vérifier que les **modules fonctionnent correctement ensemble**

Périmètre

- ❑ Intégration **verticale** : Présentation → Métier → Données
- ❑ Intégration **horizontale** : modules de même niveau
- ❑ Communication avec :
 - Bases de données
 - Services web tiers
 - Systèmes legacy
 - Services d'infrastructure (LDAP, mail...)

Approches

- ❑ **Big Bang** : tout intégrer en une seule fois (simple mais risqué)
- ❑ **Incrémentale** : Top-down, Bottom-up, Par sous-systèmes fonctionnels

b) Tests fonctionnels

Objectif

Vérifier que les **fonctionnalités du cahier des charges** sont respectées

Base des tests

- ❑ Cahier des charges & spécifications
- ❑ Traçabilité : **Exigence** ↔ **Cas de test**

Scénarios

- ❑ Cas nominaux
- ❑ Cas limites
- ❑ Cas d'erreur
- ❑ Chemins alternatifs

Vérifications

- ❑ Règles de gestion métier
- ❑ Conformité fonctionnelle
- ❑ Cohérence des données

Exécution

- ❑ Manuelle ou automatisée
- ❑ Outils : Selenium, Cypress, Katalon, UFT

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.2. Les différents types de tests

c) Tests de non-régression

Objectif

Garantir que les fonctionnalités existantes **continuent de fonctionner**

Principe

- ❑ Rejouer une **suite de tests critiques** après chaque modification
- ❑ Ajouter un test à chaque bug corrigé

Automatisation

- ❑ Fortement recommandée

d) Tests de performance

Objectif

Vérifier le comportement du système **sous charge**

Types

- ❑ **Tests de charge** : charge normale
- ❑ **Tests de stress** : au-delà des limites
- ❑ **Tests d'endurance** : longue durée
- ❑ **Tests de scalabilité** : montée en charge

Métriques

- ❑ Temps de réponse (moyen, P90, P95...)
- ❑ Débit (transactions/sec)
- ❑ Utilisation des ressources
- ❑ Taux d'erreur

Outils : JMeter, Gatling, LoadRunner, k6

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.2. Les différents types de tests

e) Tests de sécurité

Objectif

Vérifier que le système est **résistant aux attaques**

Domaines testés

- ❑ Authentification & autorisation
- ❑ Attaques courantes : SQL Injection, XSS, CSRF, Command injection
- ❑ Chiffrement (en transit / au repos)
- ❑ Conformité réglementaire : RGPD, PCI-DSS (si applicable)

Méthodes

- ❑ Tests de pénétration (pentests)
- ❑ Audits de sécurité
- ❑ Scans automatisés (OWASP ZAP, Nessus...)

f) Tests d'ergonomie et d'utilisabilité

Objectif

Évaluer l'**expérience utilisateur (UX)**

Aspects testés

- ❑ Facilité de prise en main
- ❑ Intuitivité
- ❑ Cohérence graphique
- ❑ Accessibilité (WCAG)

Méthodes

- ❑ Tests utilisateurs
- ❑ Tests d'utilisabilité formels
- ❑ Tests A/B

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.2. Les différents types de tests

g) Tests de compatibilité

Objectif

Garantir le bon fonctionnement dans **différents environnements**

Tests réalisés sur

- ❑ Navigateurs : Chrome, Firefox, Edge, Safari
- ❑ Systèmes : Windows, Linux, macOS
- ❑ Appareils : PC, mobile, tablette
- ❑ Résolutions & orientations

Versions logicielles (OS, langages, DB, navigateurs)

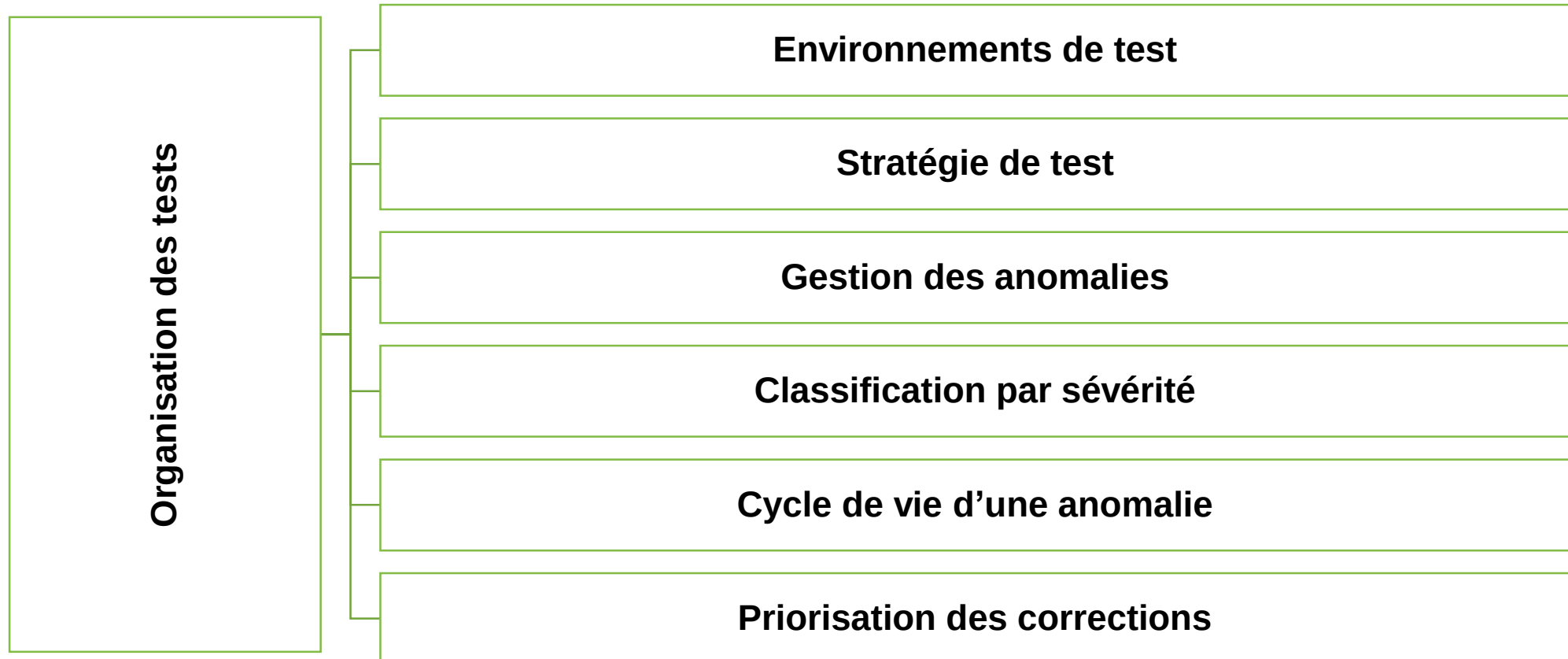
☞ La validation repose sur une **combinaison complémentaire de tests** pour garantir qualité, sécurité, performance et satisfaction utilisateur.

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.3. Organisation des tests



Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.3. Organisation des tests

Environnements de test

On distingue généralement plusieurs environnements avec des rôles spécifiques.

L'environnement de développement (DEV) est utilisé par les développeurs pour les tests unitaires et l'intégration locale. Il est instable et change fréquemment.

L'environnement d'intégration (INT) sert aux tests d'intégration avec un code plus stable que DEV. L'intégration continue y déploie automatiquement.

L'environnement de recette (UAT - User Acceptance Testing) permet les tests fonctionnels et la validation par les utilisateurs. Il est le plus proche de la production en termes de configuration.

L'environnement de pré-production (PREPROD) réplique exactement la production pour les tests finaux avant déploiement. Il permet de tester les procédures de déploiement.

L'environnement de production (PROD) est l'environnement réel où les utilisateurs finaux travaillent.

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.3. Organisation des tests

Stratégie de test

Un **plan de test** définit :

- Les objectifs des tests
- Le périmètre (inclus / exclu)
- Les ressources (équipe, environnements, données, outils)
- Le planning et les jalons
- Les critères de succès (couverture, seuil de bugs critiques)

Les **cas de test** décrivent :

- Les préconditions
- Les étapes à exécuter
- Les données d'entrée
- Les résultats attendus
- Les critères de validation

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.3. Organisation des tests

Gestion des anomalies

Les anomalies sont enregistrées dans des outils de suivi tels que : Jira, Mantis, Redmine ou Bugzilla.

Chaque anomalie comprend :

- Description du problème
- Étapes de reproduction
- Preuves (captures, logs)
- Sévérité et priorité

Classification par sévérité

- **Bloquante** : empêche l'usage ou provoque une perte de données
- **Majeure** : impact important avec solution de contournement
- **Mineure** : gêne limitée
- **Cosmétique** : défaut visuel sans impact fonctionnel

Cycle de vie d'une anomalie

États possibles :

- Nouvelle
- Assignée
- En cours de correction
- Résolue
- Vérifiée
- Fermée
- Rouverte

Priorisation des corrections

- Basée sur la **criticité métier** et l'**urgence**
- Les anomalies **bloquantes** sont traitées en **priorité**, même si cela retarde d'autres développements

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.4. Critères de sortie de la validation

Critères de sortie de la validation

Tests critiques réussis

Anomalies maîtrisées

Performances conformes

Sécurité validée

Documentation complète

Approbation formelle

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.4. Critères de sortie de la validation

La phase de validation est considérée comme terminée lorsque **toutes les conditions suivantes sont réunies** :

✓ Tests critiques réussis

- Toutes les fonctionnalités critiques et les parcours utilisateurs principaux sont validés
- Le taux de couverture minimal des exigences est atteint (ex. $\geq 80\%$)

✓ Anomalies maîtrisées

- Aucune anomalie bloquante en attente
- Les anomalies majeures sont corrigées ou formellement acceptées avec un plan d'action
- Les anomalies mineures acceptées sont documentées et planifiées

✓ Performances conformes

- Les tests confirment le respect des objectifs :
 - Temps de réponse
 - Capacité de charge
 - Stabilité du système

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.4. Critères de sortie de la validation

✓ Sécurité validée

- Aucune vulnérabilité critique détectée
- Respect des exigences de sécurité et de conformité réglementaire (ex. protection des données)

✓ Documentation complète

- Documentation utilisateur finalisée et validée
- Documentation technique à jour
- Procédures d'exploitation prêtes

✓ Approbation formelle

- Validation par le responsable qualité ou l'équipe de test
- Accord du maître d'ouvrage pour passer à la phase de recette

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.5. Livrables de la validation

Les principaux livrables de cette phase sont :

- **Plan de test et scénarios de test détaillés**
- **Rapports d'exécution des tests** (succès / échecs)
- **Indicateurs de couverture des tests**
- **Registre des anomalies** avec leur statut (corrigée, acceptée, reportée)
- **Procès-verbaux de validation** signés par les parties prenantes
- **Dossier de validation** consolidé concluant sur l'aptitude du système à passer en recette

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.1. Définition et distinction

La recette est la phase durant laquelle le **maître d'ouvrage** (ou ses représentants) vérifie que le système livré correspond bien à ses attentes et au cahier des charges initial. C'est une validation **métier** effectuée dans un environnement dédié (environnement de recette) qui reproduit au mieux les conditions réelles d'utilisation.

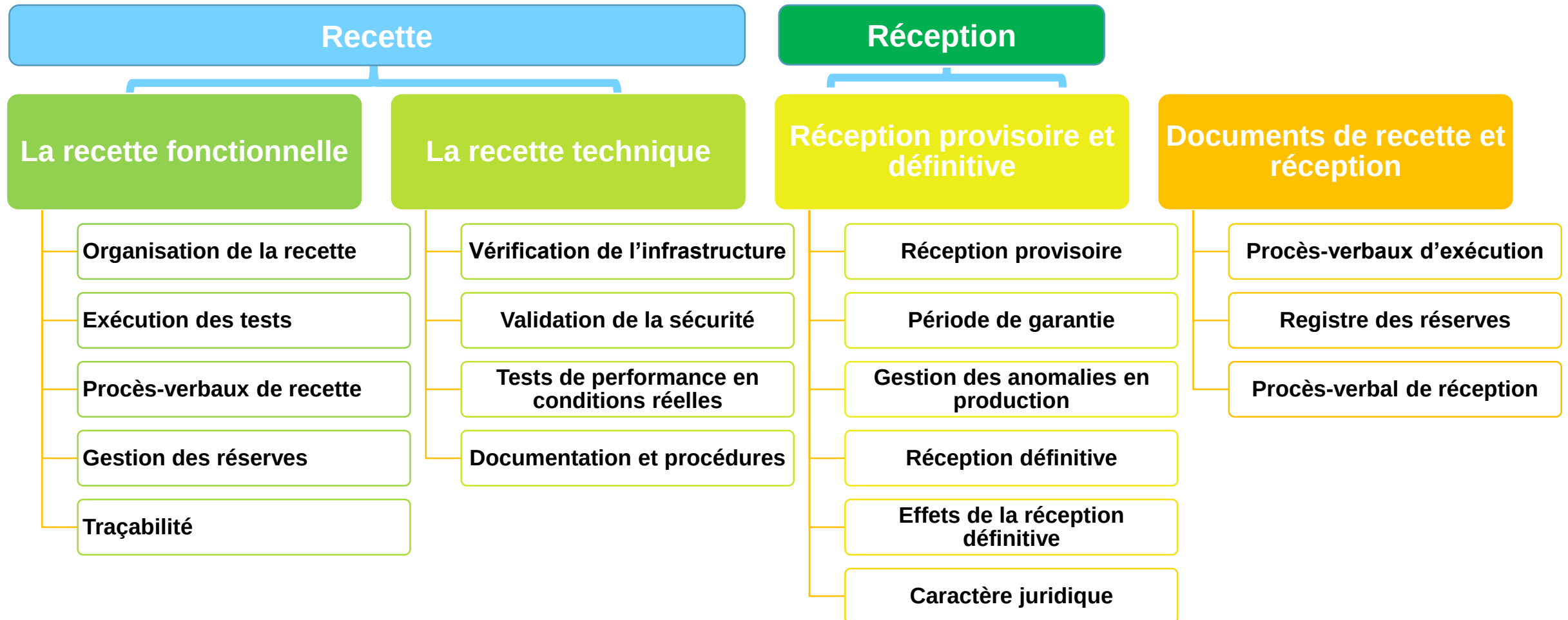
La réception est l'**acte formel** par lequel le maître d'ouvrage accepte officiellement le système et autorise sa mise en production. Elle marque le **transfert de responsabilité** du maître d'œuvre (qui a développé) vers le maître d'ouvrage (qui va exploiter). C'est un jalon contractuel majeur.

La distinction est importante : la recette est un **processus** (ensemble d'activités de vérification), tandis que la réception est un **événement** (décision formalisée par un procès-verbal).

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception



Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.2. La recette fonctionnelle

Objectifs

La recette fonctionnelle vise à :

- Vérifier que **toutes les fonctionnalités du cahier des charges** sont présentes et opérationnelles
- Valider l'adéquation du système aux **besoins métier réels**
- Confirmer que les **processus de l'organisation** sont correctement supportés
- Évaluer l'**ergonomie** et la facilité d'utilisation pour les utilisateurs finaux

Organisation de la recette

Le **maître d'ouvrage (MOA)** :

- Prépare les scénarios de recette à partir du cahier des charges et des processus métier
- Sélectionne et mobilise les **utilisateurs clés (key users)** représentant les différents profils
- Fournit des **jeux de données réalistes** (volume, variété, complexité proches du réel)

Exécution des tests

- Tests réalisés par les **utilisateurs finaux** ou leurs représentants
- Conditions proches du **fonctionnement réel**
- Utilisation de **données métier significatives**
- Chaque scénario précise :
 - Les étapes à exécuter
 - Les résultats attendus

Procès-verbaux de recette

Chaque test fait l'objet d'un procès-verbal avec un statut :

- **Succès** : résultat conforme aux attentes
- **Échec** : anomalie détectée, non-conformité
- **Réserves** : test globalement conforme avec écarts mineurs

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.2. La recette fonctionnelle

Gestion des réserves

Les écarts identifiés sont classifiés :

Réserves majeures (bloquantes)	Réserves mineures	Demandes d'évolution
▪ Empêchent l'utilisation du système pour des processus critiques ▪ Bloquent la réception ▪ Correction obligatoire avant acceptation	▪ Gênantes mais non bloquantes ▪ Contournements possibles ▪ N'empêchent pas la réception ▪ Correction planifiée dans un délai convenu	▪ Améliorations hors périmètre initial ▪ Non bloquantes ▪ Intégrées au backlog de maintenance évolutive

Traçabilité

- Chaque exigence du cahier des charges doit être liée à un ou plusieurs tests de recette
- Une **matrice de traçabilité** permet de :
 - Vérifier que toutes les exigences ont été testées
 - Garantir l'exhaustivité de la recette

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.3. La recette technique

Définition

- Menée **en parallèle de la recette fonctionnelle**
- Réalisée par :
 - Équipes d'**exploitation**
 - **Architectes techniques**
- Objectif : valider que le système est **techniquement prêt pour la production**

Validation de la sécurité

- **Contrôles d'accès**
 - Authentification fonctionnelle
 - Autorisations correctes
 - Ségrégation des données entre utilisateurs
- **Chiffrement**
 - HTTPS actif (certificats valides)
 - Données sensibles chiffrées
 - Mots de passe stockés de façon sécurisée (hash + salt)
- **Traçabilité & conformité**
 - Logs d'accès et d'actions critiques
 - Respect des exigences **RGPD**

Tests de performance en conditions réelles

- Tests avec :
 - Volumes de données réels ou représentatifs
 - Nombre réel d'utilisateurs simultanés
- Mesure des **temps de réponse réels**
- Validation de la stabilité sous charge

Vérification de l'infrastructure

- **Serveurs**
 - Dimensionnement adéquat
 - Configuration correcte
 - Sécurisation validée
- **Réseau**
 - Bande passante suffisante
 - Segmentation sécurisée
- **Sauvegardes**
 - Sauvegardes opérationnelles
 - Procédures de **restauration testées**
- **Supervision**
 - Outils de monitoring actifs
 - Alertes correctement configurées

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.3. La recette technique

Documentation et procédures

- Vérification de la documentation :
 - ✓ Architecture technique
 - ✓ Procédures d'exploitation
 - ✓ Guides de dépannage
 - ✓ Contacts d'escalade
- Validation des **procédures de déploiement**
 - ✓ Tests réels en pré-production
- Test des **procédures de rollback**
 - ✓ Capacité de retour arrière en cas d'incident

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.4. Réception provisoire et définitive

Réception provisoire (recette avec réserves)

Principe

- Prononcée si :
 - Réserves **mineures**
 - **Aucun problème bloquant**
- Autorise la **mise en production**
- Réserves à lever dans un **délai convenu**

Période de garantie

- Entre réception provisoire et définitive
- Responsabilités du **maître d'œuvre** :
 - Correction des anomalies résiduelles
 - Support renforcé
 - Ajustements mineurs (sans coût supplémentaire)
- Rôle du **maître d'ouvrage** :
 - Suivi de la levée des réserves
 - Validation des corrections

Gestion des anomalies en production

- Les nouvelles anomalies :
 - Traitées dans le cadre de la **garantie**
- Validation progressive jusqu'à levée complète des réserves

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.4. Réception provisoire et définitive

Réception définitive

Conditions

- ☐ Toutes les réserves **levées**
- ☐ Période d'exploitation **satisfaisante**

Effets de la réception définitive

- ☐ **Clôture officielle du projet**
- ☐ Passage du **mode projet** au **mode maintenance**
- ☐ **Païement final** du maître d'œuvre (si sous-traitance)
- ☐ **Transfert complet de responsabilité** au maître d'ouvrage

Caractère juridique

- ☐ Réception généralement **irrévocable**
- ☐ Plus de réclamation possible :
 - Sauf cas exceptionnels de **vices cachés graves**

☞ La réception provisoire sécurise la mise en production, la réception définitive marque la fin du projet et le début de l'exploitation durable.

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.5. Documents de recette et réception

Plan de recette

- ☐ Décrit le **déroulement global de la recette**
- ☐ Contenu :
 - Scénarios de test à exécuter
 - **Planning** (dates, durée)
 - **Responsabilités** (qui teste quoi)
 - Jeux de données
 - **Critères d'acceptation**
- ☐ Sert de **référence officielle** pour la recette

Fiches de test de recette

- ☐ Une fiche par **scénario de test**
- ☐ Contenu :
 - Identifiant unique
 - Objectif du test
 - Préconditions
 - Étapes détaillées
 - Résultats attendus
 - Résultats réels & observations
- ☐ Assurent la **traçabilité** des tests

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.5. Documents de recette et réception

Procès-verbaux d'exécution

- ☐ Rédigés après chaque test exécuté
- ☐ Contiennent :
 - Date d'exécution
 - Testeur
 - Résultat :
 - Succès
 - Échec
 - Réserve
 - Anomalies détectées
 - Signatures des parties
- ☐ Preuve formelle de l'**exécution des tests**

☐ Registre des réserves

- ☐ Centralise toutes les **réserves émises**
- ☐ Pour chaque réserve :
 - Description
 - Classification (majeure / mineure)
 - Plan de levée (qui, quand, comment)
 - Statut (en cours / levée / acceptée)
- ☐ Outil clé de **suivi post-recette**

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.5. Documents de recette et réception

Procès-verbal de réception

- Document **contractuel officiel**
- Signé par le **maître d'ouvrage**
- Contenu :
 - Conformité globale du système
 - Liste des réserves éventuelles
 - Type de réception : Provisoire, Définitive
 - Délais de levée des réserves
- Déclenche : Paiements, Garantie, Responsabilités contractuelles

☞ Les documents de recette sécurisent juridiquement et opérationnellement la mise en production et la réception du projet.

Chapitre 3 : Les étapes projet

3.5. La Livraison

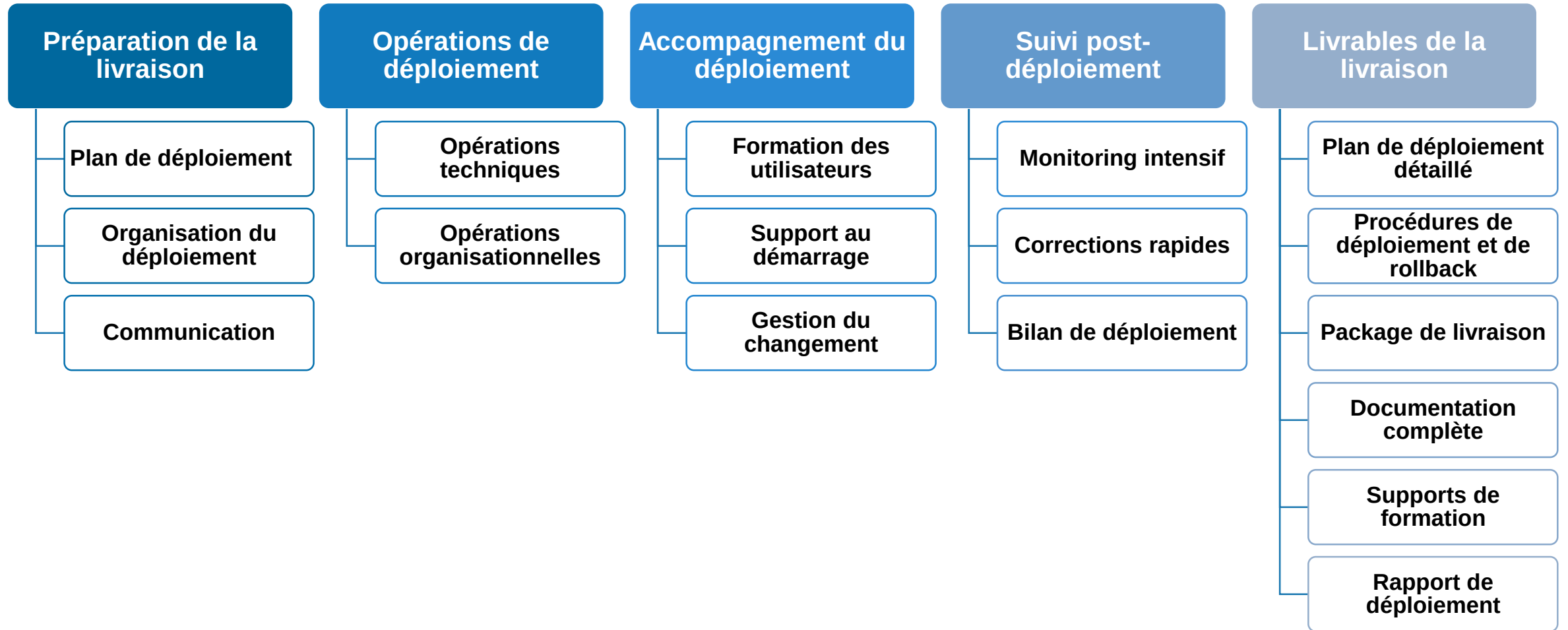
3.5. La Livraison

La livraison correspond au **passage effectif du système vers la production** et à son appropriation par les utilisateurs. Elle ne se limite pas à **un acte technique** : elle inclut **organisation, communication, formation et accompagnement.**

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison



Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.1. Préparation de la livraison

Finalisation des livrables

Avant toute mise en production, tous les éléments doivent être **complets, validés et à jour** :

- ☐ **Code** finalisé, documenté et versionné (release)
- ☐ **Exécutables** compilés, packagés et testés
- ☐ **Documentation complète** :
 - Utilisateur (manuels, guides rapides, FAQ)
 - Technique (architecture, API, code)
 - Exploitation (installation, configuration, supervision, dépannage)
- ☐ **Scripts prêts et testés** :
 - Installation et configuration
 - Migration de bases de données
 - Rollback (retour arrière)
 - Vérifications post-déploiement
- ☐ **Données préparées** :
 - Paramétrage
 - Données de test
 - Données migrées de l'ancien système

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.1. Préparation de la livraison

Plan de déploiement

Document central qui organise la mise en production.

Stratégies de déploiement

- ❑ **Big bang** : déploiement global en une seule fois
✓ rapide | ✗ risqué
- ❑ **Déploiement progressif (phased rollout)** : par site, service, profil ou fonctionnalité
✓ maîtrise des risques | ✗ transition plus longue
- ❑ **Déploiement pilote** : groupe restreint d'utilisateurs
✓ validation terrain | ✗ ressources supplémentaires
- ❑ **Déploiement parallèle** : ancien et nouveau système en coexistence
✓ très sécurisé | ✗ coûteux et lourd

Organisation du déploiement

- **Planning détaillé** : dates, horaires, durées, fenêtres de maintenance, points go/no-go
- **Rôles et responsabilités** :
 - Chef de projet : coordination et décisions
 - Équipe technique : déploiement
 - Exploitation : infrastructure et supervision
 - Support : assistance utilisateurs
 - Key users : validation et relais terrain
- **Prérequis** : infrastructure prête, licences, comptes, sauvegardes
- **Plan de rollback** : conditions, procédures, point de non-retour
- **Critères de succès** :
 - Système accessible
 - Fonctions critiques opérationnelles
 - Performances conformes
 - Aucun bug bloquant
 - Adoption conforme aux prévisions

Communication

- **Vers les utilisateurs** : planning, impacts, interruptions, contacts support
- **Vers le support** : formation, FAQ, anticipation des incidents
- **Vers la direction** : avancement, risques, arbitrages

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.2. Opérations de déploiement

Opérations techniques

- Installation / mise à jour serveurs et applications
- **Migration des données** (ETL) : extraction, transformation, chargement, contrôle
- Configuration production :
 - Connexions, paramètres, sécurité, supervision
- Activation :
 - Services, batchs, accès utilisateurs
- Vérifications :
 - Smoke tests
 - Connexions externes
 - Performances initiales
- Sauvegarde complète pour rollback

Opérations organisationnelles

- ☐ Bascule des utilisateurs vers le nouveau système
- ☐ Activation du support renforcé
- ☐ Monitoring technique et suivi de l'adoption

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.3. Accompagnement du déploiement

Formation des utilisateurs

☐ Formes :

- Présentiel
- Distanciel (webinaires)
- E-learning et vidéos
- Guides, cheat sheets, ateliers pratiques

☐ Adaptation aux profils :

- Utilisateurs finaux
- Power users
- Administrateurs
- Support

☐ Timing :

- Juste avant le déploiement
- Rappels après quelques semaines

Gestion du changement

- ☐ Communication orientée **bénéfices**
- ☐ Implication des utilisateurs en amont
- ☐ Identification des champions du changement
- ☐ Traitement des résistances
- ☐ Adaptation des processus métier

Support au démarrage

- ☐ Assistance sur site
- ☐ Hotline dédiée
- ☐ Chat / messagerie
- ☐ FAQ évolutive
- ☐ **Key users** comme relais de proximité

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.4. Suivi post-déploiement

Monitoring intensif

- **Performances** : temps de réponse, charge, disponibilité
- **Erreurs** : logs, exceptions, récurrences
- **Utilisation** : adoption, fonctionnalités utilisées
- **Incidents** : volume, délais de résolution, satisfaction

Corrections rapides

- Bugs critiques → hot fixes prioritaires
- Ajustements de configuration
- Améliorations ergonomiques rapides

Bilan de déploiement

- Respect du **planning et du budget**
- Analyse des incidents
- Satisfaction des utilisateurs
- Efficacité des formations et du support
- Ajustements pour la suite

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.5. Livrables de la livraison

- Plan de déploiement détaillé
- Procédures de déploiement et de rollback
- Package de livraison (code, exécutable, configs, données)
- Documentation complète
- Supports de formation
- Rapport de déploiement (déroulement, incidents, ajustements)

☞ *La livraison est une phase critique où la réussite dépend autant de la **technique** que de la **communication, la formation et l'accompagnement humain**.*

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

Le bilan de fin de projet permet de **capitaliser sur l'expérience**, évaluer la réussite du projet, formaliser la clôture administrative et préparer la transition vers l'exploitation et la maintenance.

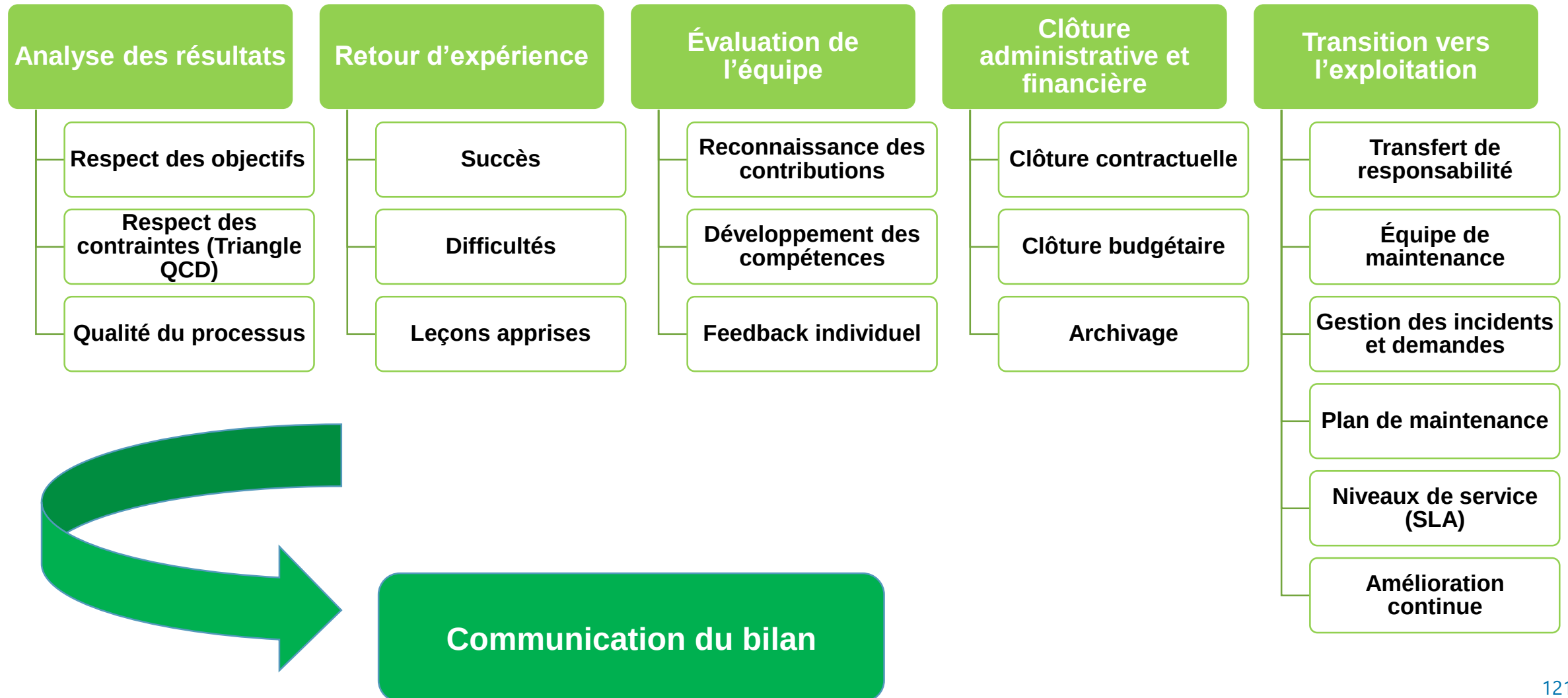
3.6.1. Objectifs du bilan

- Évaluer la réussite par rapport aux objectifs initiaux
- Identifier bonnes pratiques et erreurs à éviter
- Clôturer le projet sur les plans administratif et financier
- Préparer la transition vers l'exploitation et la maintenance
- Favoriser l'apprentissage organisationnel et l'amélioration continue

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet



Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

3.6.2. Analyse des résultats

Respect des objectifs

- Atteinte des objectifs initiaux
- Comparaison fonctionnalités livrées vs périmètre initial
- Qualité du livrable : performances, fiabilité, ergonomie, sécurité
- Bénéfices réels : gains de productivité, réduction des coûts, satisfaction client
- Adoption : taux d'utilisation, appropriation des nouvelles pratiques

Respect des contraintes (Triangle QCD)

- **Délais** : comparatif planning réel vs prévu, causes et impact des retards
- **Budget** : analyse des écarts, coûts par poste, ROI réel
- **Périmètre** : dérives éventuelles (scope creep), justification et impact

Qualité du processus

- Méthodologie : pertinence et application (Agile, Cycle en V, hybride)
- Planification et pilotage : suivi, décisions, réactivité
- Communication et collaboration : efficacité, gestion des conflits
- Outils et techniques : pertinence et adoption

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

3.6.3. Retour d'expérience

Succès

- Bonnes pratiques, facteurs de succès techniques, méthodologiques ou organisationnels
- Décisions clés judicieuses et moments de collaboration réussis
- Compétences ou expertises qui ont fait la différence

Difficultés

- **Techniques** : bugs tardifs, limites des technologies, intégration
- **Organisationnelles** : communication, coordination, processus lourds
- **Humaines** : compétences manquantes, surcharge, turnover
- **Externes** : fournisseurs, régulations, contraintes budgétaires

Leçons apprises

- Points de vigilance pour projets futurs
- Recommandations méthodologiques
- Besoins de formation
- Améliorations des processus et outils

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

3.6.4. Évaluation de l'équipe

- **Reconnaissance des contributions** : performances, initiatives, esprit d'équipe
- **Développement des compétences** : techniques, méthodologiques, transverses
- **Feedback individuel** : satisfaction, difficultés, enseignements et souhaits

3.6.5. Clôture administrative et financière

- **Clôture contractuelle** : levée des réserves, paiements, garanties
- **Clôture budgétaire** : analyse finale, comparaison budget initial vs réel, justification des écarts
- **Archivage** : cahier des charges, livrables techniques, documents contractuels, documentation opérationnelle

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

3.6.6. Transition vers l'exploitation

- **Transfert de responsabilité** vers l'équipe de maintenance
- **Équipe de maintenance** : formation, partage des points critiques et bugs connus
- **Gestion des incidents et demandes** : processus, classification, SLA, escalades
- **Plan de maintenance** :
 - Corrective : bugs production
 - Évolutive : nouvelles fonctionnalités
 - Adaptative : évolutions réglementaires et environnementales
 - Préventive : audits, optimisation, veille technologique
- **Niveaux de service (SLA)** : délais, disponibilité, résolution, plages de support
- **Amélioration continue** : quick wins, optimisations, roadmap produit

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

3.6.7. Communication du bilan

- **Réunion de clôture** : présentation des résultats, enseignements, perspectives, remerciements
- **Rapport de bilan** : synthèse des résultats, écarts, leçons, perspectives de maintenance
- **Partage des connaissances** :
 - Présentations internes pour équipes projet et métier
 - Mise à jour des référentiels méthodologiques
 - Contributions à la communauté interne ou externe (blogs, conférences)

Chapitre 4 : Les cycles de vie

- Introduction
- Les modèles séquentiels
- Le modèle incrémental
- Le cycle de vie « itératif »
- Les méthodes « agiles »
- Les bonnes pratiques
- Les normes

Chapitre 4 : Les cycles de vie

4.1 Introduction

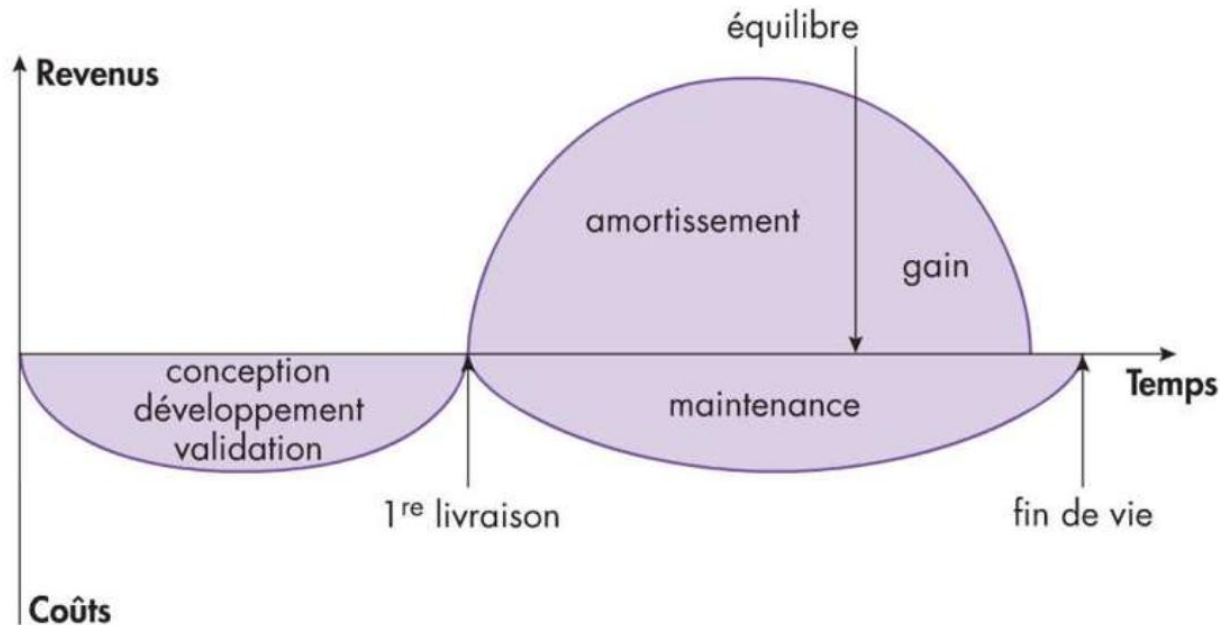
Le **cycle de vie d'un projet informatique** décrit les différentes phases allant **de la conception à la clôture du projet**. Le choix du cycle de vie est **stratégique**, car il influence l'organisation, la gestion des risques, la communication et la réussite du projet.

Il n'existe pas de modèle unique adapté à tous les projets : **chaque cycle présente des avantages et des limites** selon le contexte et les contraintes.

Le chapitre aborde les **principaux cycles de vie, des modèles traditionnels aux méthodes agiles, ainsi que les normes et bonnes pratiques** permettant de choisir et d'adapter le cycle le plus approprié.

Chapitre 4 : Les cycles de vie

4.1 Introduction



Flux financiers d'un projet (d'après Jacques Printz)

- ❖ La figure montre l'évolution des **flux financiers d'un projet dans le temps**.
- ❖ Au début (conception, développement, validation), le projet génère uniquement des **coûts**.
- ❖ Après la **première livraison**, les revenus apparaissent pendant la phase de **maintenance**.
- ❖ Les revenus servent d'abord à **amortir les coûts initiaux** jusqu'au **point d'équilibre**.
- ❖ Au-delà de ce point, le projet devient **rentable** et génère un **gain**, jusqu'à sa **fin de vie**.

Chapitre 4 : Les cycles de vie

4.1 Introduction

Définition des objectifs

Analyse des besoins et faisabilité

Conception générale

Conception détaillée

Développement

Tests

Recette

Documentation

Accompagnement

Mise en production

Assistance et maintenance

Chapitre 4 : Les cycles de vie

4.1 Introduction

Séquentiel	Nom	Description	Remarques
	Cascade	Découpage du projet en phases sans retour à la phase précédente	✓ Réduction des risques ✓ Intervention des utilisateurs en fin de cycle seulement, souvent trop tard ✓ Contrôle qualité à la fin de chaque phase ✓ Destiné à un projet < 1 an et à connotation réglementaire ✓ Pas d'évolution des besoins
	En « V »	Contrôle qualité continu tout au long du processus	✓ Projet de taille moyenne et peu complexe

Chapitre 4 : Les cycles de vie

4.1 Introduction

Cycles de vie		Description	Remarques
Incrémental		Découpage du projet en domaines ayant chacun un cycle autonome en cascade	☐ Grands projets
Itératif	UP	Méthode itérative et incrémentale organisée en 4 phases , pilotée par les cas d'utilisation et centrée sur l' architecture	☐ Générique
	Spirale	Méthode évolutive basée sur la réalisation de prototype	☐ Destiné aux grands projets complexes internes difficilement contractualisables ☐ Maîtrise et réduction des risques
	RAD	Construction de la solution avec l'utilisateur	☒ Implication forte des utilisateurs ☒ Nécessite une maîtrise des technologies ☒ Expertise de tous les participants

Chapitre 4 : Les cycles de vie

4.1 Introduction

Agile	Nom	Description	Remarques
	Scrum	Méthode itérative basée sur un «backlog» de fonctionnalité	☐ Méthode rigoureuse ☐ Contrôle continu ☐ Cycle d'un mois ☐ Petit projet
	Extreme Programming	Livraison à chaque itération d'un livrable	☐ Destiné aux petits projets ☐ Souplesse au changement ☐ Petite équipe ☐ Déconseillé dans le cadre d'une prestation forfaitaire (coûts, délais)

Chapitre 4 : Les cycles de vie

4.1 Introduction

Bonnes pratiques	Nom	Description	Remarques
	ITIL	Méthode basée sur la gestion des services	✓ Capitalisation sur les bonnes pratiques
	PRINCE2	Méthode évolutive basée sur l'organisation, la gestion et le contrôle	✓ Méthode souple et dérivée de l'expérience des chefs de projets

Chapitre 4 : Les cycles de vie

4.1 Introduction

Normes	Nom	Description	Remarques
	Z67-101	Découpage d'un projet en phases et étapes	❑ Répond aux questions « qui fait quoi et quand ? » ; ne traite pas le « comment ». ❑ Destinée aux projets de type gestion
	ISO 12207	Processus de base D'un cycle de vie	❑ Évolution de la norme Z67-101 ❑ Difficilement applicable sans adaptation ❑ Orientée gros système
	ISO 21500	Approche synthétique du management de	❑ Destinée aux décideurs, elle apporte un éclairage sur les principes et pratiques saines et constituantes de la gestion

Chapitre 4 : Les cycles de vie

4.2. Les Modèles Séquentiels

4.2.1. Le modèle en cascade (Waterfall)

Le **modèle en cascade**, formalisé par Winston Royce en 1970, organise un projet en phases séquentielles : **analyse des besoins, conception générale et détaillée, implémentation, tests et intégration, déploiement, maintenance**. Chaque phase produit des livrables et nécessite une validation avant de passer à la suivante.

Chapitre 4 : Les cycles de vie

4.2. Les Modèles Séquentiels

4.2.1. Le modèle en cascade (Waterfall)

Avantages

- Simplicité
- planification prévisible
- documentation complète
- discipline structurée
- adapté aux environnements réglementés

Inconvénients

- rigidité face au changement
- détection tardive des erreurs
- faible visibilité pour le client
- risque de non-conformité aux besoins évolutifs
- dépendance à des hypothèses initiales

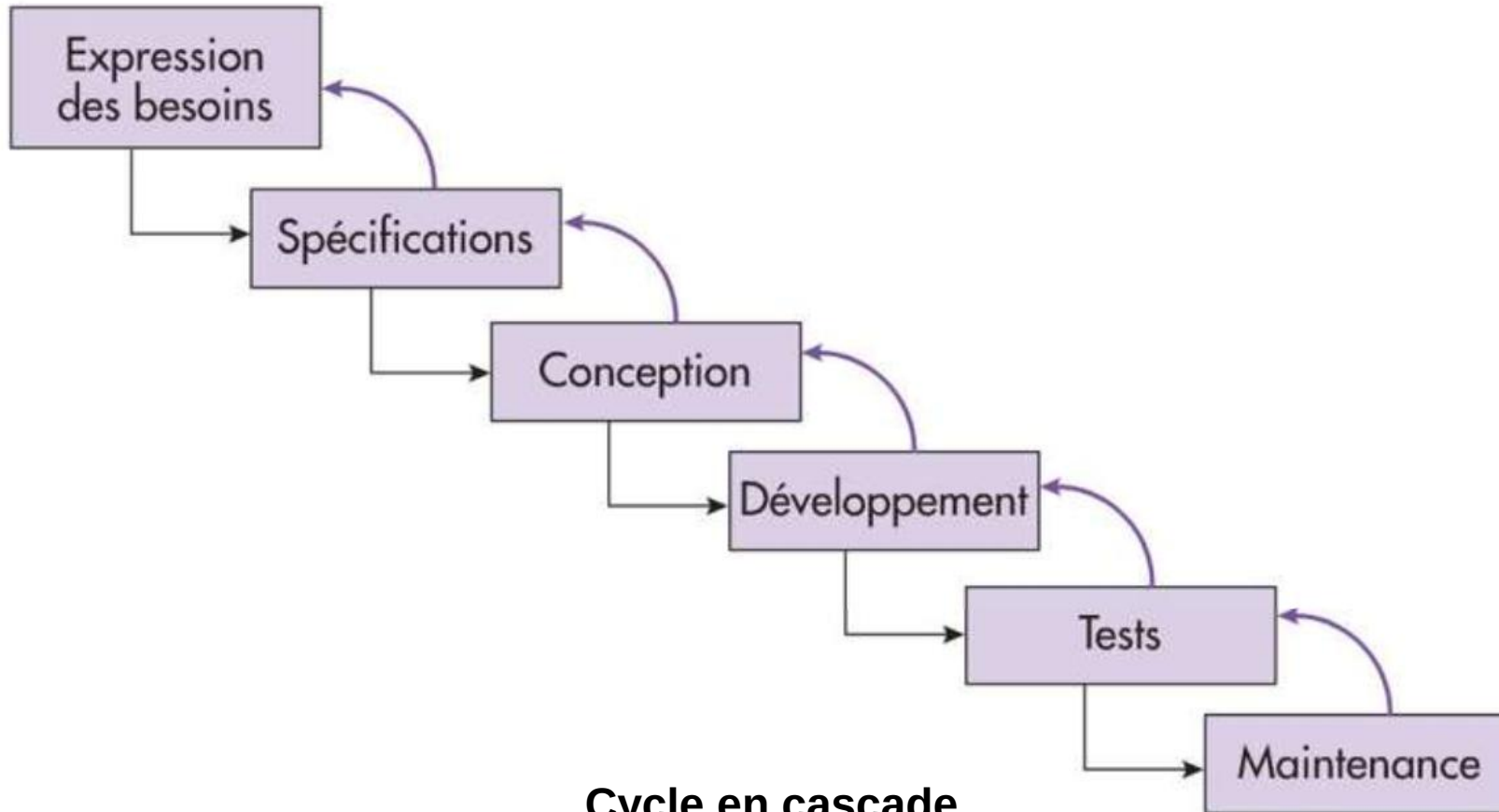
Usage approprié

- projets avec exigences stables
- technologies maîtrisées
- contraintes réglementaires fortes
- courte durée
- équipes peu expérimentées

Chapitre 4 : Les cycles de vie

4.2. Les Modèles Séquentiels

4.2.1. Le modèle en cascade (Waterfall)



Chapitre 4 : Les cycles de vie

4.2. Les Modèles Séquentiels

4.2.1. Le modèle en cascade (Waterfall)

Adoption du modèle séquentiel (cycle en cascade)

Contexte adapté

- Besoins **stables et bien connus**
- Peu de changements attendus
- Forte exigence de **documentation officielle**

◆ Étapes du projet (cascade)

1. Expression des besoins

- Réunions avec :
 - administration
 - enseignants
 - direction
- Besoins identifiés :
 - gestion des étudiants
 - gestion des notes
 - attestations
 - inscriptions

2. Spécifications

- Rédaction d'un **cahier des charges complet**
- Fonctions détaillées :
 - rôles (admin, enseignant, étudiant)
 - règles de calcul des notes
 - sécurité

3. Conception

- Architecture du système
- Base de données globale
- Interfaces principales

4. Développement

- Développement de l'application complète

5. Tests

- Tests globaux en fin de projet

6. Déploiement & maintenance

- Mise en production unique
- Corrections après livraison

✓ Avantages pour l'ENSAH

- Organisation claire
- Bon pour un projet institutionnel cadré

✗ Limites

- Peu flexible si les besoins évoluent

Chapitre 4 : Les cycles de vie

4.2. Les Modèles Séquentiels

4.2.2. Le modèle en V

Le **modèle en V** est une amélioration du modèle en cascade, centrée sur la **vérification et la validation**, largement utilisée pour les systèmes critiques.

❑ **Structure** : la branche descendante correspond aux phases de spécification et conception **(besoins, spécifications fonctionnelles, architecture, conception détaillée, codage)**, et la branche montante aux phases de tests **(unitaires, intégration, validation fonctionnelle, acceptation)**. Chaque phase descendante a un test correspondant (principe de symétrie).

Chapitre 4 : Les cycles de vie

4.2. Les Modèles Séquentiels

4.2.2. Le modèle en V

Avantages

- anticipation des tests
- traçabilité exigences-tests
- détection plus précoce des anomalies
- structuration rigoureuse
- adapté aux systèmes critiques

Inconvénients

- rigidité
- effet tunnel
- difficulté face aux changements
- lourdeur documentaire
- linéarité persistante

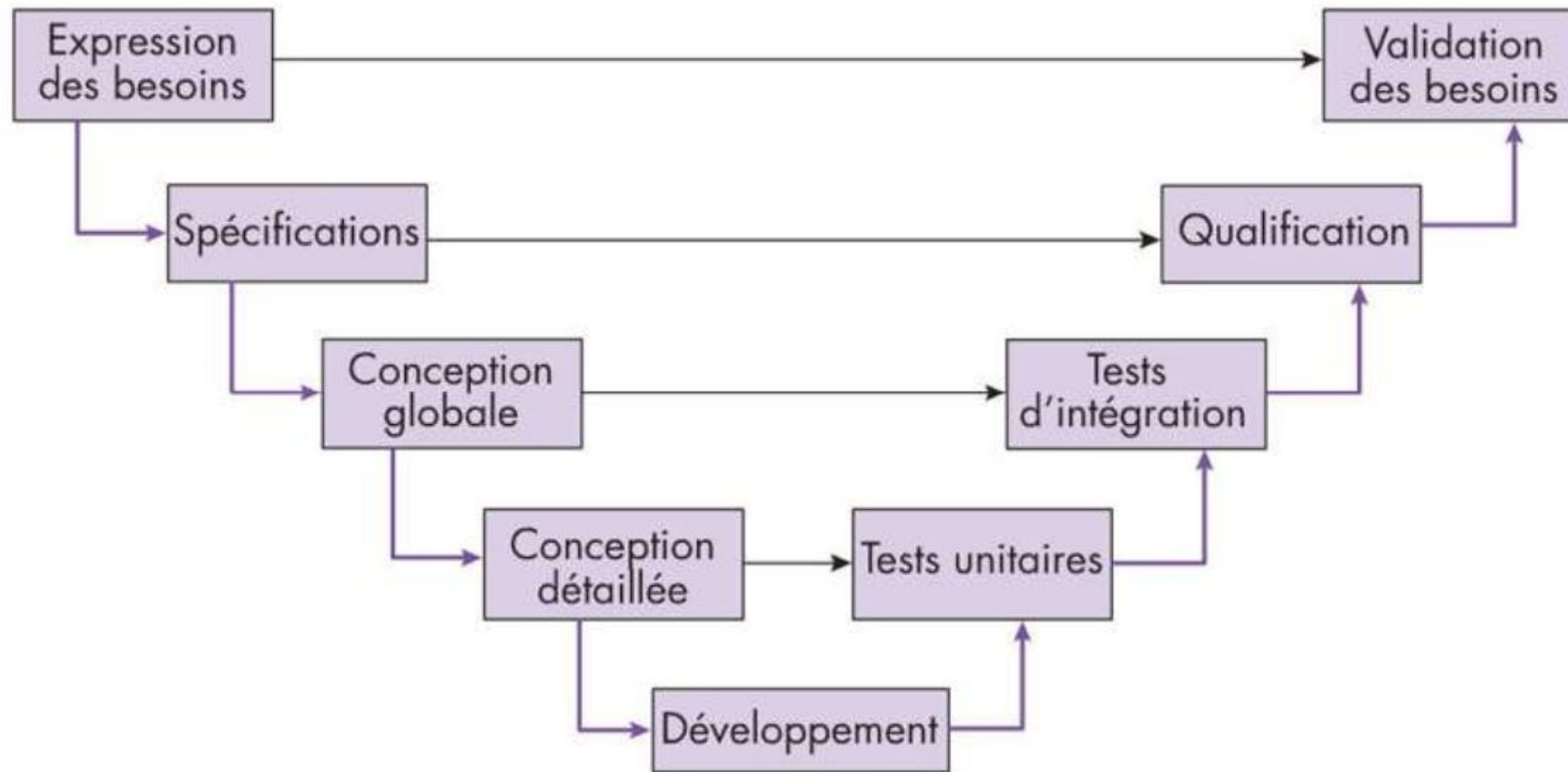
Variantes

- V itératif (cycles courts)
- V incrémental (par sous-ensembles)
- V avec prototypage (validation précoce de spécifications critiques)

Chapitre 4 : Les cycles de vie

4.2. Les Modèles Séquentiels

4.2.2. Le modèle en V



Cycle en V

Chapitre 4 : Les cycles de vie

4.2. Les Modèles Séquentiels

4.2.2. Le modèle en V

Adoption du cycle en V

✦ Contexte adapté

- Exigence forte de **qualité** et de **traçabilité**
- Système critique (notes, diplômes, données étudiantes)

◆ Étapes du projet (cycle en V)

Phase de définition	Phase de test associée
Expression des besoins	Validation par l'administration
Spécifications	Qualification fonctionnelle
Conception globale	Tests d'intégration
Conception détaillée	Tests unitaires
Développement	—

◆ Exemple concret

- ❑ Lors de la **définition des besoins** :
 - On prépare déjà les **scénarios de validation**
- ❑ Lors de la **conception détaillée** :
 - On définit les **tests unitaires** pour chaque module :
 - module notes
 - module inscriptions
 - module attestations

✓ Avantages pour l'ENSAH

- Très forte qualité
- Traçabilité complète
- Sécurité et fiabilité élevées

✗ Limites

- Rigide
- Changements coûteux

Chapitre 4 : Les cycles de vie

4.2. Les Modèles Séquentiels

4.2.3. Limites générales des modèles séquentiels

Les **modèles séquentiels** (comme cascade ou V) sont structurés et prévisibles, mais présentent des limites face aux projets modernes :

Inadéquation à l'incertitude

- difficulté à gérer des besoins incomplets ou évolutifs

Manque de réactivité

- longue durée avant livraison, retardant les opportunités

Coût élevé du changement

- modifications tardives coûteuses

Effet tunnel

- absence de livraisons intermédiaires, faible visibilité pour le client



Ces limites ont conduit au développement de **méthodes plus flexibles et adaptatives.**

Chapitre 4 : Les cycles de vie

4.3. Le Modèle incrémental

Le **modèle incrémental** développe un système par **incréments successifs**, chaque incrément ajoutant des fonctionnalités au précédent. **Chaque incrément suit un cycle complet (analyse, conception, développement, test, livraison).**

Chapitre 4 : Les cycles de vie

4.3. Le Modèle incrémental

Avantages

- livraison rapide
- retour d'expérience précoce
- réduction des risques
- flexibilité dans les priorités
- motivation des équipes grâce aux livraisons régulières

Défis

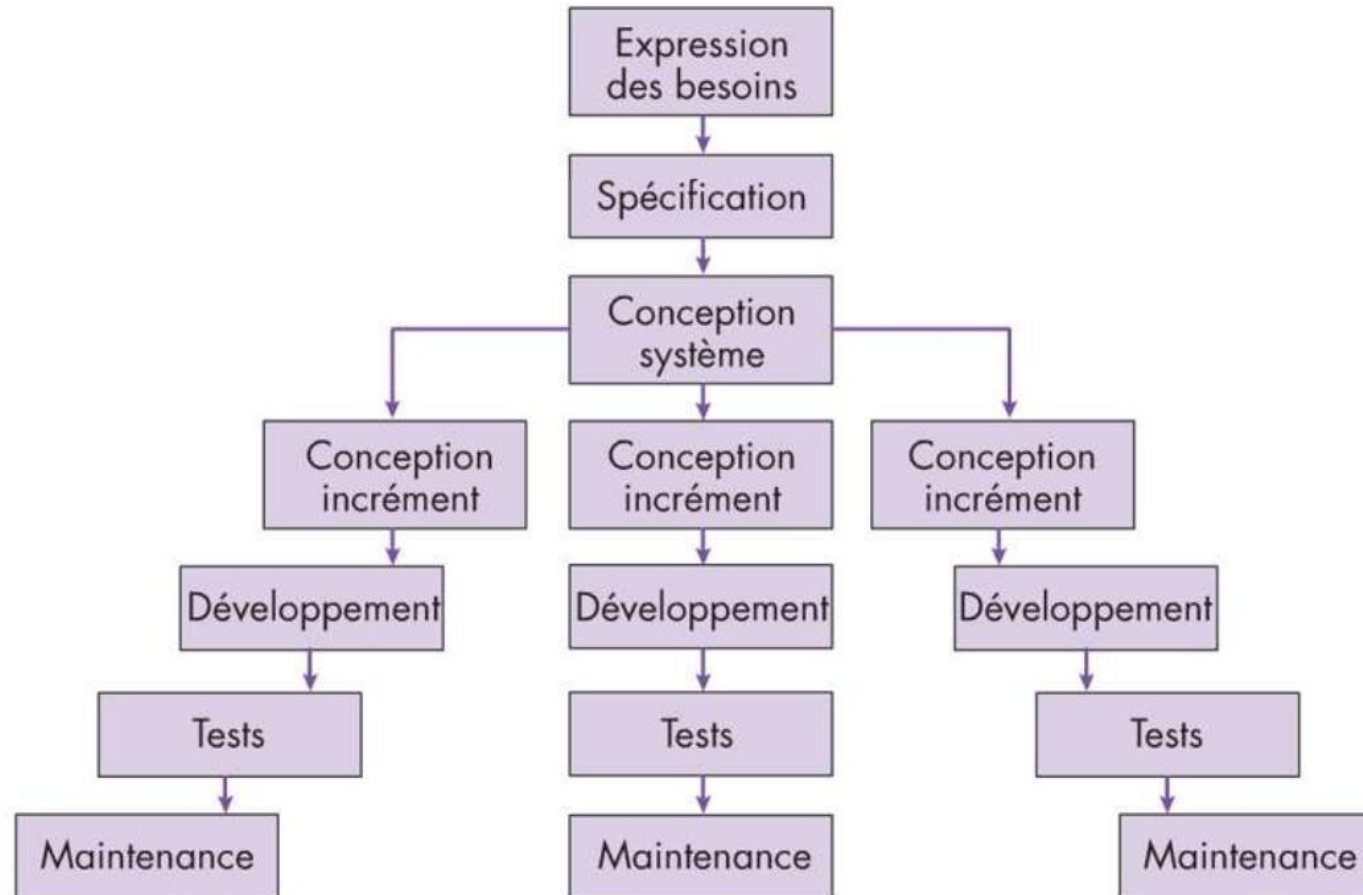
- architecture initiale robuste nécessaire
- gestion des dépendances
- tests de régression
- maintien de la cohérence du système

Différence avec le modèle itératif

- l'incrémental complète chaque fonctionnalité en un seul cycle, tandis que l'itératif raffine progressivement les mêmes fonctionnalités sur plusieurs cycles

Chapitre 4 : Les cycles de vie

4.3. Le Modèle incrémental



Le modèle incrémental

Chapitre 4 : Les cycles de vie

4.3. Le Modèle incrémental

Adoption du modèle incrémental

★ Contexte adapté

- Projet large
- Besoins évolutifs
- Volonté de livrer **progressivement**

◆ Vision globale (commune)

- Expression des besoins
- Spécifications globales
- Conception du système (architecture)

✓ Avantages pour l'ENSAH

- Valeur livrée rapidement
- Réduction des risques
- Implication progressive des utilisateurs
- Adaptation facile aux retours

✗ Limites

- Nécessite une bonne architecture initiale
- Coordination plus complexe

◆ Découpage en incréments

◆ Incrément 1 (prioritaire)

- Gestion des étudiants
 - Authentification
 - Profils utilisateurs
- Livré en premier semestre

◆ Incrément 2

- Gestion des notes
 - Consultation des résultats
 - Export PDF
- Livré après validation de l'incrément 1

◆ Incrément 3

- Inscriptions pédagogiques
- Choix des modules
- Emplois du temps

◆ Incrément 4

- Demandes administratives en ligne
- Attestations
- Notifications

◆ Pour chaque incrément

- Conception
- Développement
- Tests
- Mise en production
- Feedback utilisateurs

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.1. Principe de l'approche itérative

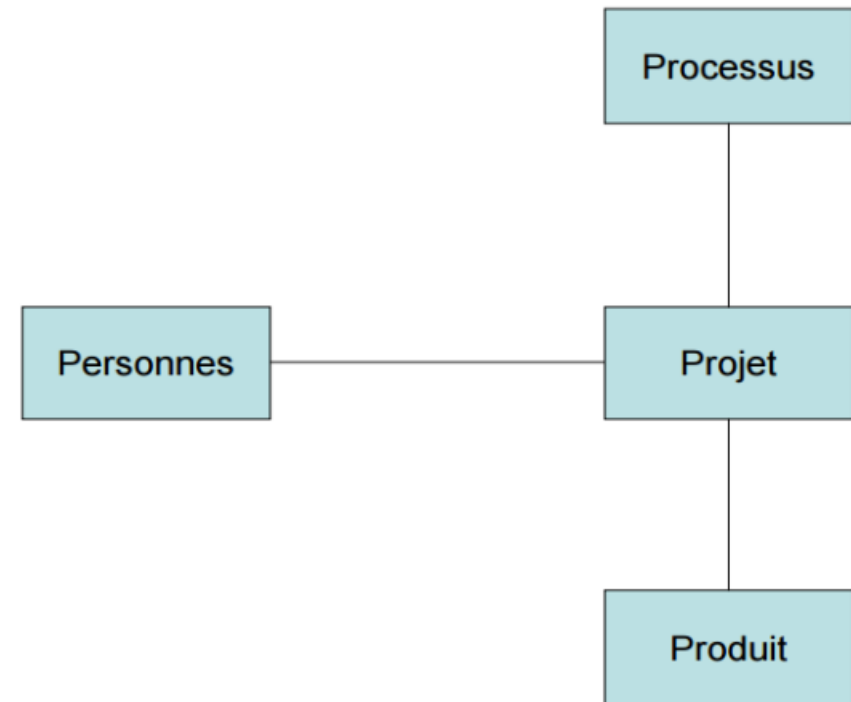
L'**approche itérative** développe le système par cycles répétés (**itérations**), en **raffinant progressivement l'ensemble du système**. Chaque **itération** couvre toutes les phases du développement (analyse, conception, implémentation, test) à un niveau croissant de détail. Les retours d'expérience de chaque itération servent à améliorer la suivante, permettant d'adapter le système au fur et à mesure de la compréhension des besoins.

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

- Un processus définit une **séquence d'étapes**, en **partie ordonnées**, qui concourent à l'obtention d'un **système logiciel** ou à **l'évolution d'un système existant**.
- L'objet d'un processus de développement est de produire des **logiciels de qualité** qui répondent aux besoins de leurs utilisateurs dans des **temps et des coûts prévisibles**.



Les 4 «P» du développement logiciel

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

« Le processus unifié est un **processus de développement logiciel**, c'est-à-dire qu'il regroupe les **activités** à mener pour **transformer** les besoins d'un utilisateur en système logiciel » (Jacobson, Booch, Rumbaugh 1999)



Grady Booch



James Rumbaugh



Ivar Jacobson

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

- ❑ Le Processus Unifié ou UP (Unified Process) est une méthode **générique** de développement de logiciel développée par les concepteurs d'UML.
- ❑ **Générique** signifie qu'il est nécessaire d'adapter UP au contexte du projet, de l'équipe, du domaine et/ou de l'organisation.
- ❑ Il existe donc un certain nombre de méthodes issues de UP comme par exemple **RUP** (Rational Unified Process), **2TUP** (Two Track Unified Process) ...
- ❑ L'objectif d'un processus unifié est de maîtriser la complexité des projets informatiques en diminuant les risques.
- ❑ UP répond aux préoccupations suivantes :
 - **QUI** participe au projet ?
 - **QUOI**, qu'est-ce qui est produit durant le projet ?
 - **COMMENT** doit-il être réalisé ?
 - **QUAND** est réalisé chaque livrable ?

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

Le processus de développement UP, associé à UML, met en œuvre les principes suivants :

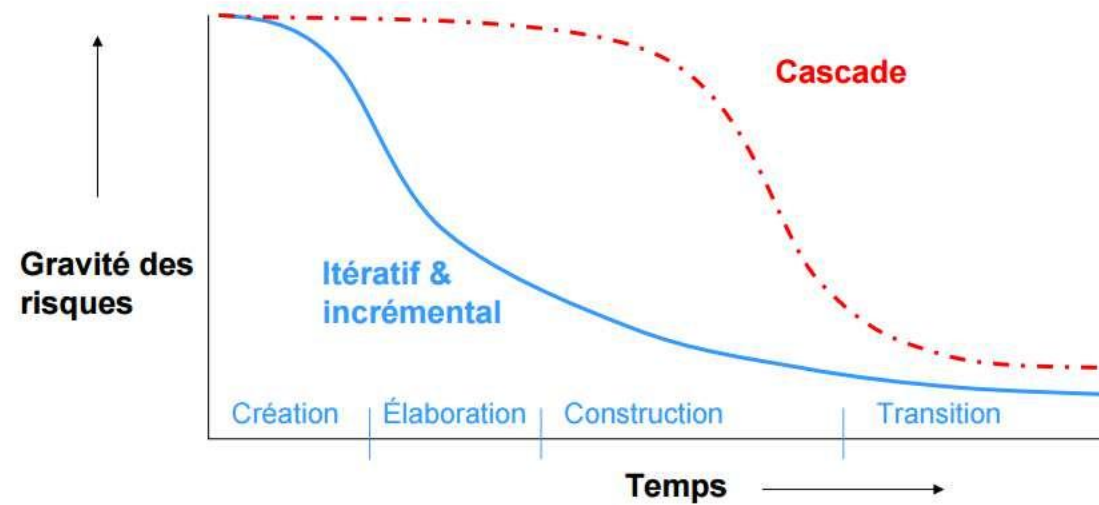
- ☐ Le processus unifié utilise le langage **UML**
- ☐ Processus **guidé par les cas d'utilisation**,
- ☐ Processus **itératif et incrémental**,
- ☐ Processus **centré sur l'architecture**,
- ☐ Processus orienté par la **réduction des risques**.

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

UP contribue à la diminution des risques au fur et à mesure du déroulement des itérations successives.

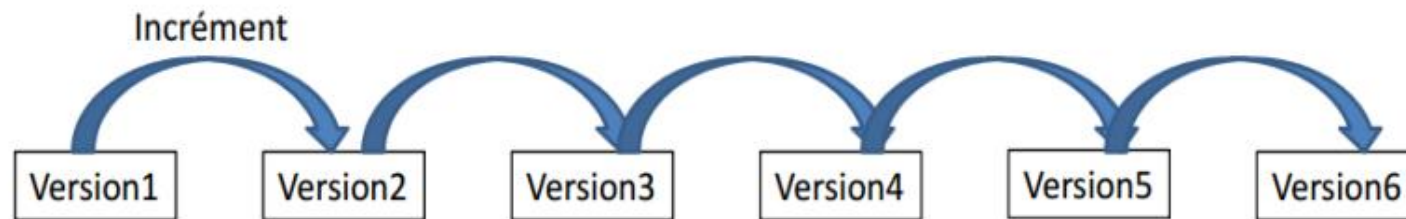


Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

- ❑ UP utilise le principe de développement par itérations successives.
 - ✓ La réalisation de maquette et prototype constitue la réponse pratique à ce principe.
- ❑ Le développement progressif, par incrément, est aussi recommandé (décomposition du système en cas d'utilisation)

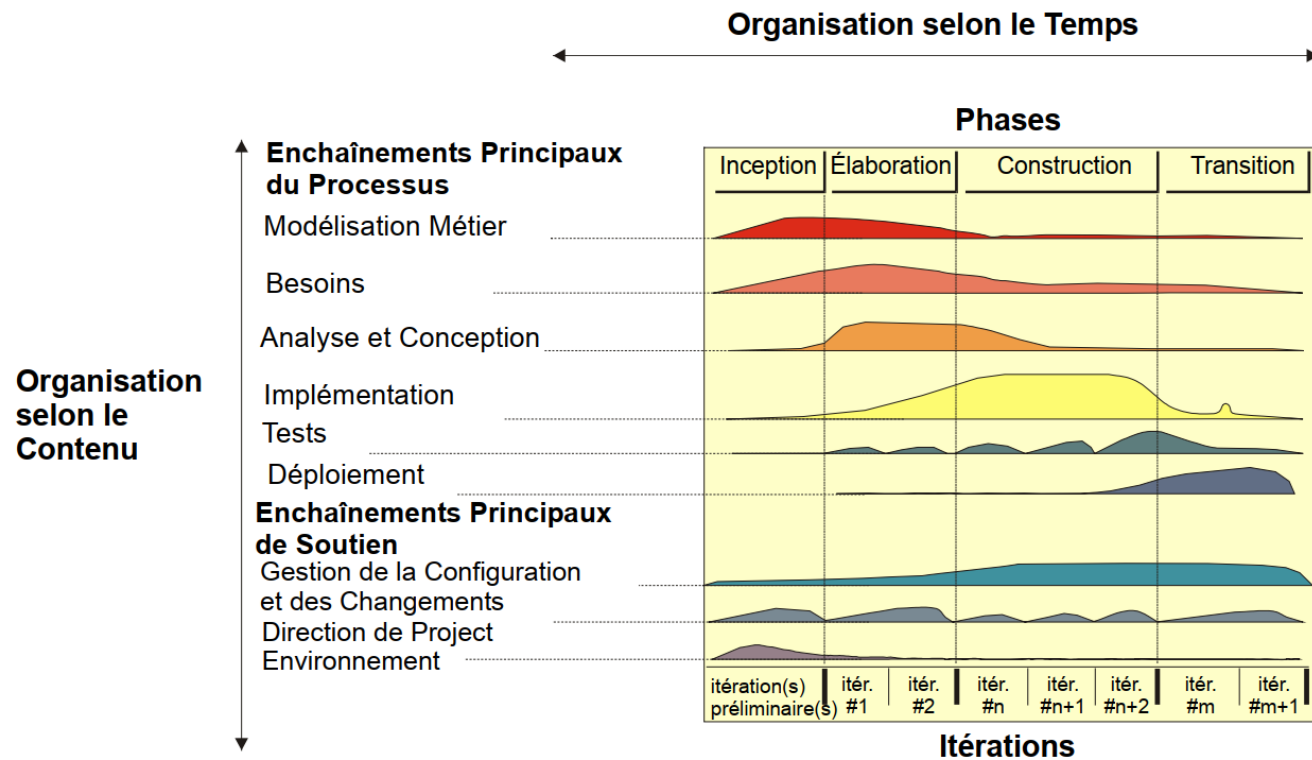


Plusieurs versions avant d'obtenir toutes les spécifications, Chaque version est testée, reçue par un utilisateur.

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)



UP gère le processus de développement par deux axes.

❑ **L'axe vertical** représente les principaux enchaînements d'activités, qui regroupent les activités selon leur nature.

❑ **L'axe horizontal** représente le temps et montre le déroulement du cycle de vie du processus

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

Le **Processus Unifié** et sa variante **RUP** sont des cadres méthodologiques **itératifs et incrémentaux** développés dans les années 1990.

❑ **Phases principales :**

Inception

- définir la vision, identifier cas d'utilisation et risques, estimer coûts et planning, décider du go/no-go.

Elaboration

- affiner la vision, analyser le domaine, établir l'architecture, traiter les risques majeurs, ajuster planning et estimations

Construction

- développer le système itérativement, finaliser fonctionnalités, préparer le déploiement et la documentation

Transition

- déployer le système, former les utilisateurs, corriger les bugs, ajuster selon les retours

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

Modélisation métier

- Forte en Inception
- Sert à comprendre le contexte métier et les processus existants
- Diminue ensuite mais reste présente

Besoins (exigences)

- Très importantes en Inception et Élaboration
- Stabilisation progressive des besoins
- Peu de nouveaux besoins en fin de projet

Analyse et conception

- Pic en Élaboration
- Mise en place de l'architecture logicielle
- Choix techniques majeurs

Implémentation

- Faible au début
- Très forte en Construction
- Le système est développé par incréments

Tests

- Présents dès le début
- Montent en puissance avec l'implémentation
- Très importants avant la Transition

Déploiement

- Faible au départ
- Principalement en Transition
- Livraison, formation, mise en production

Disciplines de soutien (transversales)

Ces activités sont **présentes tout au long du projet** :

◆ Gestion de configuration et des changements

- Versioning, gestion des modifications
- Essentielle dans un contexte itératif

◆ Direction de projet

- Planification, suivi, gestion des risques
- Continue du début à la fin

◆ Environnement

- Outils, frameworks, plateformes
- Mise en place tôt, puis maintenance

Rôle des itérations

- Itérations préliminaires
- Puis itérations successives dans chaque phase

Chaque itération produit :

- Un incrément fonctionnel
- Testé et potentiellement démontrable

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

Les phases se décomposent en activités :

1) Expression des besoins:

Recenser les **besoins fonctionnels** (du point de vue de l'utilisateur) qui conduisent à l'élaboration des modèles de cas d'utilisation appréhender les **besoins non fonctionnels** (technique)

Artefacts obtenus :

- ☐ Liste des fonctions
- ☐ Modèle des cas d'utilisation
- ☐ Exigences supplémentaires
- ☐ Prototypage de l'IHM
- ☐ Plan de tests de validation

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

2) Analyse : L'objectif de l'analyse est d'accéder à une compréhension des besoins et des exigences du client.

Artefacts obtenus :
Diagramme de classe d'analyse

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

3)Conception : La conception permet d'acquérir une compréhension approfondie des contraintes liées au **langage de programmation**, à l'utilisation des **composants** et **framework** et au **système d'exploitation**.

Artefacts obtenus :

- ☐ diagramme de classes de conception
- ☐ diagramme d'interaction (séquence)
- ☐ diagramme d'activités et/ou d'états
- ☐ diagramme de déploiement
- ☐ plan de tests d'intégration

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

4) Implémentation :

la programmation. le résultat de la conception pour implémenter le système sous formes de composants

Artefacts obtenus :

- ☐ code source, scripts, binaires
- ☐ diagramme de composant
- ☐ diagramme de déploiement
- ☐ plan de tests unitaires

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

5)Test : Pour mener à bien ces tests, il faut les planifier pour chaque itération, les implémenter en créant des cas de tests, effectuer ces tests et prendre en compte le résultat de chacun.

Artefacts obtenus :

Rapport des tests unitaires, d'intégration et de validation

6)Déploiement

Artefacts obtenus :

Manuels d'installation et d'utilisation

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

Caractéristiques

- Centré sur l'architecture
- Piloté par les cas d'utilisation
- Orienté risques
- Adaptable aux besoins du projet
- Chaque phase pouvant contenir plusieurs itérations

Avantages

- Cadre complet et structuré
- Réduction des risques
- Favorise la réutilisation
- Standardisation UML
- Flexibilité

Limites

- complexité et lourdeur
- documentation abondante
- difficile pour petites structures
- perçu comme trop processé et pouvant limiter la créativité

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

📍 Contexte du projet ENSAH

Objectif du projet

Mettre en place un **système d'information numérique** couvrant :

- ✓ inscriptions administratives et pédagogiques
- ✓ gestion des étudiants
- ✓ gestion des notes et résultats
- ✓ demandes administratives en ligne
- ✓ communication interne

Contraintes

- ✓ Données sensibles
- ✓ Plusieurs acteurs (administration, enseignants, étudiants)
- ✓ Besoins partiellement évolutifs

📖 **La méthode UP est très adaptée** car elle est :

- ✓ itérative
- ✓ incrémentale
- ✓ orientée architecture
- ✓ pilotée par les risques

Phase 1 : Inception (Lancement)

📍 Objectifs

- Définir la **vision du projet**
- Identifier les **acteurs**
- Délimiter le **périmètre**
- Identifier les **risques majeurs**
- Décider **GO / NO GO**

Application à l'ENSAH

Réunions avec :

- ✓ direction
- ✓ service scolarité
- ✓ enseignants
- ✓ représentants des étudiants

Identification des **besoins majeurs** :

- ✓ s'inscrire en ligne
- ✓ consulter les notes
- ✓ demander une attestation

Identification des **acteurs** :

- ✓ étudiant
- ✓ enseignant
- ✓ administrateur

📄 Livrables

- ✓ Document de vision
- ✓ Liste des acteurs
- ✓ Diagramme de cas d'utilisation (haut niveau)
- ✓ Analyse préliminaire des risques
- ✓ Planning global

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

Phase 2 : Elaboration (Phase clé de UP)

🎯 Objectifs

- ❑ Stabiliser les **besoins essentiels**
- ❑ Définir et valider l'**architecture**
- ❑ Réduire les **risques techniques et fonctionnels**

Application à l'ENSAH

- ❑ Détail des **cas d'utilisation critiques** :
 - gestion des notes
 - authentification et sécurité
 - ❑ Choix techniques :
 - architecture web
 - base de données centralisée
 - ❑ Modélisation UML :
 - diagramme de classes d'analyse
 - diagramme de classes de conception
- Réalisation d'un **prototype fonctionnel** (ex. : login)

■ Livrables

- Architecture validée
- Cas d'utilisation détaillés
- Diagrammes UML
- Plan de tests
- Plan de gestion des risques

👉 À la fin : **architecture et besoins critiques sont figés**

Phase 3 : Construction

🎯 Objectifs

- Développer le système **par itérations**
- Livrer des **incréments fonctionnels**
- Tester et faire des **recettes intermédiaires**

◆ Itération 1

- Authentification
- Gestion des profils utilisateurs
→ Tests + **recette partielle**

◆ Itération 2

- Gestion des étudiants
- Inscriptions pédagogiques
→ Tests + recette partielle

◆ Itération 3

- Gestion des notes
- Consultation des résultats
→ Tests + recette partielle

◆ Itération 4

- Demandes administratives
- Génération d'attestations
→ Tests + recette partielle

■ Livrables

- ✓ Incréments livrés
- ✓ Code testé
- ✓ Rapports de tests
- ✓ PV de recettes intermédiaires

Phase 4 : Transition

🎯 Objectifs

- Déployer le système
- Former les utilisateurs
- Réaliser la **recette finale**
- Mettre en production

Application à l'ENSAH

- Migration des données existantes
- Formation :
 - personnel administratif
 - enseignants
- Tests en conditions réelles
- Corrections finales

■ Livrables

- Système en production
- Guides utilisateurs
- **PV de recette finale**
- Bilan de fin de projet

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.2. Le Processus Unifié (Unified Process - UP)

◆ Inception – itérations de cadrage

But : comprendre et cadrer le projet

↻ Itération 1 :

- Identifier les acteurs
- Définir les objectifs

↻ Itération 2 :

- Affiner le périmètre
- Identifier les risques majeurs

■ Résultat :

- Vision validée
- GO / NO GO

◆ Elaboration – itérations d'architecture (phase clé)

But : sécuriser le projet

↻ Itération 1 :

- Modélisation des cas d'utilisation critiques
- Prototypage d'authentification

↻ Itération 2 :

- Diagrammes de classes
- Choix techniques
- Tests d'architecture

■ Résultat :

- Architecture stable
- Risques majeurs levés

◆ Construction – itérations fonctionnelles

But : produire le système

↻ Itération 1 :

- Gestion des étudiants

↻ Itération 2 :

- Gestion des notes

↻ Itération 3 :

- Demandes administratives

■ Résultat :

- Incréments livrés
- Recettes partielles

◆ Transition – itérations de déploiement

But : livrer et accepter le système

↻ Itération 1 :

- Déploiement pilote
- Retours utilisateurs

↻ Itération 2 :

- Corrections
- Recette finale

■ Résultat :

- Système accepté

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.3. Le prototypage

L'idée centrale est de construire une **version partielle et préliminaire** du système (**le prototype**) pour explorer, valider et affiner les besoins **avant** de développer **le produit final**.

Les étapes du modèle

- 1) **Recueil des besoins initiaux** — on collecte une première vision, souvent incomplète
- 2) **Conception rapide** — on esquisse rapidement une solution
- 3) **Construction du prototype** — une version simplifiée et fonctionnelle est créée
- 4) **Évaluation par l'utilisateur** — le client teste et donne ses retours
- 5) **Raffinement** — on améliore le prototype selon les retours
- 6) **Répétition** — les étapes 4 et 5 se répètent jusqu'à satisfaction
- 7) **Développement du produit final** — une fois les besoins validés, on construit le vrai système

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.3. Le prototypage

Prototype jetable (*throwaway*)

- on le construit uniquement pour clarifier les besoins, puis on le jette et on repart de zéro proprement

Prototype évolutif

- le prototype est progressivement amélioré jusqu'à devenir le produit final.

Prototype horizontal

- couvre de nombreuses fonctionnalités mais de façon superficielle (surtout l'interface)

Prototype vertical

- approfondit une seule fonctionnalité de bout en bout

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.3. Le prototypage

Avantages

- Réduit les malentendus entre client et développeur
- Permet de détecter les problèmes très tôt
- L'utilisateur voit quelque chose de concret rapidement
- Très utile quand les besoins sont mal définis

Inconvénients

- Risque que le client confonde le prototype avec le produit fini
- Peut encourager une mauvaise gestion si on néglige la documentation
- Le prototype jetable peut sembler "du travail perdu"
- Risque de dérive si on ne sait pas quand arrêter les itérations

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.4. Le modèle en spirale

Le **modèle en spirale**, proposé par Barry Boehm en 1986, est un modèle **itératif centré sur la gestion des risques**.

- ❑ **Structure** : chaque itération (spirale) comporte quatre quadrants :
 - Détermination des objectifs (objectifs, alternatives, contraintes)
 - Analyse et gestion des risques (identification, évaluation, mitigation, prototypage éventuel)
 - Développement et validation (réalisation de l'itération, vérification des objectifs)
 - Planification (revue avec les parties prenantes, planification de l'itération suivante)

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.4. Le modèle en spirale

🔄 Chaque tour de spirale comprend 4 étapes :

1. Planification

- ✓ Définir les objectifs du cycle
- ✓ Identifier les besoins à traiter

2. Analyse et évaluation des risques

- ✓ Identifier les risques techniques, fonctionnels, organisationnels
- ✓ Proposer des solutions pour les réduire

3. Développement & tests

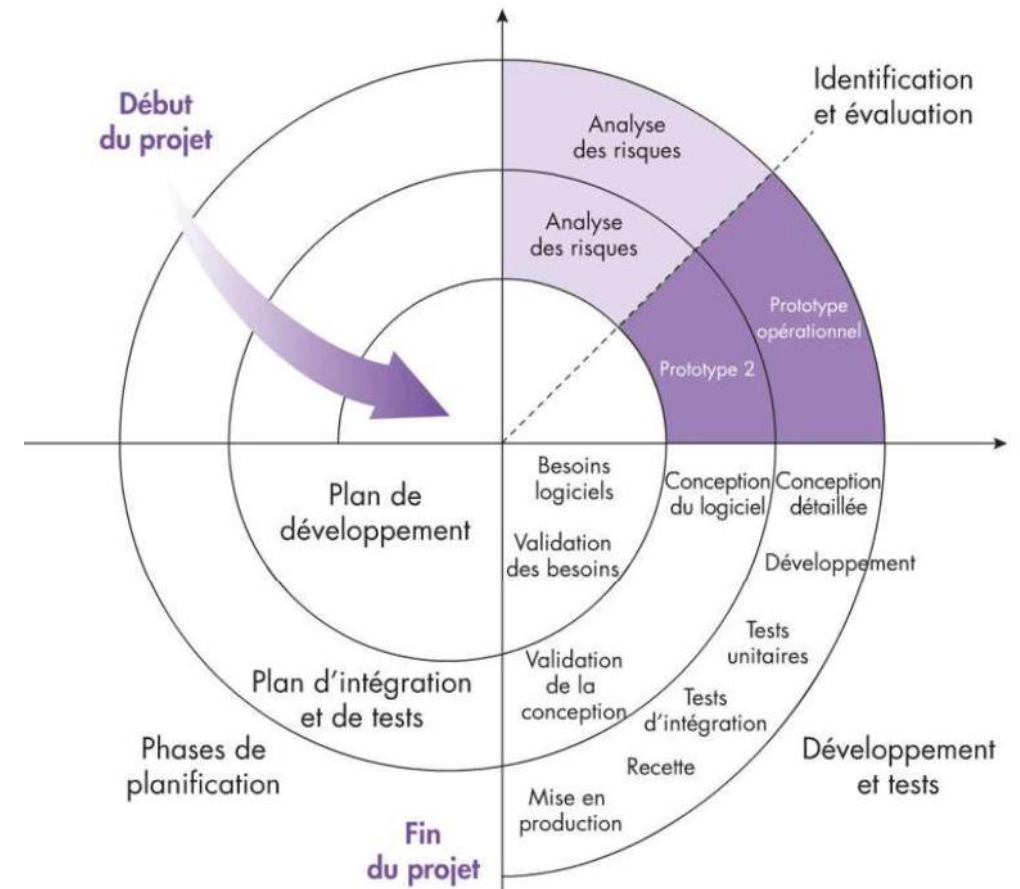
- ✓ Conception, prototypage, développement
- ✓ Tests (unitaires, intégration, etc.)

4. Évaluation & décision

- ✓ Validation par les parties prenantes
- ✓ Décider de continuer, ajuster ou arrêter le projet

Concrètement, le projet commence par :

- La définition des objectifs initiaux du système
- L'identification des besoins globaux (encore partiels)
- L'analyse des risques majeurs (techniques, fonctionnels, coûts, délais)



Modèle en spirale

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.4. Le modèle en spirale

Caractéristiques :

- Orienté risques
- Flexible
- Combine différentes approches selon les besoins de chaque itération

Avantages

- Bonne gestion de l'incertitude
- Flexibilité
- Détection précoce des problèmes
- Adapté aux projets complexes ou innovants

Limites

- Nécessite une expertise en gestion des risques
- Planification complexe
- Coûts d'analyse élevés
- Complexité pour les équipes peu expérimentées

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.4. Le modèle en spirale

1. Application au projet ENSAH

Tour 1 : Prototype du service d'inscription en ligne

- **Objectifs et alternatives :**
 - Objectif : permettre aux étudiants de s'inscrire en ligne pour le semestre.
 - Alternatives : utiliser un logiciel existant (SaaS) vs développer une application interne.
- **Évaluation des risques :**
 - Risque de sécurité des données.
 - Risque d'acceptation par le personnel administratif.
- **Développement et tests :**
 - Créer un prototype simple avec formulaire d'inscription et validation basique.
 - Tester avec un petit groupe d'étudiants.
- **Planification suivante :**
 - Collecter les retours pour améliorer l'interface, ajouter authentification, notifications email, etc.

Tour 2 : Gestion des dossiers étudiants

- **Objectifs et alternatives :**
 - Objectif : gérer les dossiers (notes, documents, archives).
 - Alternatives : base de données relationnelle vs solution cloud sécurisée.
- **Évaluation des risques :**
 - Risque de perte de données.
 - Risque de complexité pour les utilisateurs.
- **Développement et tests :**
 - Ajouter la gestion des dossiers au prototype initial.
 - Tests avec un département pilote.
- **Planification suivante :**
 - Intégrer un module de recherche, filtres, export PDF, sauvegardes automatiques.

Tour 3 : Services complémentaires et interface utilisateur

- **Objectifs et alternatives :**
 - Objectif : intégrer services comme notifications, demandes administratives, calendrier académique.
- **Évaluation des risques :**
 - Risque de surcharge fonctionnelle.
 - Risque d'interface confuse.
- **Développement et tests :**
 - Ajouter fonctionnalités progressivement.
 - Tests d'ergonomie et expérience utilisateur (UX).
- **Planification suivante :**
 - Préparer le déploiement complet et formation du personnel.

Tour 4 : Déploiement et maintenance

- **Objectifs et alternatives :**
 - Déploiement global pour tous les étudiants et le personnel.
- **Évaluation des risques :**
 - Risque technique (serveur, réseau).
 - Risque organisationnel (adaptation des utilisateurs).
- **Développement et tests :**
 - Mise en production.
 - Tests de charge et sécurité.
- **Planification suivante :**
 - Plan de maintenance, support et évolutions futures (ajout de modules par exemple pour la gestion des examens en ligne).

2. Avantages du modèle en spirale pour l'ENSAH

- Réduction des **risques élevés** dès le début.
- Possibilité de **valider chaque fonctionnalité** avant de passer à la suivante.
- Adaptation continue aux besoins réels des étudiants et du personnel.
- Déploiement **progressif et contrôlé**, ce qui est idéal pour une institution publique comme ENSAH.

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.5. Le modèle RAD

Le modèle RAD (Rapid Application Development) est un modèle de développement logiciel **itératif et incrémental** qui vise à **concevoir rapidement des applications fonctionnelles** grâce à des cycles courts, à l'utilisation de **prototypes** et à une **forte implication des utilisateurs** tout au long du projet.

Il privilégie la **rapidité de livraison**, l'**adaptation continue aux besoins** et le **développement modulaire**, plutôt qu'une planification détaillée et rigide dès le départ.

☞ En résumé : **RAD = développement rapide + prototypes + retours utilisateurs fréquents.**

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.5. Le modèle RAD

Avantages

Livraison rapide d'un premier produit fonctionnel

Réduction du risque d'un produit final inadapté

Flexibilité face aux changements de besoins

Meilleure satisfaction client grâce à l'implication continue

Inconvénients

Moins adapté aux très grands projets ou aux équipes dispersées

Nécessite une forte disponibilité du client

Peut souffrir d'un manque de documentation si mal encadré

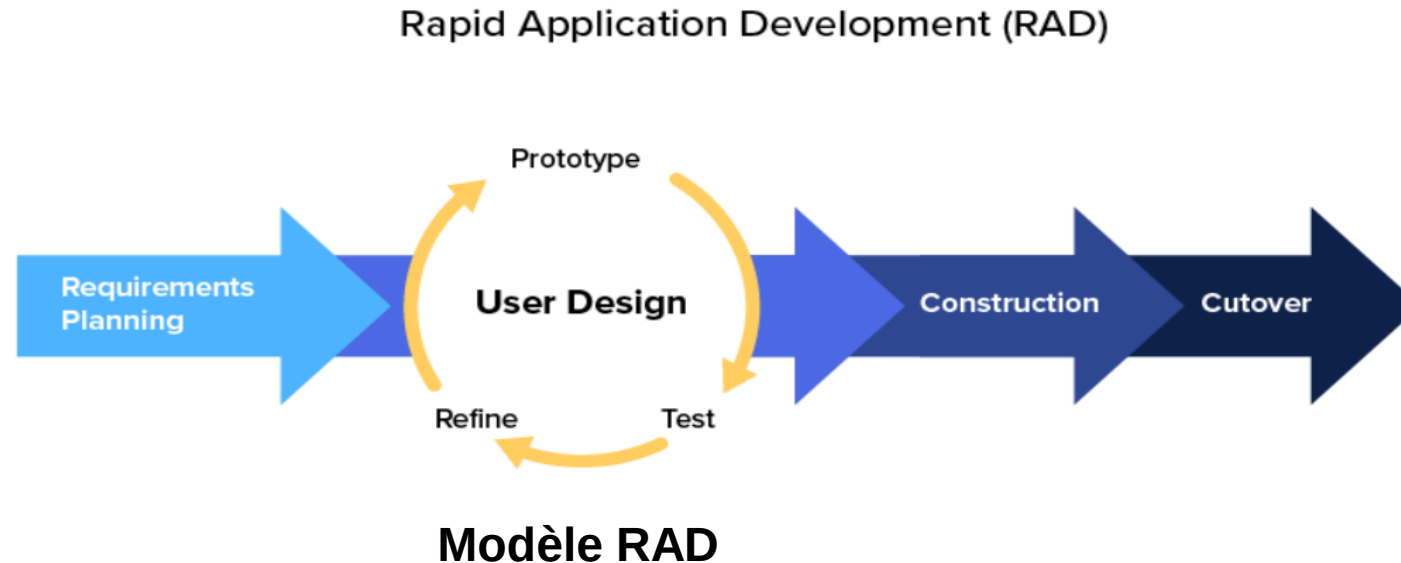
Risque de dérive du périmètre (scope creep)

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.5. Le modèle RAD

- 1. **Planification des besoins** — définition rapide des objectifs et contraintes avec toutes les parties prenantes
- 2. **Design orienté utilisateur** — ateliers collaboratifs, maquettes, prototypes interactifs
- 3. **Construction** — développement par itérations courtes avec feedback continu
- 4. **Transition** — tests, déploiement et formation



Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.6. Le cycle en Y

2TUP est une méthode de conduite de projet informatique basée sur le **Processus Unifié (UP)**, développée par Stéphane Rochet. Elle est particulièrement adaptée aux projets orientés objet utilisant **UML**.

2TUP est une approche **itérative et incrémentale** qui organise le développement logiciel en isolant les préoccupations fonctionnelles des contraintes techniques, jusqu'à leur intégration lors de la conception finale.

Avantages clés

- ❑ Permet de **gérer les changements** fonctionnels sans impacter l'architecture technique, et vice versa
- ❑ Favorise la **réutilisabilité** des composants techniques
- ❑ Bien adaptée aux projets avec des **contraintes techniques fortes**
- ❑ Structure claire pour les équipes travaillant en **parallèle**

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.6. Le cycle en Y

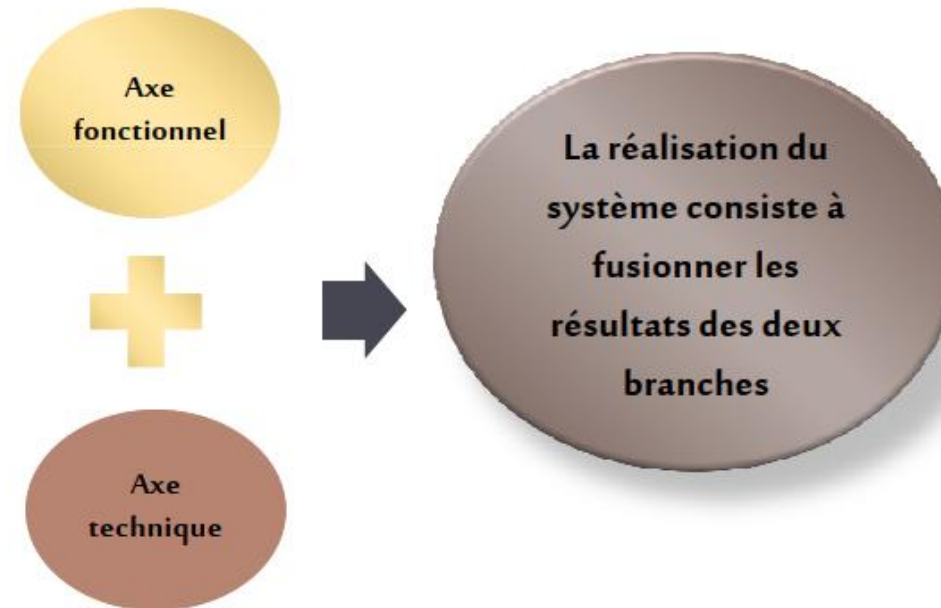
Principe

2 track signifie littéralement que le processus suit deux chemins :

- ☐ Fonctionnel
- ☐ Technique

Caractéristiques du 2 TUP

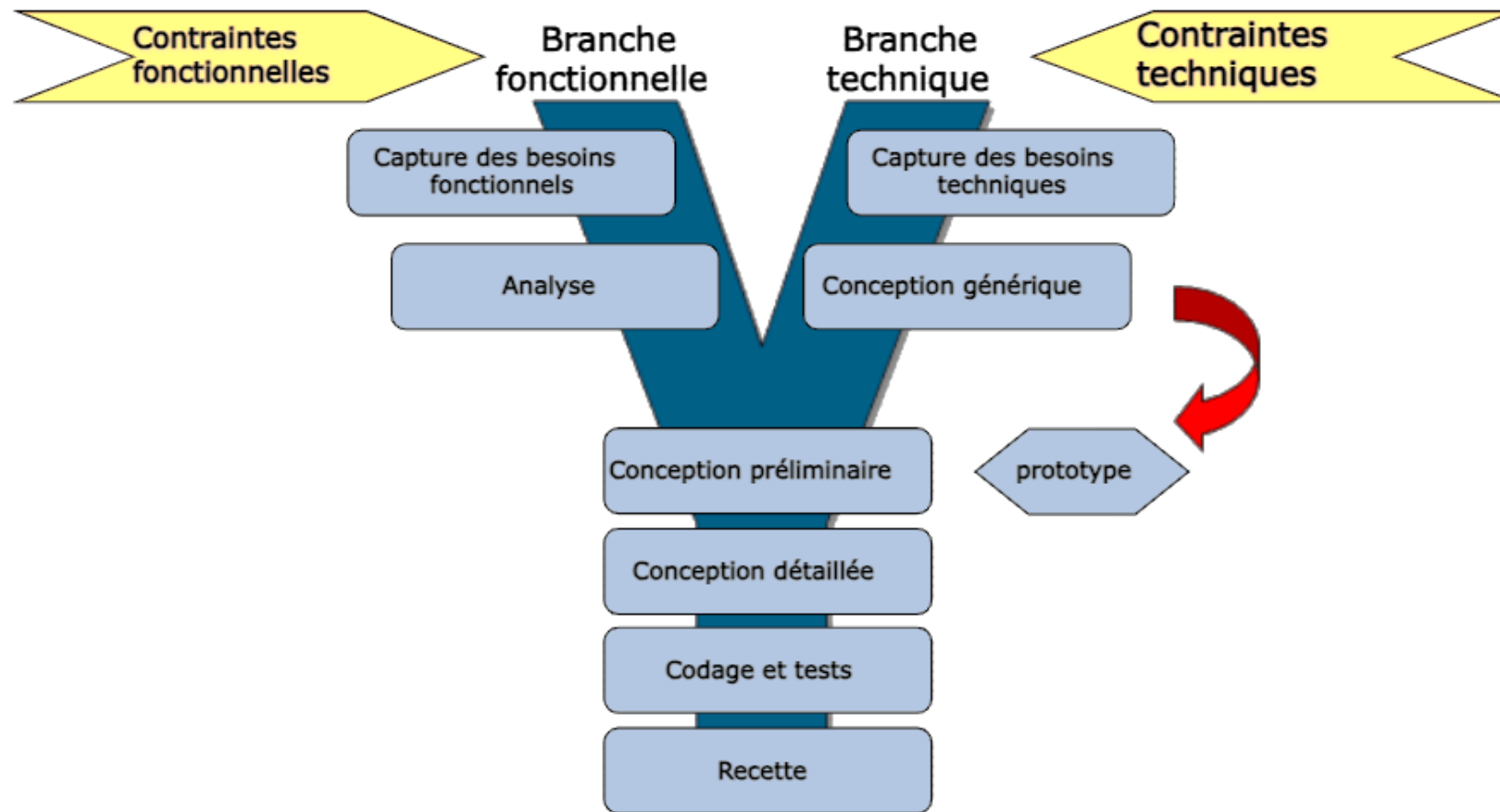
- ☐ Itératif et incrémental piloté par les risques
- ☐ Piloté par les exigences des utilisateurs
- ☐ Modélisé avec UML
- ☐ Centré sur l'architecture.



Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.6. Le cycle en Y



Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.6. Le cycle en Y

Branche fonctionnelle

Les principales étapes de la branche fonctionnelle se présentent comme suit :

❑ L'étape capture des besoins fonctionnels:

Cette phase a pour objectif de définir :

- ❑ La frontière fonctionnelle entre le système et son environnement.
- ❑ Les activités attendues des différents utilisateurs par rapport au système.

❑ L'étape d'analyse:

consiste à étudier précisément les spécifications fonctionnelles de manière à obtenir une idée de ce que va réaliser le système en terme de métier

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.6. Le cycle en Y

Branche technique

Les principales étapes de la branche technique se présentent comme suit

❑ L'étape capture des besoins techniques :

Cette étape recense toutes les contraintes sur les choix de technologies pour la conception du système. Les outils (et les Frameworks) et le matériel sélectionnés ainsi que la prise en compte des contraintes d'intégration avec l'existant (pré requis d'architecture technique)

❑ Conception générique:

Il s'agit du découpage en composants nécessaires à la construction de l'architecture technique. Il est généralement conseillé de réaliser un prototype pour assurer la validité de l'architecture.

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.6. Le cycle en Y

Phase conception - réalisation

Les principales étapes de cette branche se présentent comme suit :

❑ L'étape conception préliminaire :

Cette étape permet de produire le modèle de conception système. Ce dernier organise le système en composants, délivrant les services techniques et fonctionnels, Ce qui induit le regroupement des informations des branches technique et fonctionnelle.

❑ L'étape conception détaillée:

permet d'étudier comment réaliser chaque composant. le résultat fournit l'image prête { fabriquer du système complet.

❑ L'étape de codage :

permet d'effectuer la production des composants et les tests des unités de code au fur et à mesure de leur réalisation.

❑ L'étape de recette :

consiste à valider les fonctionnalités du système développé.

Chapitre 4 : Les cycles de vie

4.4. Le Cycle de Vie Itératif

4.4.7. Développement itératif vs. incrémental : clarification

- ❑ **Développement incrémental** : le système évolue par ajout progressif de fonctionnalités à chaque cycle. Chaque fonctionnalité est développée complètement en une seule fois.
- ❑ **Développement itératif** : le système est amélioré progressivement ; les mêmes fonctionnalités sont revues et enrichies sur plusieurs cycles successifs.
- ❑ **Approche combinée** : la plupart des projets modernes associent les deux approches, en ajoutant des fonctionnalités par incréments tout en les affinant de manière itérative, comme dans le RUP.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.1. L'émergence de l'agilité

À la fin des années 1990, les limites des approches traditionnelles ont conduit à l'émergence de méthodes de développement **agiles** adaptées aux projets innovants et aux besoins évolutifs (XP, Scrum, DSDM, FDD , Crystal...). En février 2001, des praticiens ont formalisé ces idées dans le **Manifeste Agile**, qui définit les valeurs et principes fondamentaux de l'agilité.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.2. Le Manifeste Agile

Le **Manifeste Agile** repose sur **quatre valeurs** qui privilégient :

- ☐ les individus et leurs interactions,
- ☐ les logiciels opérationnels,
- ☐ la collaboration avec le client,
- ☐ l'adaptation au changement, plutôt que les processus rigides, la documentation exhaustive, la négociation contractuelle et le suivi strict d'un plan.

Il est complété par douze principes mettant l'accent sur :

- ☐ la satisfaction du client par des livraisons fréquentes et à forte valeur,
- ☐ l'acceptation du changement, même tardif,
- ☐ la collaboration continue entre métiers et développeurs,
- ☐ la confiance accordée à des équipes motivées et auto-organisées,
- ☐ la communication directe,
- ☐ le logiciel fonctionnel comme principale mesure d'avancement,
- ☐ un rythme de travail soutenable,
- ☐ l'excellence technique, la simplicité et l'amélioration continue grâce à des rétrospectives régulières.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.3. Scrum

Scrum est le framework agile le plus utilisé, conçu dans les années 1990 par **Ken Schwaber** et **Jeff Sutherland**, visant à organiser le développement en cycles courts afin de livrer régulièrement de la valeur.

Scrum repose sur trois piliers : la **transparence** (tout le monde voit ce qui se passe), l'**inspection** (on vérifie régulièrement les progrès) et l'**adaptation** (on ajuste si nécessaire).

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.3. Scrum

Les rôles dans Scrum (3 rôles)	Les artefacts Scrum
1. Product Owner ☐ Représente le client / utilisateur ☐ Définit les besoins sous forme de Product Backlog ☐ Priorise les fonctionnalités selon la valeur métier et valide les livrables 2. Scrum Master ☐ Garant de la méthode Scrum ☐ Facilite le travail de l'équipe ☐ Supprime les obstacles (impediments) 3. Équipe de développement ☐ Équipe auto-organisée ☐ Développe, teste et livre le produit ☐ Multidisciplinaire (dev, test, UX, etc.)	Product Backlog ☐ Liste priorisée de toutes les fonctionnalités ☐ Exprimées sous forme de User Stories Sprint Backlog ☐ Tâches sélectionnées pour un sprint Incrément ☐ Version fonctionnelle du produit livrée à la fin du sprint

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.3. Scrum

Événements Scrum

Scrum fonctionne avec des **sprints**, typiquement de 1 à 4 semaines. Chaque sprint inclut :

Événement	Objectif
Sprint	☐ Durée fixe : 1 à 4 semaines ☐ Objectif clair à atteindre
Sprint Planning	Définir ce qui sera fait dans le sprint.
Daily Scrum (Daily Standup)	Réunion quotidienne (15 min) pour synchroniser l'équipe. ☐ 3 questions : ○ Quels travaux ont été accomplis lors de la période précédente ? ○ Quels travaux sont planifiés pour la prochaine itération ? ○ Quels risques, difficultés ou obstacles pourraient impacter la progression du projet ?
Sprint Review	Présenter l'incrément aux parties prenantes et recevoir du feedback.
Sprint Retrospective	L'équipe discute de ce qui a bien/mal fonctionné pour s'améliorer.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.3. Scrum

Avantages

- Livraisons fréquentes et valeur rapide pour le client
- Grande adaptabilité aux changements
- Transparence et meilleure visibilité sur l'avancement
- Responsabilisation de l'équipe
- Collaboration renforcée
- Réduction des risques grâce aux retours rapides

Limites

- Forte implication requise (notamment du Product Owner)
- Difficulté dans les organisations rigides
- Pression liée aux livraisons fréquentes et nécessité d'un changement culturel important
- Nécessite une équipe disciplinée et un Product Owner disponible

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.3. Scrum

I. Initialisation

Point de départ du projet

- ✓ Définition de la vision du produit
- ✓ Identification des besoins principaux
- ✓ Constitution du **Product Backlog** (liste priorisée des fonctionnalités)
- ✓ Mise en place de l'équipe Scrum
- On ne développe pas encore, on prépare le cadre du projet.

II. Itération « Backlog »

Sélection du travail à faire

- ✓ À partir du Product Backlog, l'équipe choisit les éléments à réaliser
- ✓ Ces éléments constituent le **Sprint Backlog**
- ✓ Cette sélection se fait lors de la **Sprint Planning**
- On décide quoi développer pendant le Sprint.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.3. Scrum

III. Sprint

Phase centrale du cycle Scrum

- ❑ Durée fixe (souvent 1 à 4 semaines)
 - ❑ L'équipe :
 - conçoit
 - développe
 - teste
 - ❑ Le but est de produire un **incrément fonctionnel** du produit
- Chaque Sprint = une mini-version utilisable du produit.

IV. Point quotidien (Daily Scrum)

Boucle au-dessus du Sprint

- ❑ Réunion quotidienne (≈ 15 minutes)
 - ❑ Chaque membre répond à :
 - Qu'ai-je fait hier ?
 - Que vais-je faire aujourd'hui ?
 - Ai-je des blocages ?
- Sert à **synchroniser l'équipe**, pas à faire des rapports.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.3. Scrum

V. Mise en production

Livraison du produit

- ☐ L'incrément peut être :
 - livré aux utilisateurs
 - ou simplement prêt à être livré
 - ☐ En Scrum, la mise en production peut se faire **à chaque Sprint**
- Livraison fréquente = valeur rapide pour le client.

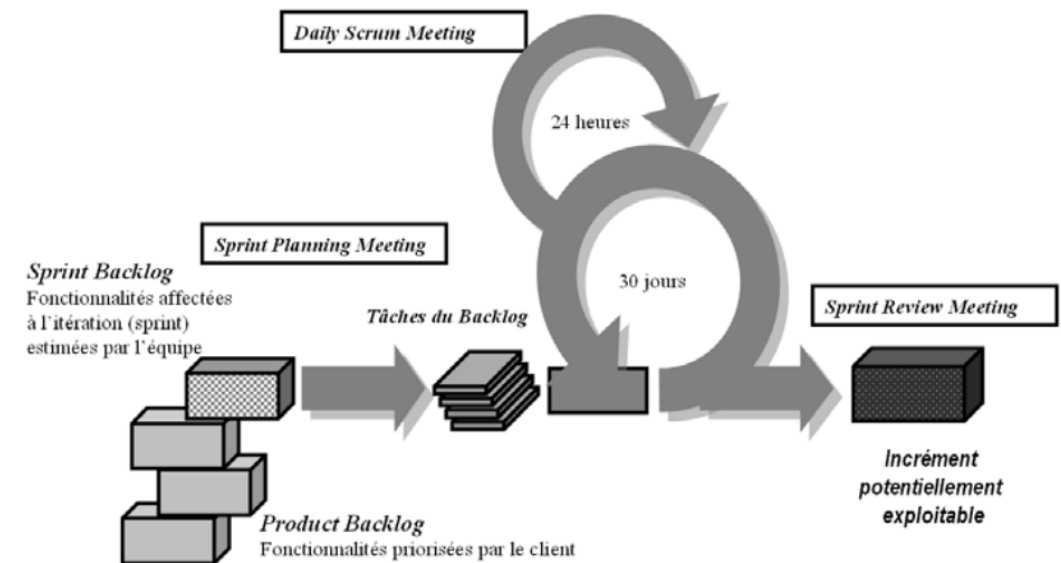
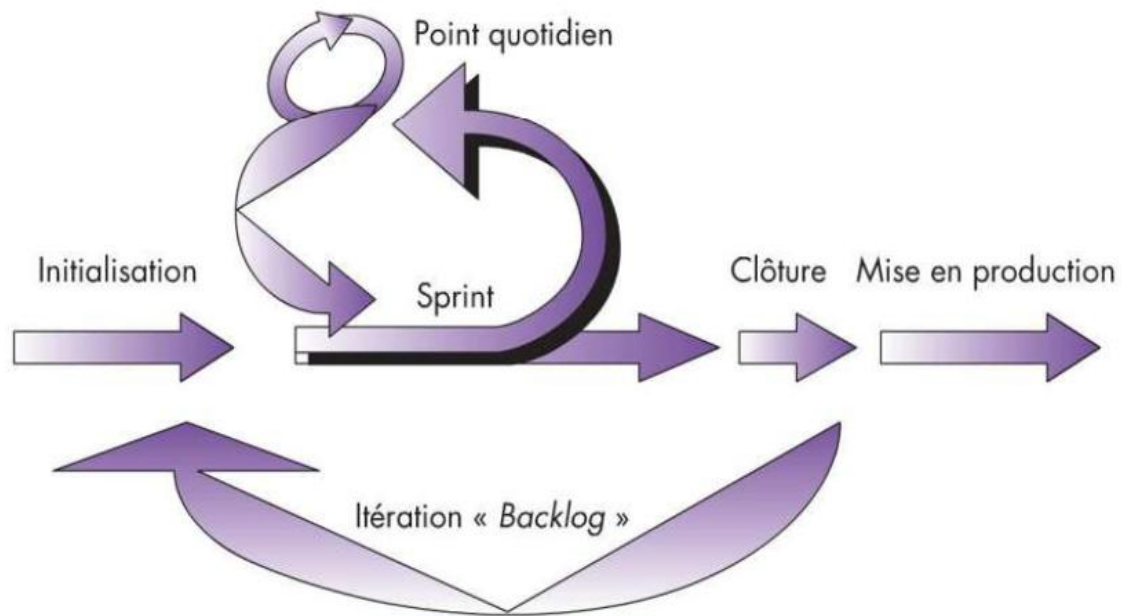
VI. ↻ Logique globale du schéma

- ☐ Scrum est **itératif et incrémental**
- ☐ Chaque Sprint :
 - répète le même cycle
 - enrichit progressivement le produit
- ☐ Le Backlog évolue en permanence selon les retours

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.3. Scrum



Source : Rally Software Development

Cycle Scrum

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.4. Extreme Programming (XP)

Extreme Programming (XP) est une méthode agile axée sur les **pratiques d'ingénierie** et la **qualité technique**, créée par **Kent Beck**. Elle pousse à l'extrême les bonnes pratiques de développement.

❑ **Valeurs XP** : communication, simplicité, feedback rapide, courage et respect.

❑ **Pratiques principales** :

- **Retour d'information** : programmation en binôme, tests unitaires écrits avant le code, intégration continue, planification collaborative, forte implication du client.
- **Processus continu** : petites livraisons fréquentes, refactoring permanent, rythme soutenable.
- **Compréhension partagée** : conception simple, métaphore système, propriété collective du code, standards de codage.
- **Bien-être** : semaine de 40 heures pour éviter l'épuisement.

❑ **Cycle XP** : itérations courtes (1 à 3 semaines) comprenant planification, développement en binôme avec tests d'abord, intégration continue, tests d'acceptation et livraison.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.4. Extreme Programming (XP)

Les 5 Valeurs Fondamentales

XP ne repose pas seulement sur du code, mais sur une culture d'équipe :

- ❑ **Communication** : Tout le monde se parle. Le client est souvent présent dans les bureaux.
- ❑ **Simplicité** : On fait ce qui est nécessaire aujourd'hui, pas ce qu'on *pense* être utile dans 6 mois.
- ❑ **Feedback** : Des cycles très courts pour corriger le tir immédiatement.
- ❑ **Courage** : Le courage consiste à supprimer un code défaillant et à refactoriser des structures complexes afin d'améliorer la qualité du système.
- ❑ **Respect** : Entre les développeurs et envers le travail fourni.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.4. Extreme Programming (XP)

Pratiques clés

Côté développement

- ❑ **TDD (Test-Driven Development)** — on écrit le test avant le code
- ❑ **Pair Programming** — deux développeurs travaillent ensemble sur le même poste
- ❑ **Refactoring continu** — amélioration constante du code sans changer son comportement
- ❑ **Intégration continue** — le code est intégré et testé plusieurs fois par jour
- ❑ **Collective Code Ownership** — tout le monde peut modifier n'importe quelle partie du code
- ❑ **Coding Standards** — conventions partagées par toute l'équipe

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.4. Extreme Programming (XP)

Pratiques clés

Côté planification

- ❑ **Small Releases** — Livraisons fréquentes de petits incréments fonctionnels
- ❑ **Planning Game** — le client et l'équipe négocient les fonctionnalités à chaque itération
- ❑ **Itérations courtes** — cycles de 1 à 2 semaines

Côté organisation

- ❑ **On-site Customer** — un représentant client disponible en permanence
- ❑ **40-hour week** — pas de surmenage, pour garder un rythme durable
- ❑ **Métaphore système** — une vision commune et simple de l'architecture

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.4. Extreme Programming (XP)

1) Analyse technique

Objectif : comprendre le problème à résoudre d'un point de vue technique

- Étude des besoins exprimés par le client
- Identification des contraintes techniques (architecture, performances, sécurité...)
- Découpage du système en fonctionnalités simples

Contrairement aux méthodes classiques, l'analyse n'est pas exhaustive dès le départ.

2) Planification

Objectif : décider quoi développer en priorité

- Sélection des fonctionnalités à forte valeur métier
- Estimation de l'effort (en points, jours, itérations)
- Construction d'un plan court terme

La planification est révisée régulièrement, pas figée.

3) Itération de développement ↻

C'est le cœur de XP

Chaque itération est courte (1 à 2 semaines) et comprend :

- Conception simple
- Codage
- Tests automatisés (écrits avant ou pendant le code)
- Refactoring (amélioration continue du code)

4) Mise en production

Objectif : livrer une version fonctionnelle

- Déploiement fréquent (souvent à la fin de chaque itération)
- Le client peut tester et donner un feedback réel

En XP, on livre tôt et souvent.

5) Maintenance

Objectif : faire évoluer le système

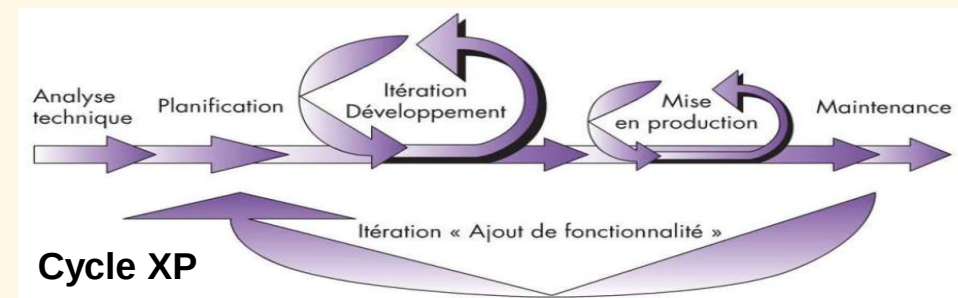
- Correction des bugs
- Amélioration des performances
- Adaptation aux nouveaux besoins

La maintenance est intégrée naturellement au cycle.

6) Grande itération « Ajout de fonctionnalité »

- Chaque cycle ajoute de nouvelles fonctionnalités
- Le produit s'enrichit progressivement

Le système évolue par incréments successifs.



Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.4. Extreme Programming (XP)

Avantages

- Code de haute qualité
- Réduction des bugs
- Forte adaptabilité au changement
- Partage des connaissances
- Rythme de travail durable

Limites

- Forte discipline requise
- Acceptation difficile du pair programming
- Implication client parfois irréaliste
- Difficulté à passer à l'échelle
- Résistance culturelle aux pratiques jugées « extrêmes ».

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.4. Extreme Programming (XP)

Pourquoi choisir XP ?

C'est la méthode idéale pour les projets où **les besoins changent très vite** et où **le risque technique est élevé**. Elle réduit drastiquement le coût du changement et produit un code d'une propreté exemplaire.

Le saviez-vous ?

XP est souvent critiquée par les managers car le "Pair Programming" donne l'impression de payer deux personnes pour faire le travail d'une seule. Pourtant, les études montrent que la réduction des bugs et de la dette technique compense largement ce coût.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.5. Feature-Driven Development (FDD)

Le **FDD** est une méthode agile créée par **Jeff De Luca et Peter Coad** en 1997, conçue pour des projets de grande envergure. Contrairement à XP ou Scrum, elle structure le développement autour des **fonctionnalités métier** (features) et met l'accent sur la **planification rigoureuse** et la **progression mesurable**.

Valeurs & Principes

Le FDD repose sur l'idée que :

- ✓ Le système doit être décomposé en **fonctionnalités concrètes et utiles** pour le client
- ✓ Chaque feature doit être **livrable en moins de 2 semaines**
- ✓ La progression doit être **visible et mesurable** à tout moment

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.5. Feature-Driven Development (FDD)

Les 5 Processus FDD

Développer un modèle global (Develop an Overall Model)

- L'équipe et les experts métier construisent ensemble une **vue d'ensemble du domaine** (diagrammes de classes, modèles objets). C'est une phase de compréhension collective du système.

Construire la liste des fonctionnalités (Build a Features List)

- On décompose le système en **domaines** → **activités métier** → **features**

Planifier par fonctionnalité (Plan by Feature)

- Les features sont **priorisées et assignées** aux développeurs principaux (Chief Programmers). On établit un plan de livraison basé sur les dépendances et la complexité.

Concevoir par fonctionnalité (Design by Feature)

- Pour chaque feature, l'équipe produit des **diagrammes de séquence**, révisé le modèle objet et prépare les classes à implémenter. **Durée : quelques jours.**

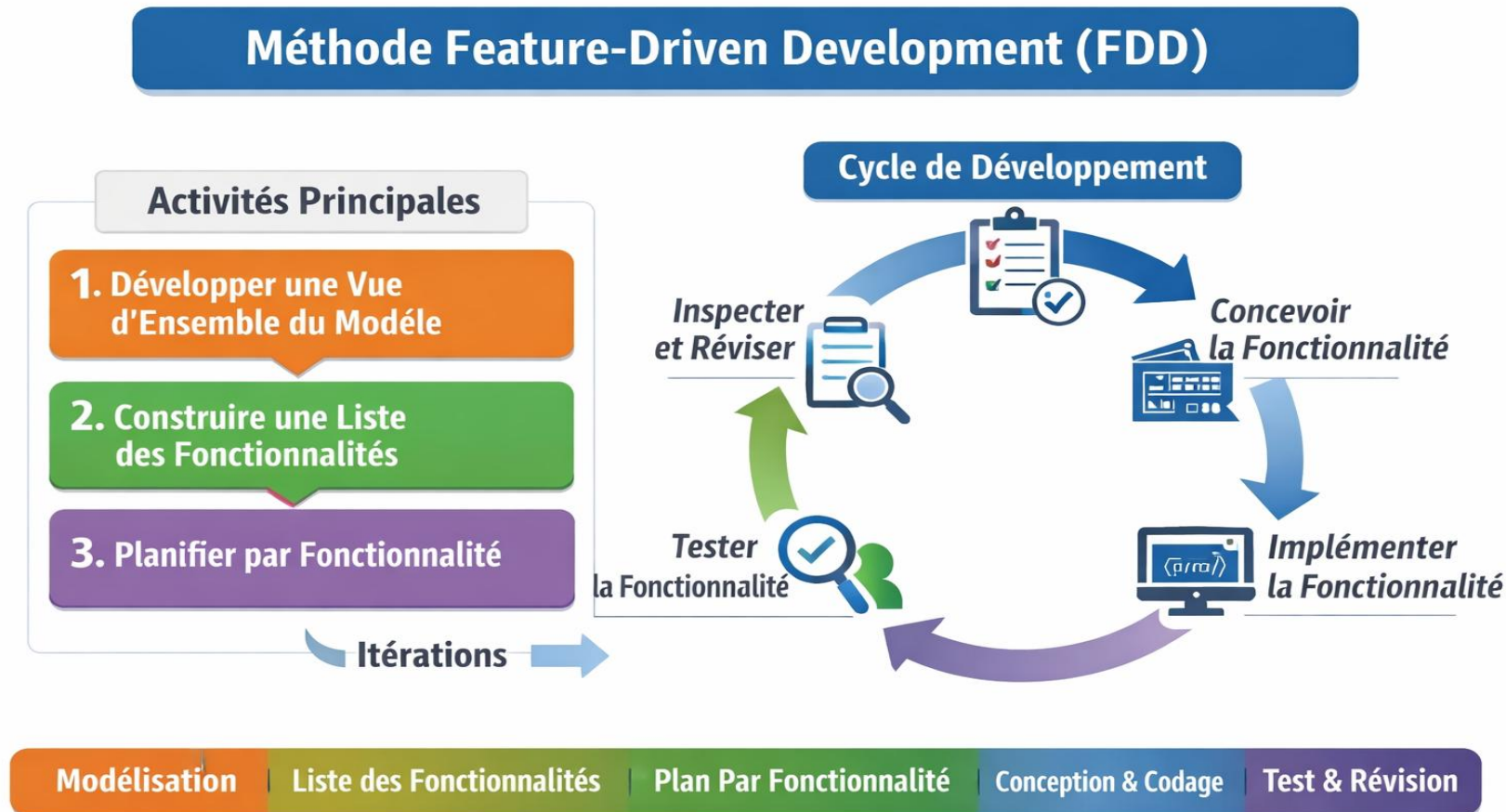
Construire par fonctionnalité (Build by Feature)

- Le code est **implémenté, testé et intégré**. Une feature terminée = code revu + tests passés + intégrée dans le build principal.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.5. Feature-Driven Development (FDD)



Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.5. Feature-Driven Development (FDD)

Les rôles principaux

Rôle	Responsabilité
Chef de projet	Responsable administratif et financier du projet. Il gère les ressources, le budget, le calendrier et fait le lien avec les parties prenantes externes.
Architecte en chef	Responsable de la conception globale du système. Il prend les décisions d'architecture techniques et préside les sessions de conception
Directeur du développement	Il encadre l'équipe de développement au quotidien, résout les conflits et s'assure que les activités de développement se déroulent correctement.
Chef programmeur	C'est un développeur senior qui dirige une petite équipe. Il participe à la planification, à la conception et supervise le développement des fonctionnalités (features). C'est un rôle central dans FDD.
Propriétaire de classes	Développeur responsable de la conception, du codage, des tests et de la documentation d'une classe spécifique. Il travaille sous la supervision du Programmeur en chef.
Expert métier	Représente le client ou l'utilisateur final. Il fournit les exigences et les connaissances métier nécessaires à la définition des fonctionnalités.
Responsable qualité	Suivi des standards et des tests

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.5. Feature-Driven Development (FDD)

Les pratiques techniques

- ❑ **Modélisation de domaine par objets** — compréhension commune du métier
- ❑ **Développement par fonctionnalité** — unité de travail atomique
- ❑ **Propriété individuelle du code** (Class Ownership) — chaque classe a un responsable
- ❑ **Équipes par fonctionnalité** (Feature Teams) — petites équipes pluridisciplinaires
- ❑ **Inspections** — revues de code et de conception formelles
- ❑ **Gestion de configuration** — contrôle de version rigoureux
- ❑ **Builds réguliers** — intégration fréquente
- ❑ **Reporting par couleur** (Color Reporting) — suivi visuel de l'avancement (Rouge / Jaune / Vert)

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.5. Feature-Driven Development (FDD)

Avantages & Limites

✓ Avantages

- ✓ Adapté aux **grands projets** et **grandes équipes**
- ✓ Progression **transparente** et **mesurable**
- ✓ Centré sur la **valeur métier**
- ✓ Bon équilibre entre **rigueur** et **agilité**

⚠ Limites

- Dépend fortement de **développeurs expérimentés**
- Moins adapté aux **petites équipes**
- La phase de modélisation peut être **longue**
- Peu de pratiques techniques intégrées (pas de TDD, pas de pair programming obligatoire)

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.6. Dynamic Systems Development Method (DSDM)

DSDM est un **cadre agile de gestion de projet** créé en 1994 par un consortium d'entreprises britanniques. C'est l'une des premières méthodes agiles formalisées, bien avant le Manifeste Agile (2001). Elle est aujourd'hui maintenue par l'**Atern consortium** et très utilisée au Royaume-Uni, notamment dans le secteur public.

Son principe fondateur :

"Le projet s'adapte aux contraintes fixes de temps et de budget — pas l'inverse."

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.6. Dynamic Systems Development Method (DSDM)

Les 8 principes DSDM

1. **Se concentrer sur le besoin métier** — toute décision doit servir la valeur business
2. **Livrer dans les délais** — respecter les échéances est non négociable
3. **Collaborer** — équipes intégrées client/développeur en continu
4. **Ne jamais compromettre la qualité** — le niveau de qualité est fixé dès le départ
5. **Construire de manière incrémentale** — livrer par petits morceaux validés
6. **Développer de manière itérative** — améliorer en cycles successifs
7. **Communiquer en continu** — favoriser les échanges directs et fréquents
8. **Démontrer le contrôle** — visibilité et gouvernance à tout moment

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.6. Dynamic Systems Development Method (DSDM)

Le cycle de vie DSDM

Pre-project → Feasibility → Foundations → Exploration → Engineering → Deployment → Post-project



Phase	Description
Pre-project	Identification du projet, faisabilité initiale
Feasibility	Étude rapide de faisabilité technique et métier (quelques jours)
Foundations	Définition des exigences de haut niveau, architecture, planning
Exploration	Développement itératif des fonctionnalités (investigation)
Engineering	Finalisation, tests et préparation au déploiement
Deployment	Mise en production, formation utilisateurs
Post-project	Bilan, mesure des bénéfices métier

La Timebox — unité de base

Une **timebox** est une période fixe (typiquement **2 à 4 semaines**) au sein de laquelle l'équipe livre un incrément. Elle suit un schéma interne strict : **Kick-off** → **Investigation** → **Raffinement** → **Consolidation** → **Clôture**

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.6. Dynamic Systems Development Method (DSDM)

Le principe MoSCoW

DSDM utilise la **priorisation MoSCoW** pour gérer le périmètre :

Priorité	Signification	% recommandé
M — Must have	Indispensable, non négociable	~60%
S — Should have	Important mais pas critique	~20%
C — Could have	Souhaitable si le temps le permet	~20%
W — Won't have	Hors périmètre pour cette itération	—

L'idée clé :

si le temps manque, on livre les **Must** et **Should**, et on reporte les **Could**. On ne dépasse jamais le délai.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.6. Dynamic Systems Development Method (DSDM)

Les rôles DSDM

Côté Métier (Business Side)

1. **Sponsor Exécutif (Executive Sponsor)** Le champion du projet au plus haut niveau. Il détient l'autorité et le budget, prend les décisions stratégiques et lève les obstacles organisationnels.
2. **Visionnaire (Visionary)** Il porte la vision globale du projet et s'assure que la solution finale correspond aux objectifs métier. C'est souvent l'initiateur du projet.
3. **Ambassadeur Utilisateur (Ambassador User)** Représentant des utilisateurs finaux au sein de l'équipe. Il fournit les exigences du quotidien, participe aux ateliers et valide les livrables. C'est un rôle **critique** en DSDM.
4. **Conseiller Utilisateur (Advisor User)** Utilisateur final impliqué ponctuellement pour donner un avis spécifique sur certaines fonctionnalités ou décisions. Il complète l'Ambassadeur Utilisateur.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.6. Dynamic Systems Development Method (DSDM)

Les rôles DSDM

Côté Solution (Technical Side)

5. **Chef de projet (Project Manager)** Coordonne le projet, gère le planning, les risques et la communication entre les parties. Il travaille en binôme avec le Business Visionary.
6. **Coordinateur Technique (Technical Coordinator)** Responsable de l'architecture et de la cohérence technique. Il définit les standards de développement et s'assure de la qualité technique de la solution.
7. **Chef d'équipe (Team Leader)** Il anime l'équipe au quotidien, facilite la collaboration et s'assure que l'équipe travaille efficacement dans les délais impartis.
8. **Développeur de solution (Solution Developer)** Conçoit et construit la solution selon les exigences définies. Il travaille en étroite collaboration avec le testeur et l'ambassadeur utilisateur.
9. **Testeur de solution (Solution Tester)** Responsable des tests techniques et fonctionnels. Il vérifie que la solution livrée correspond aux critères d'acceptation définis.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.6. Dynamic Systems Development Method (DSDM)

Les pratiques clés

- ❑ **Timeboxing** — chaque itération a une durée fixe et non extensible
- ❑ **MoSCoW prioritization** — gestion du périmètre par priorités
- ❑ **Prototypage** — maquettes et prototypes pour valider rapidement
- ❑ **Ateliers facilités** — sessions collaboratives intensives (workshops)
- ❑ **Modélisation** — diagrammes légers pour partager la compréhension
- ❑ **Tests continus** — qualité intégrée dès le début, pas en fin de projet
- ❑ **Développement itératif et incrémental** — livraisons fréquentes et validées

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.6. Dynamic Systems Development Method (DSDM)

Ce qui distingue XP des autres

XP est la méthode la plus **orientée ingénierie**. Là où Scrum et DSDM s'intéressent surtout à l'organisation et à la gouvernance, XP impose des **pratiques techniques strictes** :

- ❑ **TDD** — Test-Driven Development
- ❑ **Pair Programming** — deux développeurs sur le même code
- ❑ **Intégration continue** — builds et tests automatisés en permanence
- ❑ **Refactoring** — amélioration continue du code
- ❑ **Client embarqué** — le client est physiquement présent dans l'équipe
- ❑ **Petites releases** — livraisons très fréquentes
- ❑ **Propriété collective du code** — tout le monde peut modifier n'importe quelle partie

XP et Scrum sont souvent **combinés** en pratique : Scrum pour le cadre organisationnel, XP pour les pratiques techniques.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.6. Dynamic Systems Development Method (DSDM)

Avantages et limites

✓ Avantages

- ✓ Cadre **complet et structuré** couvrant tout le cycle de vie
- ✓ Forte **gouvernance** rassurante pour les organisations et le secteur public
- ✓ **MoSCoW** permet de toujours livrer à temps sans sacrifier la qualité
- ✓ Très bonne **intégration avec PRINCE2** (méthode de gestion de projet très répandue)
- ✓ Implication forte et continue du **client/métier**

✗ Limites

- ❑ Peut sembler **trop lourd** pour les petites équipes ou startups
- ❑ Courbe d'apprentissage plus steep que Scrum
- ❑ Moins populaire internationalement → **communauté plus restreinte**
- ❑ Nécessite un **engagement fort** des parties prenantes métier, pas toujours disponibles
- ❑ Peu adapté aux projets très **exploratoires ou innovants**

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.7. Scaling Agile : SAFe, LeSS, Nexus

L'agilité, initialement conçue pour de **petites équipes**, a été étendue aux **grandes organisations** grâce à des frameworks de **scaling agile**.

- ❑ **SAFe (Scaled Agile Framework)** : framework le plus répandu pour déployer l'agilité à grande échelle. Il définit plusieurs niveaux (équipe, programme, large solution, portefeuille) avec des rôles, événements et artefacts précis. Très structuré, il facilite la coordination dans les grandes organisations.
- ❑ **LeSS (Large-Scale Scrum)** : approche minimaliste qui étend Scrum à plusieurs équipes travaillant sur un même produit, en limitant les structures supplémentaires et en privilégiant la simplicité.
- ❑ **Nexus** : framework créé par les auteurs de Scrum pour coordonner **3 à 9 équipes Scrum** autour d'un même produit.

Ces frameworks visent à conserver les bénéfices de l'agilité tout en répondant aux contraintes de coordination et de gouvernance des grandes structures.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.8. Avantages généraux des méthodes agiles

Les **méthodes agiles** offrent une forte **adaptation au changement**, des **livraisons fréquentes de valeur**, et une **collaboration étroite avec le client** garantissant l'alignement sur les besoins réels. Elles apportent **visibilité et transparence**, facilitent la **détection précoce des problèmes**, encouragent l'**amélioration continue**, renforcent la **motivation et l'autonomie des équipes**, et améliorent la **qualité technique** grâce aux bonnes pratiques d'ingénierie.

Chapitre 4 : Les cycles de vie

4.5. Les Méthodes Agiles

4.5.9. Limites et défis de l'agilité

Les **méthodes agiles**, malgré leurs bénéfices, présentent des **défis** :

- ❑ Engagement élevé requis de toutes les parties, surtout du Product Owner,
- ❑ Difficulté dans les organisations hiérarchiques ou bureaucratiques,
- ❑ Manque de vision long terme détaillée pour certains projets,
- ❑ Documentation parfois insuffisante pour la maintenance,
- ❑ Complexité du **scaling** à grande échelle,
- ❑ Contraintes réglementaires pouvant nécessiter plus de formalisme,
- ❑ Risque de fatigue de l'équipe dû au rythme soutenu et à la pression constante de livrer.

Chapitre 4 : Les cycles de vie

4.6. Les Bonnes Pratiques

4.6.1. Kanban

Kanban est une méthode inspirée du **Toyota Production System**, centrée sur la **visualisation du travail** et la **limitation du travail en cours (WIP)** afin d'améliorer le flux.

- ❑ **Principes** : améliorer l'existant sans rupture brutale, conserver les rôles actuels, encourager le leadership à tous les niveaux, et viser une amélioration incrémentale continue.
- ❑ **Pratiques clés** : visualiser le flux via un tableau Kanban, limiter le WIP, gérer et optimiser le flux, expliciter les règles de processus, mettre en place des boucles de feedback, améliorer collaborativement.
- ❑ **Tableau Kanban** : colonnes représentant les étapes (À faire, En cours, En test, Terminé) et cartes déplacées de gauche à droite ; les limites WIP évitent la surcharge et favorisent la finalisation des tâches.
- ❑ **Métriques** : lead time, cycle time, débit, work in progress, utilisées pour piloter l'amélioration continue.

Chapitre 4 : Les cycles de vie

4.6. Les Bonnes Pratiques

4.6.1. Kanban

Les 4 piliers

1. **Visualiser le travail** Toutes les tâches sont représentées sur un tableau divisé en colonnes (ex : *À faire* → *En cours* → *Terminé*).
2. **Limiter le travail en cours (WIP)** Chaque colonne a un nombre maximum de tâches simultanées. Cela évite la surcharge et force à finir avant de commencer.
3. **Gérer le flux** On surveille comment les tâches progressent, on détecte les goulots d'étranglement et on les élimine.
4. **Amélioration continue (Kaizen)** L'équipe s'améliore régulièrement en analysant ses métriques (temps de cycle, débit, etc.)

Chapitre 4 : Les cycles de vie

4.6. Les Bonnes Pratiques

4.6.1. Kanban

Le tableau Kanban typique

À faire	En cours (max 3)	En révision	Terminé
Tâche A	Tâche C	Tâche E	Tâche F
Tâche B	Tâche D		Tâche G

Métriques clés

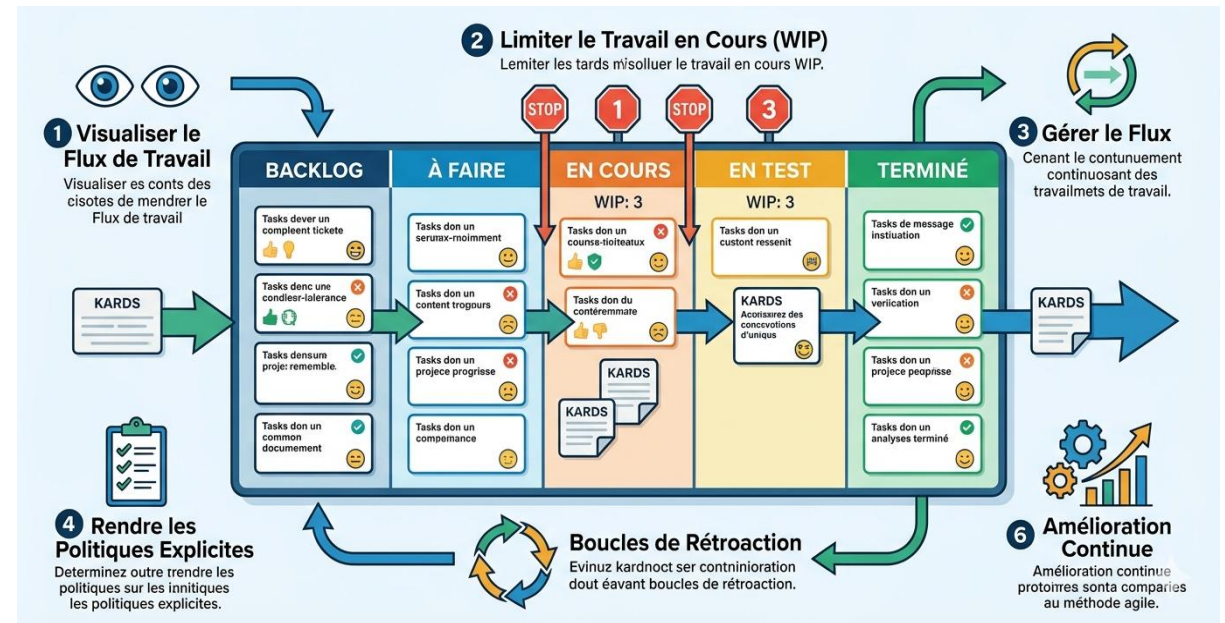
- ❑ **Lead time** : temps entre la demande et la livraison
- ❑ **Cycle time** : temps d'exécution active d'une tâche
- ❑ **Débit (throughput)** : nombre de tâches terminées par période

Chapitre 4 : Les cycles de vie

4.6. Les Bonnes Pratiques

4.6.1. Kanban

Kanban est une **bonne pratique à caractère agile**. Il est agile par ses valeurs et son orientation vers la livraison continue de valeur, mais il reste une bonne pratique par son origine empirique, son absence de cadre normatif officiel et sa flexibilité totale d'adoption. C'est précisément cette **double nature** qui explique sa popularité : il peut être adopté seul, combiné avec Scrum (*Scrumban*), ou intégré dans un framework d'échelle comme **SAFe**, sans jamais imposer de rupture organisationnelle brutale.



Chapitre 4 : Les cycles de vie

4.6. Les Bonnes Pratiques

4.6.1. Kanban

Avantages :

- ☐ Forte visibilité
- ☐ Flexibilité
- ☐ Amélioration continue basée sur les données
- ☐ Limitation de la surcharge
- ☐ Mise en place progressive

Différence avec Scrum :

Kanban fonctionne en **flux continu** sans rôles ni événements prescrits, tandis que Scrum s'appuie sur des **sprints**, des rôles définis et des événements réguliers.

Chapitre 4 : Les cycles de vie

4.6. Les Bonnes Pratiques

4.6.2. PRINCE2 (Projects IN Controlled Environments)

PRINCE2 est une méthode de gestion de projet structurée, développée au Royaume-Uni et largement utilisée en Europe, reposant sur une approche très processée et documentée.

- ❑ **Principes** : justification continue du projet, apprentissage par l'expérience, rôles clairs, gestion par phases et par exception, focalisation sur les produits, adaptation au contexte.
- ❑ **Structure** : sept processus (élaborer, diriger, initialiser, contrôler séquence, gérer livraison, gérer limite de séquence, clore le projet) et sept thèmes transversaux (cas d'affaire, organisation, qualité, plans, risques, changement, progrès).

Chapitre 4 : Les cycles de vie

4.6. Les Bonnes Pratiques

4.6.2. PRINCE2 (Projects IN Controlled Environments)

PRINCE2 repose sur **7 principes** non négociables qui définissent l'esprit de la méthode :

1. **Justification commerciale continue** — le projet doit avoir une raison d'être valable tout au long de son cycle de vie (Business Case)
2. **Apprendre de l'expérience** — retours d'expérience capitalisés à chaque étape
3. **Rôles et responsabilités définis** — chaque acteur a un rôle clair
4. **Management par séquences** — le projet est découpé en phases contrôlables
5. **Management par exception** — délégation avec des tolérances définies
6. **Focalisation sur les produits** — on gère des livrables, pas des activités
7. **Adaptation au contexte** — la méthode s'adapte à la taille et à la complexité du projet

Chapitre 4 : Les cycles de vie

4.6. Les Bonnes Pratiques

4.6.2. PRINCE2 (Projects IN Controlled Environments)

Les 7 thèmes

Les thèmes répondent à la question "*Comment gérer ?*" en continu :

Thème	Objet
Business Case	Justification et valeur du projet
Organisation	Structure et responsabilités
Qualité	Définition et contrôle des livrables
Plans	Niveaux de planification
Risques	Identification et réponse
Changements	Contrôle des modifications
Progression	Suivi de l'avancement

Chapitre 4 : Les cycles de vie

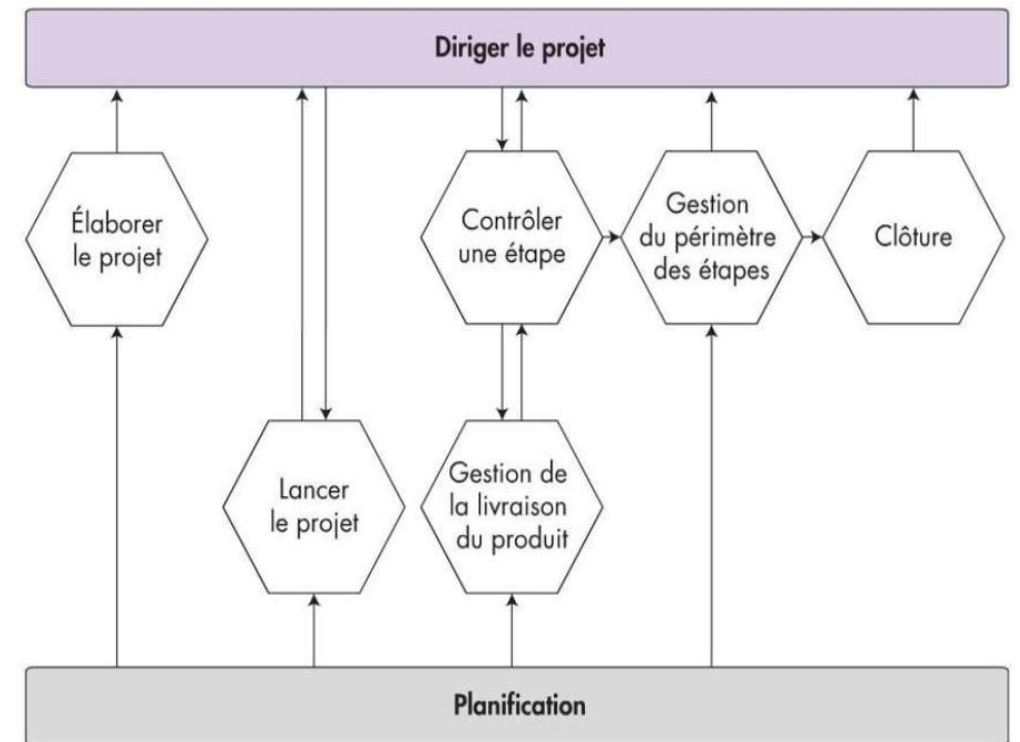
4.6. Les Bonnes Pratiques

4.6.2. PRINCE2 (Projects IN Controlled Environments)

Les 7 processus

Les processus décrivent **"Quoi faire et quand ?"** :

1. **Élaborer le projet** (*Starting Up a Project*) — faisabilité initiale
2. **Diriger le projet** (*Directing a Project*) — rôle du Comité de pilotage
3. **Initialiser le projet** (*Initiating a Project*) — documentation de référence (PID)
4. **Contrôler une séquence** (*Controlling a Stage*) — suivi quotidien par le chef de projet
5. **Gérer la livraison des produits** (*Managing Product Delivery*) — travail des équipes
6. **Gérer une limite de séquence** (*Managing a Stage Boundary*) — bilan et autorisation de passer à la phase suivante
7. **Clore le projet** (*Closing a Project*) — bilan final, transfert, leçons apprises



Méthodologie PRINCE2

Chapitre 4 : Les cycles de vie

4.6. Les Bonnes Pratiques

4.6.3. PMBOK (Project Management Body of Knowledge)

Le **PMBOK**, publié par le PMI, est un référentiel largement utilisé en gestion de projet, y compris en informatique. Il organise la gestion de projet en **cinq groupes de processus** (démarrage, planification, exécution, surveillance/maîtrise, clôture) et **dix domaines de connaissance** (intégration, périmètre, échéancier, coûts, qualité, ressources, communications, risques, approvisionnements, parties prenantes).

Les éditions récentes, notamment le PMBOK 7 (2021), intègrent les approches agiles et adoptent une approche par principes et domaines de performance plutôt que des processus stricts.

Chapitre 4 : Les cycles de vie

4.6. Les Bonnes Pratiques

4.6.4. Capability Maturity Model Integration (CMMI)

Le **CMMI**, développé par le SEI (Software Engineering Institute), est un modèle d'amélioration des processus qui définit **cinq niveaux de maturité** : initial, géré, défini, géré quantitativement et en optimisation. Il sert à évaluer la maturité des processus d'une organisation et à identifier les axes d'amélioration, notamment dans les secteurs exigeants comme l'aéronautique, la défense ou les systèmes critiques.

Niveau 5 —► OPTIMISATION

- Amélioration continue, innovation

Niveau 4 —► GÉRÉ QUANTITATIVEMENT

- Processus mesurés et contrôlés statistiquement

Niveau 3 —► DÉFINI

- Processus standardisés à l'échelle de l'organisation

Niveau 2 —► GÉRÉ

- Processus planifiés et suivis par projet

Niveau 1 —► INITIAL

- Processus chaotiques, succès dépend des individus

Chapitre 4 : Les cycles de vie

4.6. Les Bonnes Pratiques

4.6.5. IEEE et standards pour l'ingénierie logicielle

L'**IEEE** publie plusieurs standards utiles pour les projets informatiques :

- ❑ **IEEE 829** : documentation de test (plans, spécifications, rapports).
- ❑ **IEEE 830** : documents d'exigences logicielles (SRS).
- ❑ **IEEE 1012** : vérification et validation du logiciel.
- ❑ **IEEE 1074** : processus du cycle de vie du développement logiciel.

Ces standards fournissent des modèles et bonnes pratiques pour les livrables techniques.

Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.1. L'importance des normes en gestion de projet

Les normes sont des **référentiels** qui formalisent les **bonnes pratiques**, **harmonisent le vocabulaire et facilitent la communication entre professionnels**.

Elles servent de guides adaptables au contexte de chaque projet, permettant de s'appuyer sur **l'expérience accumulée** plutôt que de tout réinventer.

Chapitre 4 : Les cycles de vie

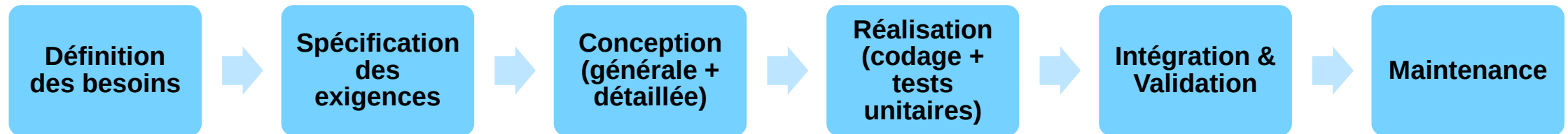
4.7. Les Normes

4.7.2. Z67-101 (AFNOR)

La **Z67-101** est une norme **française** publiée par l'**AFNOR** (Association Française de Normalisation). C'est l'une des premières normes structurant le développement logiciel en France.

Objet

Elle définit le **vocabulaire et les concepts** fondamentaux du génie logiciel, ainsi qu'un cadre méthodologique pour le **cycle de vie du logiciel**.



Limites

- Norme ancienne, peu mise à jour
- Remplacée progressivement par les normes ISO internationales
- Approche essentiellement **séquentielle**, peu compatible avec l'agilité

Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.2. ISO/IEC 12207 - Processus du cycle de vie du logiciel

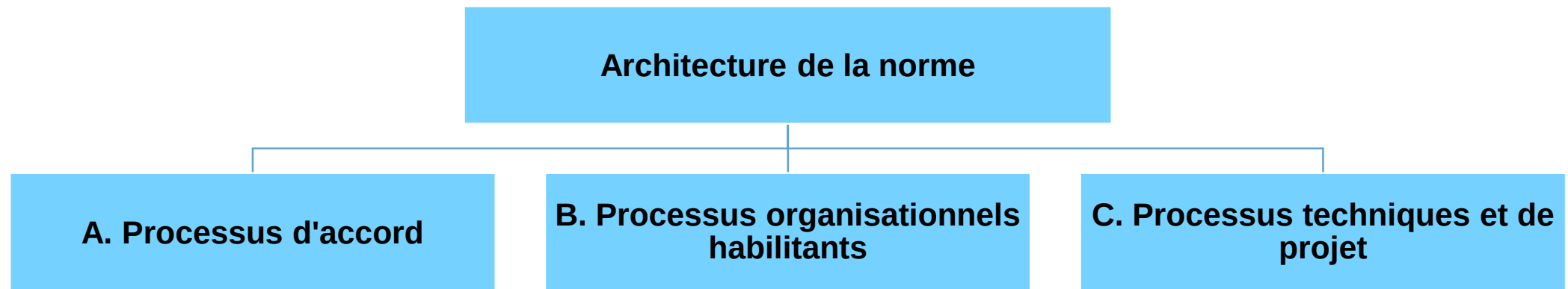
La **ISO/IEC 12207** est la norme internationale de référence pour les **processus du cycle de vie du logiciel**, publiée par l'**ISO** et l'**IEC**. Première version en **1995**, révisée en **2008** puis en **2017**.

Objet

Fournir un **cadre commun de processus** pour acquérir, fournir, développer, exploiter et maintenir des logiciels, applicable à tout type de projet.

Architecture de la norme

ISO 12207 organise les processus en **3 grandes catégories** :



Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.2. ISO/IEC 12207 - Processus du cycle de vie du logiciel

A. Processus d'accord

Régissent les relations entre organisations :

Processus	Description
Acquisition	Obtenir un produit ou service logiciel
Fourniture	Livrer un produit ou service à un acquéreur

B. Processus organisationnels habilitants

Soutiennent l'ensemble des projets de l'organisation :

Processus	Description
Gestion du cycle de vie	Définir et maintenir les politiques et procédures
Gestion des infrastructures	Fournir les ressources techniques nécessaires
Gestion du portefeuille	Initier et soutenir les projets
Gestion des ressources humaines	Compétences et formation
Gestion de la qualité	Assurance qualité organisationnelle
Gestion des connaissances	Capitalisation et partage du savoir

Chapitre 4 : Les cycles de vie

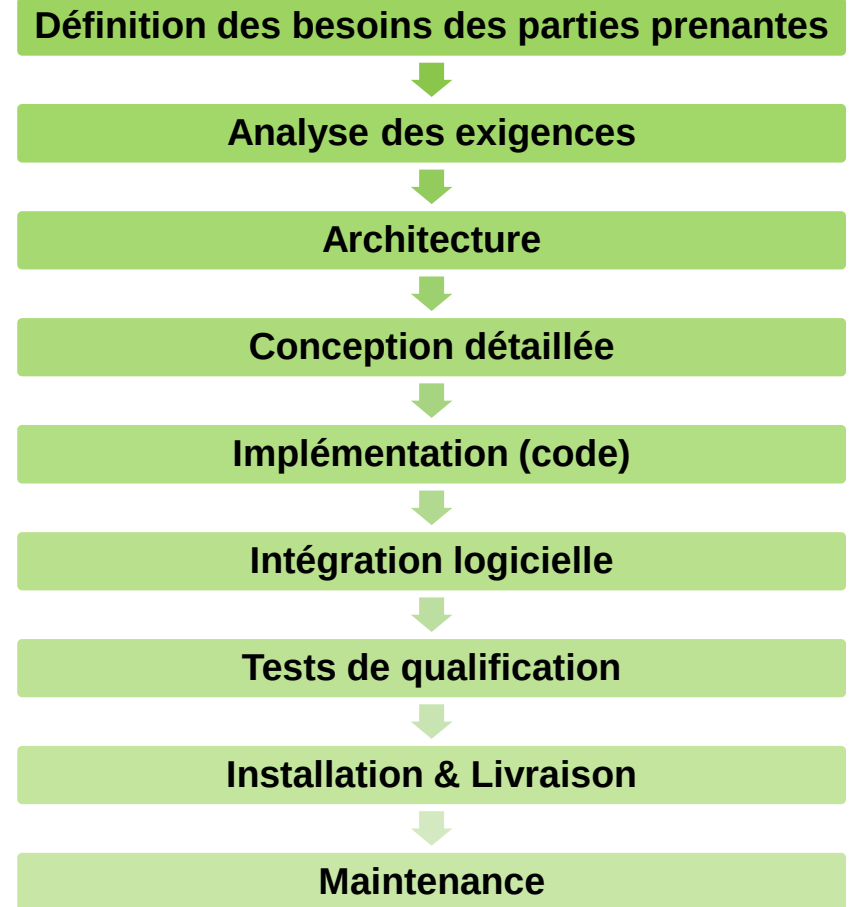
4.7. Les Normes

4.7.2. ISO/IEC 12207 - Processus du cycle de vie du logiciel

C. Processus techniques et de projet

Processus	Description
Planification	Définir le plan du projet
Évaluation et contrôle	Suivre et piloter l'avancement
Gestion des décisions	Analyser et trancher les options
Gestion des risques	Identifier et traiter les risques
Gestion de la configuration	Contrôler les versions et changements
Assurance qualité	Vérifier la conformité aux exigences
Gestion des mesures	Collecter et analyser les métriques

Processus de projet



Processus techniques

Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.2. ISO/IEC 12207 - Processus du cycle de vie du logiciel

Révision ISO 12207:2017

La version **2017** apporte des évolutions majeures :

- ☐ Alignement avec **ISO 15288** (systèmes) pour une cohérence logiciel/système
- ☐ Introduction de la notion de **partie prenante** élargie
- ☐ Meilleure compatibilité avec les **approches agiles**
- ☐ Structure basée sur des **outcomes** (résultats attendus) plutôt que des activités prescrites

Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.3. ISO 21500 — Gestion de projet

La **ISO 21500** est la norme internationale de référence pour le **management de projet**, publiée en **2012** et révisée en **2021**. Elle est souvent présentée comme l'équivalent international du **PMBOK** (Project Management Body of Knowledge).

Objet

Fournir des **lignes directrices** sur les concepts, processus et pratiques de management de projet, applicables à tout type de projet, quelle que soit sa taille ou son secteur.

Structure de la norme (version 2012)

ISO 21500 organise les processus selon **2 axes** :

- **Axe 1 — Groupes de processus (phases)**
- **Axe 2 — Domaines de sujets (knowledge areas)**

Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.3. ISO 21500 — Gestion de projet

Axe 1 — Groupes de processus (phases)



Groupe	Objectif
Initialisation	Autoriser et cadrer le projet
Planification	Définir le plan détaillé
Exécution	Réaliser le travail planifié
Maîtrise	Surveiller, mesurer, corriger
Clôture	Formaliser la fin du projet

Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.3. ISO 21500 — Gestion de projet

Axe 2 — Domaines de sujets (knowledge areas)

Domaine	Processus couverts
Intégration	Élaborer la charte, piloter l'ensemble
Parties prenantes	Identifier, gérer les attentes
Périmètre	Définir, valider, contrôler le contenu
Ressources	Planifier, constituer, développer l'équipe
Délais	Séquencer, estimer, établir l'échéancier
Coûts	Estimer, budgéter, contrôler
Risques	Identifier, analyser, traiter
Qualité	Planifier, assurer, contrôler
Approvisionnements	Planifier, sélectionner les fournisseurs
Communication	Planifier, diffuser, gérer l'information

Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.3. ISO 21500 — Gestion de projet

ISO 21500:2021 — Évolution majeure

La révision **2021** transforme profondément la norme :

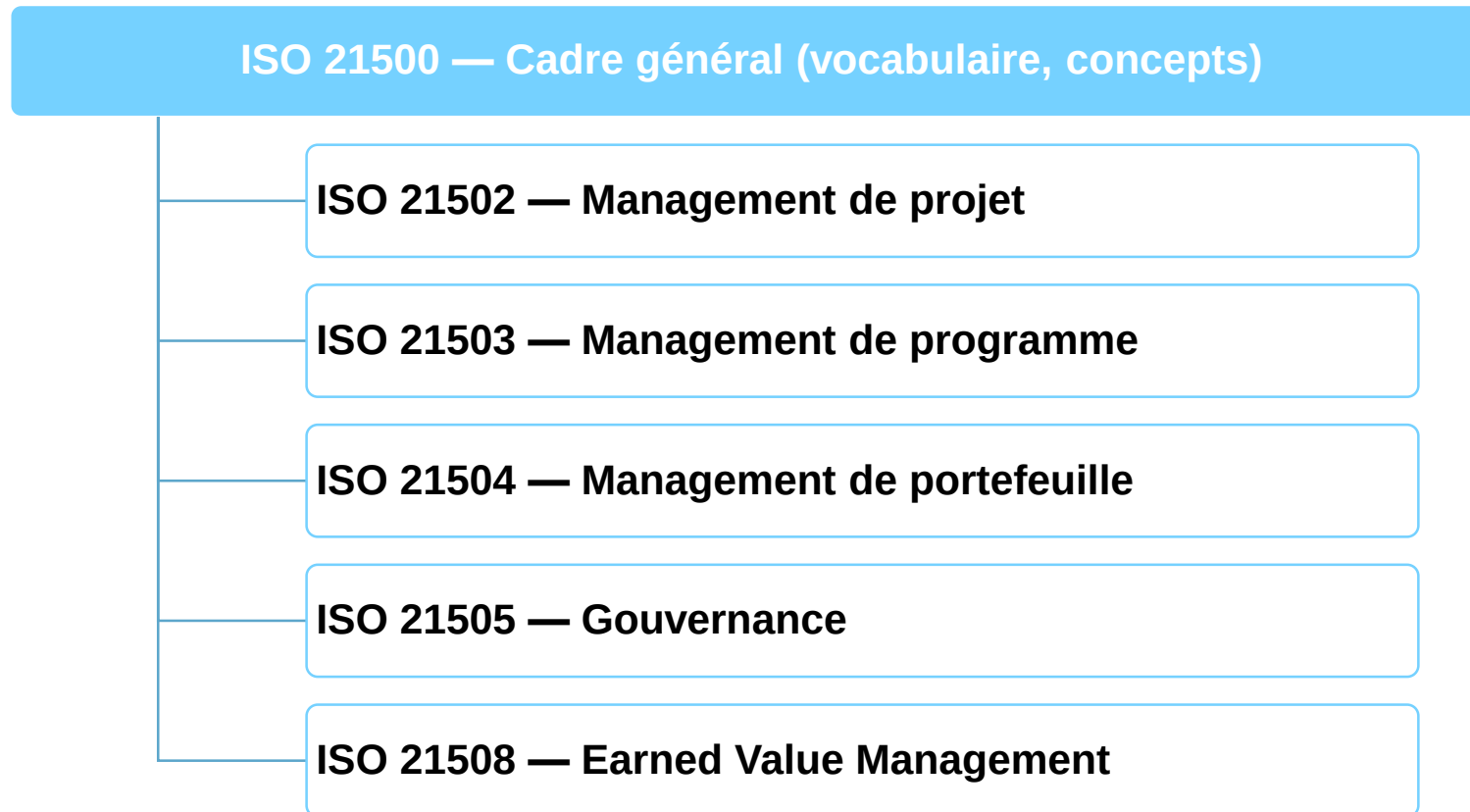
- ❑ Devient une **série** : ISO 21500 + ISO 21502 + ISO 21503...
- ❑ **ISO 21502:2020** remplace le cœur du management de projet
- ❑ Intègre explicitement les **approches agiles et hybrides**
- ❑ Introduit la notion de **gouvernance de projet**
- ❑ S'aligne avec **ISO 21505** (gouvernance) et **ISO 21504** (portefeuille)

Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.3. ISO 21500 — Gestion de projet

La famille ISO 21500 (2021+)



Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.3. ISO 21500 — Gestion de projet

Complémentarité des normes

Ces trois normes sont complémentaires et peuvent coexister dans un même projet :

ISO 21500 → Comment **PILOTER** le projet

ISO 12207 → Comment **DÉVELOPPER** le logiciel

Z67-101 → **Vocabulaire** et cadre de référence (contexte français)

Un projet informatique mené dans un contexte de marché public français peut s'appuyer sur la **Z67-101** pour le vocabulaire, l'**ISO 12207** pour structurer le cycle de vie logiciel, et l'**ISO 21500** pour cadrer le management du projet.

Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.4. ISO 9001 et qualité

La **norme ISO 9001** spécifie les exigences pour un système de management de la qualité. Elle est utilisée en informatique pour garantir la qualité des processus et livrables, en mettant l'accent sur l'orientation client, le leadership, l'implication du personnel, l'approche par processus, l'amélioration continue et les décisions basées sur des preuves.

Chapitre 4 : Les cycles de vie

4.7. Les Normes

4.7.5. Utilisation pratique des normes

Les organisations n'appliquent généralement pas **une norme de manière exhaustive**, mais s'en inspirent pour créer un référentiel adapté, combinent plusieurs normes et certifient certains processus pour rassurer clients et partenaires. Les normes évoluent pour intégrer les pratiques modernes (agile, DevOps) et privilégient désormais les principes plutôt que des processus stricts.

Chapitre 4 : Les cycles de vie

4.8. En Synthèse

4.8.1. Récapitulatif des cycles de vie

❑ Modèles séquentiels (Cascade, V)

- Organisation linéaire en phases successives.
- Adaptés aux projets à besoins stables, technologies maîtrisées, contraintes formelles.
- **Limites** : rigidité, effet tunnel, détection tardive des problèmes.

❑ Modèle incrémental

- Développement par ajouts successifs de fonctionnalités.
- Livraisons rapides de valeur et étalement des risques.
- **Usage** : souvent combiné avec itératif ou agile pour maximiser flexibilité et apprentissage.

❑ Modèles itératifs (RUP, Spirale...)

- Développement par cycles successifs de raffinement.
- Permettent d'apprendre progressivement et de gérer l'incertitude.
- **Convienent à** : projets complexes ou innovants.
- **Limites** : nécessitent maturité et expertise.

❑ Méthodes agiles (Scrum, XP...)

- Adaptation continue, collaboration, livraisons fréquentes.
- Excellentes pour environnements changeants et projets innovants.
- **Limites** : engagement fort des parties prenantes, changement culturel nécessaire.

Chapitre 4 : Les cycles de vie

Conclusion

Le cycle de vie d'un projet informatique **structure l'approche, influence l'équipe et impacte les chances de succès**. Les modèles séquentiels, itératifs et agiles offrent chacun avantages et limites selon le contexte.

Les normes et standards guident sans imposer, et les bonnes pratiques universelles (implication du client, livraisons fréquentes, automatisation, communication, amélioration continue) restent essentielles.

L'évolution vers plus d'agilité répond à l'incertitude, mais un équilibre entre **structure et flexibilité** est nécessaire. Le chef de projet doit **maîtriser les approches, diagnostiquer le contexte et adapter le cycle de vie pour assurer la réussite des projets**, tout en appliquant les fondamentaux universels de gestion.

Chapitre 5 : Le suivi du projet

- Les réunions
- La communication
- Les indicateurs et les tableaux de bord
- La gestion du portefeuille projets

Chapitre 5 : Le suivi du projet

Introduction

Le suivi de projet est une **activité centrale du management de projet informatique**. Il permet de s'assurer que le **projet avance conformément au plan défini**, de **détecter les écarts au plus tôt**, et de **prendre les décisions correctives nécessaires** pour garantir l'atteinte des **objectifs** en termes de **périmètre**, de **délais**, de **coûts** et de **qualité**.

Le suivi repose sur trois piliers fondamentaux :

- ❑ **La collecte** d'informations fiables et régulières sur l'avancement réel du projet
- ❑ **L'analyse** comparative entre le réalisé et le planifié (baseline)
- ❑ **La prise de décision et la communication** vers les parties prenantes

Définition du suivi de projet

Le suivi de projet (ou pilotage) désigne l'ensemble des **activités de surveillance, de mesure et de contrôle mises en œuvre** tout au long du cycle de vie d'un projet pour garantir que les objectifs seront atteints. Il s'appuie sur des **outils** (indicateurs, tableaux de bord), des **processus** (réunions, reportings) et des **compétences managériales** (communication, leadership).

Chapitre 5 : Le suivi du projet

5.1. Les Réunions de Projet

Les réunions constituent un moment privilégié **d'échange**, de **synchronisation** et de **prise de décision** au sein d'un projet. Bien menées, elles sont un levier de performance. Mal organisées, elles deviennent une source de perte de temps et de démotivation.

5.1.1 Pourquoi les réunions sont-elles indispensables ?

Dans un projet informatique, plusieurs acteurs aux profils différents (développeurs, chefs de projet, clients, direction) doivent collaborer. Les réunions permettent de :

- ☐ **Synchroniser** les équipes sur l'avancement et les priorités
- ☐ **Partager** les informations critiques en temps réel
- ☐ **Prendre des décisions** collectives et formaliser les engagements
- ☐ **Détecter et résoudre** les problèmes et blocages
- ☐ **Maintenir la cohésion** et la **motivation** de l'équipe
- ☐ **Valider** les livrables et jalons du projet

Chapitre 5 : Le suivi du projet

5.1. Les Réunions de Projet

5.1.2 Typologie des réunions de projet

On distingue plusieurs catégories de réunions selon leur objectif, leur fréquence et leurs participants :



Chapitre 5 : Le suivi du projet

5.1. Les Réunions de Projet

5.1.2 Typologie des réunions de projet

a) La réunion de lancement (Kick-off)

Objectif : Formaliser le **démarrage** officiel du projet, présenter les **objectifs**, le **périmètre**, **l'équipe**, le **planning** et les **règles de fonctionnement**. Elle marque l'engagement de toutes les parties prenantes.

Participants : Chef de projet, sponsor, équipe projet, client/maîtrise d'ouvrage

Fréquence : Une seule fois, en début de projet

Durée typique : 2 à 4 heures

b) Le comité de pilotage (COPIL)

Objectif : Instance de **gouvernance** au niveau stratégique. Permet de **présenter l'avancement global**, **valider les orientations**, **débloquer les ressources**, **prendre les décisions de haut niveau** (arbitrages, go/no-go).

Participants : Direction, sponsors, chefs de projet, responsables métier

Fréquence : Mensuelle ou à chaque jalon majeur

Durée typique : 1 à 2 heures

c) Le comité de projet (COPROJ)

Objectif : **Suivi opérationnel** du projet. **Revue** détaillée de l'avancement, des **risques**, des **actions en cours et à venir**. Lieu de résolution des problèmes opérationnels.

Participants : Chef de projet, équipe projet, référents fonctionnels

Fréquence : Hebdomadaire ou bi-hebdomadaire

Durée typique : 1 heure

Chapitre 5 : Le suivi du projet

5.1. Les Réunions de Projet

5.1.2 Typologie des réunions de projet

d) La réunion de revue de sprint (Scrum Review)

Dans les projets agiles, la **sprint review** est une réunion **de démonstration des fonctionnalités** développées durant le **sprint**, organisée avec les parties prenantes pour recueillir leurs retours.

- Fréquence : À la fin de chaque sprint (toutes les 2 à 4 semaines)
- Participants : Équipe de développement, Product Owner, parties prenantes
- Durée : 2 heures max pour un sprint de 4 semaines

e) La rétrospective

Réunion **d'amélioration continue** propre aux méthodes agiles. L'équipe analyse ce qui s'est bien passé, ce qui doit être amélioré, et définit des actions concrètes pour le sprint suivant.

Pendant le Sprint, le client peut être impliqué

⚠ Le périmètre du sprint (Sprint Goal) ne doit pas être modifié une fois le sprint démarré, même si le client le demande. Toute nouvelle demande est ajoutée au **Product Backlog** pour un prochain sprint.

Chapitre 5 : Le suivi du projet

5.1. Les Réunions de Projet

5.1.2 Typologie des réunions de projet

f) Le stand-up meeting (Daily Scrum)

Réunion quotidienne très courte (15 minutes maximum) où chaque membre de l'équipe répond à trois questions :

- Qu'ai-je accompli **hier** ?
- Que vais-je faire **aujourd'hui** ?
- Y a-t-il des **obstacles** qui me bloquent ?

g) La réunion de clôture

Marque **la fin officielle du projet**. Elle permet de **valider la livraison finale**, de **recueillir les retours d'expérience (REX)**, de formaliser le **bilan** et de **célébrer les succès de l'équipe**.

h) Sprint Planning (réunion de planification du sprint)

C'est **l'événement qui démarre officiellement un sprint**. Pendant cette réunion, l'équipe Scrum : **Définit l'objectif du sprint** (Sprint Goal), **Sélectionne les éléments du Product Backlog à réaliser**, **Découpe le travail en tâches techniques**, **Établit un plan pour atteindre l'objectif (2 heures par semaine de sprint)**

Participants : Product Owner, Scrum Master, Équipe de développement

Chapitre 5 : Le suivi du projet

5.1. Les Réunions de Projet

5.1.2 Typologie des réunions de projet

Tableau récapitulatif des types de réunions :

Type de réunion	Objectif principal	Fréquence	Participants clés
Kick-off	Lancement officiel du projet	1 fois	Tous les acteurs
COFIL	Gouvernance stratégique	Mensuelle / Jalon	Direction, Sponsors
COPROJ	Suivi opérationnel	Hebdomadaire	Équipe projet
Sprint Planning	Lancement du sprint	Début de sprint	Product Owner, Scrum Master, Équipe
Sprint Review	Démo et feedback	Fin de sprint	Équipe + PO + Parties prenantes
Rétrospective	Amélioration continue	Fin de sprint	Équipe projet
Daily Scrum	Synchronisation quotidienne	Quotidienne	Équipe de développement
Clôture	Bilan et validation finale	1 fois	Tous les acteurs

Chapitre 5 : Le suivi du projet

5.1. Les Réunions de Projet

5.1.3 Organisation et animation d'une réunion efficace

Une réunion efficace repose sur une préparation rigoureuse et une animation structurée. On distingue trois phases :

Avant la réunion : la préparation

- Définir clairement l'objectif et le type de réunion (information, décision, brainstorming...)
- Établir et envoyer l'ordre du jour en avance (minimum 48h avant)
- Sélectionner les participants pertinents (principe du besoin d'être présent)
- Réserver la salle ou configurer la solution de visioconférence
- Préparer les documents supports (tableaux de bord, présentations, rapports)
- Collecter les points d'avancement auprès des contributeurs

Pendant la réunion : l'animation

- Démarrer à l'heure et rappeler l'objectif et les règles (temps de parole, téléphones...)
- Suivre l'ordre du jour et gérer le temps pour chaque point
- Encourager la participation de tous et éviter les monopoles de parole
- Reformuler et synthétiser les échanges pour valider la compréhension commune
- Formaliser les décisions et actions au fil des échanges
- Clore la réunion avec un récapitulatif : décisions prises, actions, responsables, échéances

Après la réunion : le suivi

- Rédiger et diffuser le compte rendu dans les 24-48 heures
- Assurer le suivi des actions décidées lors de la prochaine réunion
- Mettre à jour les outils de suivi (plan d'action, tableau de bord...)

Chapitre 5 : Le suivi du projet

5.1. Les Réunions de Projet

5.1.3 Organisation et animation d'une réunion efficace

Les **7 règles d'or** d'une réunion réussie

1. Pas de réunion sans **objectif** clair
2. Inviter uniquement les **personnes indispensables**
3. Envoyer **l'ordre du jour** en avance
4. Commencer et terminer à **l'heure**
5. Prendre des **notes** et formaliser les **décisions**
6. Attribuer chaque **action à un responsable** avec une **échéance**
7. **Diffuser** le compte rendu rapidement

Chapitre 5 : Le suivi du projet

5.1. Les Réunions de Projet

5.1.4 Le compte rendu de réunion

Le compte rendu (CR) est un document officiel qui trace les **échanges**, **décisions** et **engagements** pris lors d'une réunion. Il doit être **synthétique**, **précis** et **actionnable**. Il comprend généralement :

- **En-tête** : date, lieu, type de réunion, participants présents/absents
- **Ordre du jour traité**
- **Synthèse** des échanges point par point
- **Décisions** prises et validations
- **Plan d'action** : action, responsable, date limite
- **Prochaine réunion prévue**

Chapitre 5 : Le suivi du projet

5.2. La Communication en Projet

La **communication** est souvent citée comme le **premier facteur de succès ou d'échec** d'un projet. Une communication claire, régulière et adaptée aux différents publics permet de maintenir **l'alignement de toutes les parties prenantes** et de prévenir les **malentendus et conflits**.

5.2.1 Les enjeux de la communication projet

Dans un projet informatique, la **communication** doit répondre à plusieurs enjeux :

- **Aligner les attentes** entre la maîtrise d'ouvrage (MOA) et la maîtrise d'œuvre (MOE)
- **Inform**er les parties prenantes de l'avancement, des risques et des changements
- **Fédérer** l'équipe autour d'une **vision commune** et maintenir la **motivation**
- **Gérer les situations de crise** et les **mauvaises** nouvelles avec **transparence**
- **Documenter** et tracer les échanges pour éviter les litiges

Principe de communication en projet

Selon le PMBOK (Project Management Body of Knowledge), le chef de projet consacre environ **90% de son temps à des activités de communication**. La communication **n'est pas une activité annexe** du management de projet, c'est son activité principale.

Chapitre 5 : Le suivi du projet

5.2. La Communication en Projet

5.2.2 Le plan de communication

Le **plan de communication** est un document stratégique qui définit, pour chaque type de communication nécessaire dans le projet, les modalités précises de mise en œuvre. C'est un outil de planification et de pilotage de la communication.

Un plan de communication répond aux questions suivantes :

- ☐ **QUOI** communiquer ? (nature et contenu de l'information)
- ☐ **À QUI** ? (cibles, récepteurs de l'information)
- ☐ **PAR QUI** ? (émetteur, responsable de la communication)
- ☐ **QUAND** ? (fréquence, planning de communication)
- ☐ **COMMENT** ? (canal, format, support)
- ☐ **POURQUOI** ? (objectif de chaque communication)

Chapitre 5 : Le suivi du projet

5.2. La Communication en Projet

5.2.2 Le plan de communication

Exemple de plan de communication :

Communication	Émetteur	Destinataires	Fréquence	Canal	Objectif
Rapport d'avancement	Chef de projet	Sponsors, Direction	Hebdomadaire	Email + PDF	Informar sur l'avancement
Compte rendu COPIL	Chef de projet	Toutes parties prenantes	Mensuelle	Email + SharePoint	Traçabilité décisions
Newsletter projet	Chef de projet	Équipe + Parties prenantes	Bi-mensuelle	Email	Fédérer et informer
Alerte risque/issue	Chef de projet	Sponsor, Direction	À chaud	Email urgent / Appel	Escalade et réactivité
Démo sprint	Équipe dev + PO	Clients, Utilisateurs	Fin de sprint	Réunion / Visio	Validation et feedback
Communication interne	Chef de projet	Équipe projet	Quotidienne	Slack / Teams	Coordination opérationnelle

Chapitre 5 : Le suivi du projet

5.2. La Communication en Projet

5.2.3 Les acteurs de la communication

La communication en projet implique des acteurs aux rôles distincts et aux besoins d'information différents :

La maîtrise d'ouvrage (MOA)

Elle a besoin d'informations synthétiques sur l'avancement, les coûts et les risques majeurs (tableaux de bord, reporting de haut niveau).

La maîtrise d'œuvre (MOE)

Elle a besoin d'informations opérationnelles précises (spécifications, priorités, retours de recette, accès aux ressources).

Le chef de projet

Interface centrale entre MOA et MOE. Il est le principal émetteur et récepteur de la communication. Son rôle est de filtrer, synthétiser et diffuser l'information au bon niveau de détail selon l'interlocuteur.

Les utilisateurs finaux

Destinataires souvent oubliés mais essentiels. La communication envers les utilisateurs porte sur les changements à venir, les formations, les impacts sur leurs processus et les bénéfices attendus. Elle est cruciale pour la conduite du changement.

La direction générale / sponsors

Attendent une communication synthétique et orientée valeur métier : retour sur investissement, avancement par rapport aux objectifs stratégiques, alertes sur les risques majeurs.

Chapitre 5 : Le suivi du projet

5.2. La Communication en Projet

5.2.4 Les canaux et outils de communication

Le choix du canal est aussi important que le contenu du message. Il doit être adapté au public, à l'urgence et au contexte :

Canal	Avantages	Inconvénients	Usage recommandé
Réunion présentielle	Riche, interactions directes, décisions rapides	Coûteuse en temps, déplacements	Décisions importantes, kick-off, conflits
Visioconférence	Pratique, économique, flexible	Moins d'interactions non-verbales	Réunions régulières, équipes distantes
Email	Traçable, asynchrone, diffusion large	Surcharge, risque de mal-compris	Reportings, comptes rendus, diffusion d'info
Chat (Slack, Teams)	Rapide, informel, collaboratif	Information dispersée, bruit	Communication quotidienne, questions rapides
Intranet / SharePoint	Centralise les documents, accessible	Nécessite maintenance, adoption	Base documentaire, référentiels, archivage
Téléphone	Rapide, direct, personnel	Non traçable, chronophage	Urgences, situations sensibles
Tableau de bord en ligne	Visuel, partagé, temps réel	Demande un outillage	Suivi d'avancement continu

Chapitre 5 : Le suivi du projet

5.2. La Communication en Projet

5.2.5 La communication en situation de crise

La gestion des mauvaises nouvelles est une compétence clé du chef de projet. En cas de dérive importante (dépassement de budget, retard critique, problème technique majeur), la communication doit être :

- ❑ **Rapide** : informer sans délai, ne pas masquer les problèmes
- ❑ **Transparente** : présenter les faits objectivement, sans minimiser ni dramatiser
- ❑ **Constructive** : accompagner la mauvaise nouvelle d'un plan d'action correctif
- ❑ **Ciblée** : adapter le niveau de détail selon les interlocuteurs
- ❑ **Formalisée** : documenter la communication pour tracer les échanges

Piège à éviter

La **tentation de cacher ou retarder les mauvaises nouvelles** est l'une des **erreurs** les plus courantes et les plus **graves** en gestion de projet. Elle **détruit la confiance, aggrave les problèmes et rend les corrections plus coûteuses**. Un problème signalé tôt est presque toujours **moins grave** qu'un problème révélé trop tard.

Chapitre 5 : Le suivi du projet

5.3. Les Indicateurs et les Tableaux de Bord

Les **indicateurs et tableaux de bord** sont les instruments de **mesure du pilotage** de projet. Ils permettent de transformer des **données** brutes en **informations exploitables** pour la **prise de décision**, la **communication** et l'**amélioration continue**.

5.3.1 Les indicateurs de performance (KPIs)

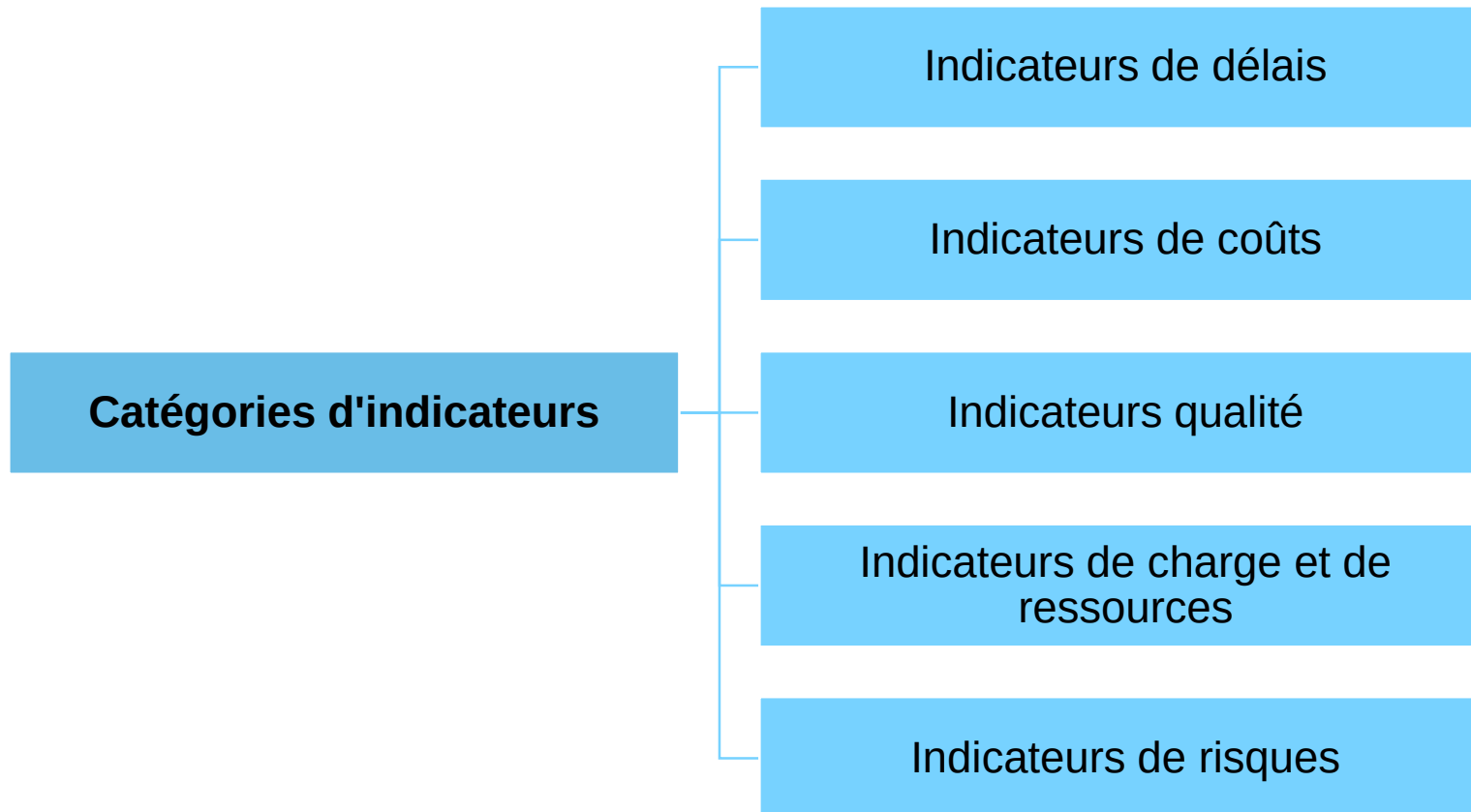
Un **indicateur** (ou KPI : Key Performance Indicator) est une **mesure quantitative** ou **qualitative** permettant d'évaluer un aspect spécifique du projet par rapport à un objectif défini. Un bon indicateur doit être SMART :

- ❑ **Spécifique** : mesure un aspect précis et bien défini
- ❑ **Mesurable** : basé sur des données collectables de façon fiable
- ❑ **Atteignable** : objectif réaliste et challengeant
- ❑ **Relevant** (Pertinent) : en lien direct avec les objectifs du projet
- ❑ **Temporellement défini** : associé à une période ou une échéance

Chapitre 5 : Le suivi du projet

5.3. Les Indicateurs et les Tableaux de Bord

5.3.2 Catégories d'indicateurs



Chapitre 5 : Le suivi du projet

5.3. Les Indicateurs et les Tableaux de Bord

5.3.2 Catégories d'indicateurs

Indicateurs de délais

- Taux d'avancement global
- Écart de planning : différence entre travail prévu et travail réalisé
- Indice de performance planning (SPI) : ratio travail accompli / travail planifié (SPI < 1 = retard)
- Nombre de jalons respectés / total de jalons
- Date prévisionnelle de fin de projet

Indicateurs de coûts

- Taux de consommation du budget : budget consommé / budget total
- Écart de coût (Cost Variance - CV) : différence entre valeur acquise et coût réel
- Indice de performance coût (CPI) : ratio valeur acquise / coût réel (CPI < 1 = dépassement)
- Estimation à l'achèvement (EAC) : coût total prévu en tenant compte des tendances actuelles

Indicateurs qualité

- Taux de défauts (bugs par fonctionnalité ou par sprint)
- Taux de couverture des tests
- Nombre d'anomalies bloquantes / critiques ouvertes
- Taux de satisfaction utilisateurs (mesurable via enquêtes)
- Taux de livraisons conformes aux critères d'acceptation

Indicateurs de charge et de ressources

- Taux d'occupation des ressources (charge réelle / charge disponible)
- Nombre de jours de formation réalisés
- Taux d'absentéisme de l'équipe
- Vitesse de l'équipe (méthodes agiles) : nombre de points de story par sprint

Indicateurs de risques

- Nombre de risques identifiés / risques traités / risques ouverts
- Niveau d'exposition globale aux risques (criticité moyenne)
- Nombre d'issues (problèmes survenus) résolues / en cours

Chapitre 5 : Le suivi du projet

5.3. Les Indicateurs et les Tableaux de Bord

5.3.2 Catégories d'indicateurs

Tableau synthétique des principaux KPIs :

Indicateur	Formule / Calcul	Valeur cible	Interprétation
SPI (Schedule Performance Index)	Valeur Acquisée / Valeur Planifiée	$SPI = 1$	< 1 : retard / > 1 : avance
CPI (Cost Performance Index)	Valeur Acquisée / Coût Réel	$CPI = 1$	< 1 : dépassement budget
Taux d'avancement	$(\text{Tâches réalisées} / \text{Tâches totales}) \times 100$	Selon planning	Comparer au % planifié
Taux de consommation budget	$(\text{Dépenses réelles} / \text{Budget total}) \times 100$	$\leq$ avancement %	Dépassement si $>$ avancement
Vélocité (Agile)	Points story réalisés par sprint	Stable / croissant	Chute = problème équipe
Taux de défauts	Nb bugs / Nb fonctionnalités livrées	Faible et décroissant	Qualité du développement

Chapitre 5 : Le suivi du projet

5.3. Les Indicateurs et les Tableaux de Bord

5.3.3 La méthode de la Valeur Acquise (Earned Value Management)

L'Earned Value Management (EVM) est une méthode standardisée (IEEE, PMI) permettant de mesurer objectivement la performance d'un projet en intégrant trois dimensions : **périmètre, délais et coûts**. Elle repose sur trois valeurs fondamentales :

Valeur	Acronyme	Définition	Exemple
Budget à l'achèvement	BAC	Budget total du projet planifié	100 000 €
Valeur Planifiée	PV (Planned Value)	Travail qui devrait être fait à la date de mesure (en €)	40 000 € à mi-projet
Valeur Acquise	EV (Earned Value)	Valeur du travail réellement accompli (en €)	35 000 € réellement fait
Coût Réel	AC (Actual Cost)	Coût réellement dépensé pour le travail accompli	42 000 € dépensés

Chapitre 5 : Le suivi du projet

5.3. Les Indicateurs et les Tableaux de Bord

5.3.4 Les tableaux de bord de projet

Le **tableau de bord** (dashboard) est un **outil visuel de synthèse** qui **regroupe les indicateurs les plus importants** du projet sous une forme **lisible et exploitable**. C'est l'outil de pilotage par excellence du chef de projet et de la direction.

Principes de conception d'un tableau de bord efficace

Pertinence	• n'afficher que les indicateurs vraiment utiles à la décision
Lisibilité	• représentations visuelles claires (couleurs, graphiques, jauges)
Synthèse	• une page = une vue d'ensemble (ne pas noyer dans les détails)
Actualité	• données fraîches et fréquence de mise à jour adaptée
Personnalisation	• adapter au public cible (direction vs équipe vs client)

Chapitre 5 : Le suivi du projet

5.3. Les Indicateurs et les Tableaux de Bord

5.3.4 Les tableaux de bord de projet

Structure type d'un tableau de bord projet

Un tableau de bord projet complet comprend généralement les sections suivantes :

- ❑ En-tête : nom du projet, date de mise à jour, responsable, phase en cours
- ❑ Feux tricolores (RAG Status) : rouge/orange/vert sur les axes délais, coûts, qualité, risques
- ❑ Avancement global : % d'avancement réel vs prévu, courbe S
- ❑ Jalons : état des jalons passés et prévus
- ❑ Budget : consommé vs planifié, prévision à fin
- ❑ Risques principaux : top 3-5 des risques actifs
- ❑ Actions clés : points d'attention et décisions requises

Chapitre 5 : Le suivi du projet

5.3. Les Indicateurs et les Tableaux de Bord

5.3.4 Les tableaux de bord de projet

Le système RAG (Red-Amber-Green)

Statut	Signification	Actions requises
VERT (Green)	Le projet avance conformément au plan. Situation sous contrôle.	Maintenir le cap, aucune escalade nécessaire
ORANGE (Amber)	Des écarts sont détectés mais le projet reste récupérable. Vigilance requise.	Plan d'action correctif en cours, surveiller de près
ROUGE (Red)	Le projet est en difficulté sérieuse. Objectifs menacés sans intervention.	Escalade immédiate, décision de la direction requise

Chapitre 5 : Le suivi du projet

5.3. Les Indicateurs et les Tableaux de Bord

5.3.5 Les outils de reporting et de tableaux de bord

De nombreux outils permettent de créer et partager des tableaux de bord projet :

- ❑ **Outils de gestion de projet** : MS Project, Jira, Asana, Monday.com — tableaux de bord intégrés
- ❑ **Outils BI** (Business Intelligence) : Power BI, Tableau, Qlik — pour des analyses avancées
- ❑ **Suites bureautiques** : Excel, Google Sheets — flexibles et universels
- ❑ **Outils de reporting agile** : Jira, Rally, VersionOne — burndown charts, vélocité
- ❑ **Outils de documentation** : Confluence, SharePoint — pour la diffusion des rapports

Chapitre 5 : Le suivi du projet

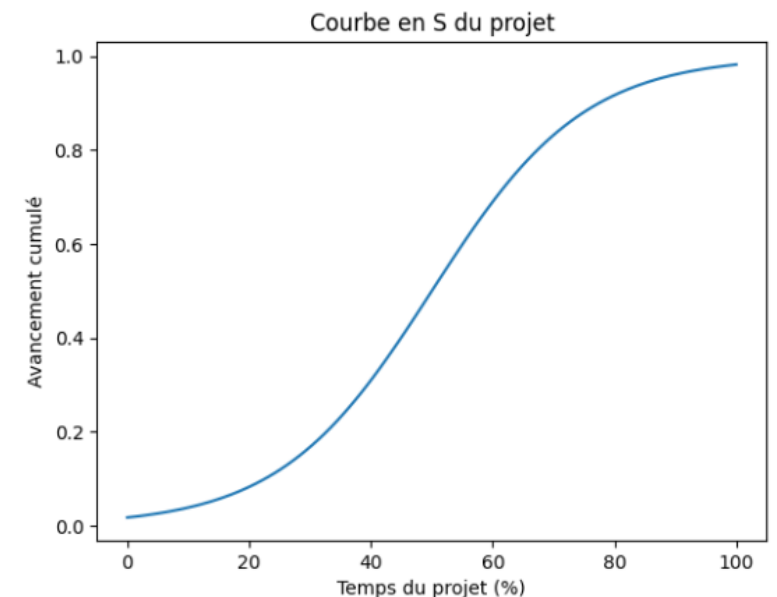
5.3. Les Indicateurs et les Tableaux de Bord

5.3.6 La courbe S (S-Curve)

La **courbe S** est une représentation graphique de l'avancement cumulé d'un projet (en % ou en budget). Elle tire son nom de sa forme caractéristique en S, typique de la plupart des projets :

- ❑ **Démarrage lent** : montée en charge progressive de l'équipe
- ❑ **Phase centrale intensive** : pic d'activité, courbe croissante
- ❑ **Phase finale** : ralentissement, finalisations, recette

En superposant la **courbe S** planifiée (baseline) et la courbe S réelle, le chef de projet visualise instantanément les écarts de planning et peut anticiper les risques de dérive.



Chapitre 5 : Le suivi du projet

5.4. La Gestion du Portefeuille Projets

Alors que la gestion de projet porte sur un **projet unique**, la gestion du **portefeuille** projet (Project Portfolio Management - PPM) concerne la **gestion cohérente et optimisée d'un ensemble de projets au sein d'une organisation**. C'est une discipline de management stratégique qui vise à maximiser la valeur globale créée par l'ensemble des projets en allouant efficacement les ressources limitées de l'entreprise.

5.4.1 Définitions et périmètre

Définition du portefeuille projet

Un portefeuille projet est **un ensemble de projets, programmes et/ou opérations regroupés** pour **faciliter** une gestion efficace visant à **atteindre les objectifs stratégiques** d'une organisation. Les composants d'un portefeuille partagent les mêmes ressources organisationnelles et peuvent être interdépendants ou pas. (Source : PMI - Standard for Portfolio Management)

Chapitre 5 : Le suivi du projet

5.4. La Gestion du Portefeuille Projets

5.4.1 Définitions et périmètre

On distingue trois niveaux de management :

Niveau	Définition	Horizon temporel	Objectif principal
Projet	Initiative unique avec objectif défini	Court terme (mois à 2 ans)	Livrer un résultat spécifique
Programme	Groupe de projets liés avec bénéfices coordonnés	Moyen terme (1 à 5 ans)	Réaliser des bénéfices qui ne peuvent l'être séparément
Portefeuille	Ensemble de projets/programmes alignés sur la stratégie	Long terme (permanent)	Maximiser la valeur stratégique et l'utilisation des ressources

Chapitre 5 : Le suivi du projet

5.4. La Gestion du Portefeuille Projets

5.4.2 Les objectifs de la gestion de portefeuille

- ✓ **Aligner** les projets sur la **stratégie** de l'entreprise
- ✓ **Optimiser** l'allocation des **ressources** humaines, financières et matérielles
- ✓ **Prioriser** les projets en fonction de leur **valeur stratégique** et de leur **faisabilité**
- ✓ **Équilibrer** le portefeuille : risque/bénéfice, court terme/long terme, innovation/maintenance
- ✓ **Gérer** les interdépendances entre projets
- ✓ **Contrôler** les coûts et les bénéfices globaux du portefeuille
- ✓ **Fournir** une **visibilité** globale à la direction sur l'ensemble des initiatives en cours

Chapitre 5 : Le suivi du projet

5.4. La Gestion du Portefeuille Projets

5.4.3 Le cycle de vie du portefeuille projet

La gestion du portefeuille est un processus continu et itératif comprenant plusieurs étapes clés :

1. Identification et collecte des projets
2. Catégorisation des projets
3. Évaluation et scoring
4. Priorisation et sélection
5. Exécution et suivi du portefeuille
6. Révision et rebalancement

Chapitre 5 : Le suivi du projet

5.4. La Gestion du Portefeuille Projets

5.4.3 Le cycle de vie du portefeuille projet

1. Identification et collecte des projets

Recenser toutes les demandes **de projets et initiatives issues des différentes directions métiers**. Chaque demande est formalisée dans une **fiche projet** initiale précisant **les objectifs, le budget estimé, la valeur attendue et les ressources nécessaires**.

2. Catégorisation des projets

Classer les projets selon des critères définis par l'organisation. Les catégories courantes sont :

- ☐ **Projets obligatoires** / réglementaires (conformité légale, sécurité)
- ☐ **Projets stratégiques** (croissance, innovation, transformation digitale)
- ☐ **Projets d'amélioration opérationnelle** (optimisation de processus, efficacité)
- ☐ **Projets de maintenance et d'infrastructure** (run, maintien en condition)

Chapitre 5 : Le suivi du projet

5.4. La Gestion du Portefeuille Projets

5.4.3 Le cycle de vie du portefeuille projet

3. Évaluation et scoring

Chaque projet est évalué sur des critères **quantitatifs et qualitatifs** pour permettre leur comparaison objective. Les critères courants incluent :

- ☐ Valeur stratégique / alignement avec les objectifs de l'entreprise (scoring 1-5)
- ☐ Retour sur investissement (ROI) et valeur actualisée nette (VAN)
- ☐ Niveau de risque et complexité
- ☐ Ressources nécessaires et disponibilité
- ☐ Urgence et délai de mise sur le marché (Time-to-Market)
- ☐ Dépendances avec d'autres projets ou systèmes

La matrice de scoring

La **matrice de scoring** (ou matrice de priorisation) est un outil qui attribue des notes pondérées à chaque projet sur plusieurs critères. La somme pondérée donne **un score global** permettant de **comparer et classer** les projets de manière **objective et transparente**. Elle facilite les discussions de priorisation en Comité de Pilotage du Portefeuille.

Chapitre 5 : Le suivi du projet

5.4. La Gestion du Portefeuille Projets

5.4.3 Le cycle de vie du portefeuille projet

4. Priorisation et sélection

Sur la base des scores obtenus et des contraintes de ressources, le comité de gouvernance du portefeuille sélectionne les projets à lancer, à reporter ou à abandonner.

5. Exécution et suivi du portefeuille

Une fois le portefeuille constitué, il s'agit de piloter l'ensemble des projets en cours. Le suivi du portefeuille porte sur :

- ☐ **L'avancement global du portefeuille** (synthèse des états des projets)
- ☐ **La consommation des ressources** partagées et la détection des surconsommations
- ☐ **La gestion des interdépendances et des risques** croisés entre projets
- ☐ **La révision périodique du portefeuille** pour intégrer les nouveaux projets et sortir les projets terminés ou abandonnés

6. Révision et rebalancement

Le portefeuille doit être **revu régulièrement** (trimestriellement en général) pour **s'adapter aux évolutions de la stratégie de l'entreprise, aux résultats des projets et aux changements de contexte** (économique, technologique, réglementaire).

Chapitre 5 : Le suivi du projet

5.4. La Gestion du Portefeuille Projets

5.4.4 La gouvernance du portefeuille

La gouvernance est l'ensemble des mécanismes organisationnels qui encadrent la gestion du portefeuille. Elle répond aux questions : **qui décide ? Sur quelle base ? À quelle fréquence ?**

Les instances de gouvernance

- ☐ Le **Comité Stratégique du Portefeuille** (CSP) : niveau direction générale, aligne le portefeuille sur la stratégie, arbitre les priorités majeures
- ☐ Le **PMO (Project Management Office)** : bureau de gestion de projet, centralise les méthodes, outils, reporting et expertises
- ☐ Le **Comité de Pilotage du Portefeuille** : revue périodique des projets, décisions d'allocation de ressources
- ☐ **Les chefs de projet** : reporting vers le PMO, escalade des problèmes

Le rôle du PMO (Project Management Office)

Le PMO est l'organe central de la gestion de portefeuille. Ses responsabilités incluent :

- ☐ Standardisation des méthodologies, outils et templates de gestion de projet
- ☐ Consolidation du reporting projet pour la direction
- ☐ Formation et accompagnement des chefs de projet
- ☐ Gestion et allocation des ressources partagées
- ☐ Animation des comités de gouvernance
- ☐ Gestion du référentiel des projets (registre de portefeuille)
- ☐ Contrôle de la qualité des livrables et du respect des processus

Chapitre 5 : Le suivi du projet

5.4. La Gestion du Portefeuille Projets

5.4.5 Les outils de gestion de portefeuille

La gestion d'un portefeuille de projets implique de traiter de grandes quantités de données provenant de nombreux projets. Des outils spécialisés facilitent ce travail :

- ❑ **MS Project Server / MS Project Online (Microsoft)** : solution complète de PPM intégrée à l'écosystème Microsoft
- ❑ **Clarity PPM (Broadcom)** : plateforme PPM entreprise
- ❑ **Planview** : solution PPM pour la gestion des ressources et du portefeuille
- ❑ **Jira Portfolio / Jira Advanced Roadmaps** : gestion du portefeuille en contexte agile
- ❑ **ServiceNow PPM** : intégration dans les plateformes ITSM
- ❑ **Smartsheet** : solution collaborative flexible
- ❑ **Power BI / Tableau** : pour la visualisation et le reporting du portefeuille

Chapitre 5 : Le suivi du projet

5.4. La Gestion du Portefeuille Projets

5.4.6 La matrice risque / valeur

La matrice risque/valeur (ou matrice de portefeuille) est un outil graphique qui positionne chaque projet selon deux axes :

- **Axe horizontal** : valeur stratégique ou ROI (de faible à élevée)
- **Axe vertical** : niveau de risque (de faible à élevé)

Les quatre quadrants obtenus permettent de catégoriser les projets :

Quadrant	Caractéristiques	Stratégie recommandée
Étoiles (Haute valeur / Faible risque)	Projets phares de l'organisation	Lancer en priorité, investir
Pains quotidiens (Faible valeur / Faible risque)	Projets de maintenance nécessaires	Optimiser, automatiser, réduire le coût
Paris risqués (Haute valeur / Haut risque)	Innovations à fort potentiel	Gérer avec soin, prototyper, piloter en petit
Poids morts (Faible valeur / Haut risque)	Projets peu rentables et risqués	Stopper ou réorienter

Chapitre 5 : Le suivi du projet

5.4. La Gestion du Portefeuille Projets

5.4.7 Indicateurs clés du portefeuille

- ❑ **Taux d'utilisation des ressources** : % de ressources engagées dans des projets
- ❑ **ROI moyen du portefeuille** : retour sur investissement agrégé
- ❑ **Taux de projets en bonne santé** (statut vert) vs **en difficulté** (orange/rouge)
- ❑ **Taux de réalisation des bénéfices attendus** (post-implémentation)
- ❑ **Nombre de projets par phase** (pipeline, en cours, terminés)
- ❑ **Délai moyen de livraison vs délai planifié**
- ❑ **Taux d'abandon** et motifs d'arrêt des projets

Le suivi de projet n'est pas une fin en soi : c'est un moyen au service de la **livraison de valeur**. Les meilleurs outils et indicateurs ne remplaceront jamais le **jugement d'un chef de projet expérimenté**, capable de **prendre des décisions difficiles**, de **communiquer avec courage et transparence**, et de **mobiliser son équipe** autour d'un **objectif commun**.

Chapitre 6 : Organiser le changement

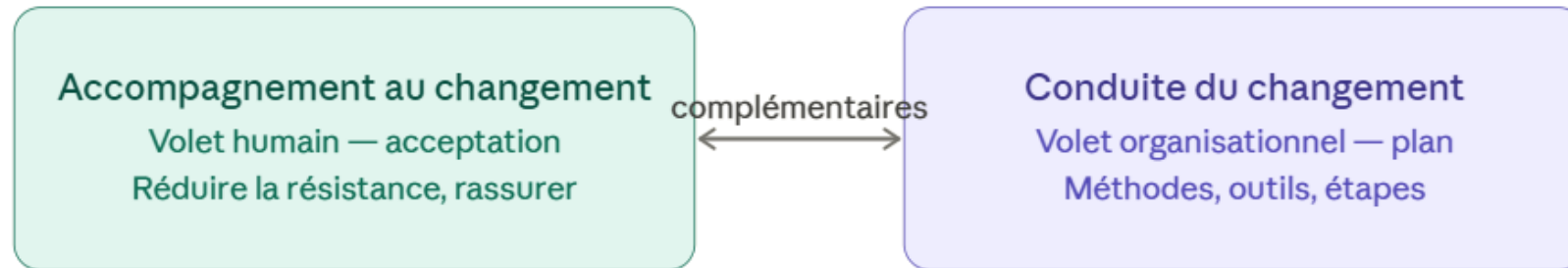
- L'accompagnement au changement
 - La conduite du changement
- 

Chapitre 6 : Organiser le changement

6.1. I. Introduction — Pourquoi le changement est-il un enjeu majeur en informatique ?

Tout projet informatique ne se limite pas à une livraison technique. Il modifie profondément les **habitudes**, les **processus** et parfois **l'identité professionnelle** des utilisateurs. C'est pourquoi **organiser le changement** est une composante à part entière du management de projet.

On distingue deux notions complémentaires : l'**accompagnement au changement** (le volet **humain** et **psychologique**) et la **conduite du changement** (le volet **organisé** et **méthodologique**).



Chapitre 6 : Organiser le changement

6.2. L'accompagnement au changement

6.2.1. Définition

L'accompagnement au changement désigne l'ensemble des actions visant à **préparer, soutenir et aider les individus** à traverser une transition induite par un projet. Il agit sur les **dimensions psychologiques, relationnelles et culturelles.**

Chapitre 6 : Organiser le changement

6.2. L'accompagnement au changement

6.2.2. Pourquoi les individus résistent-ils au changement ?

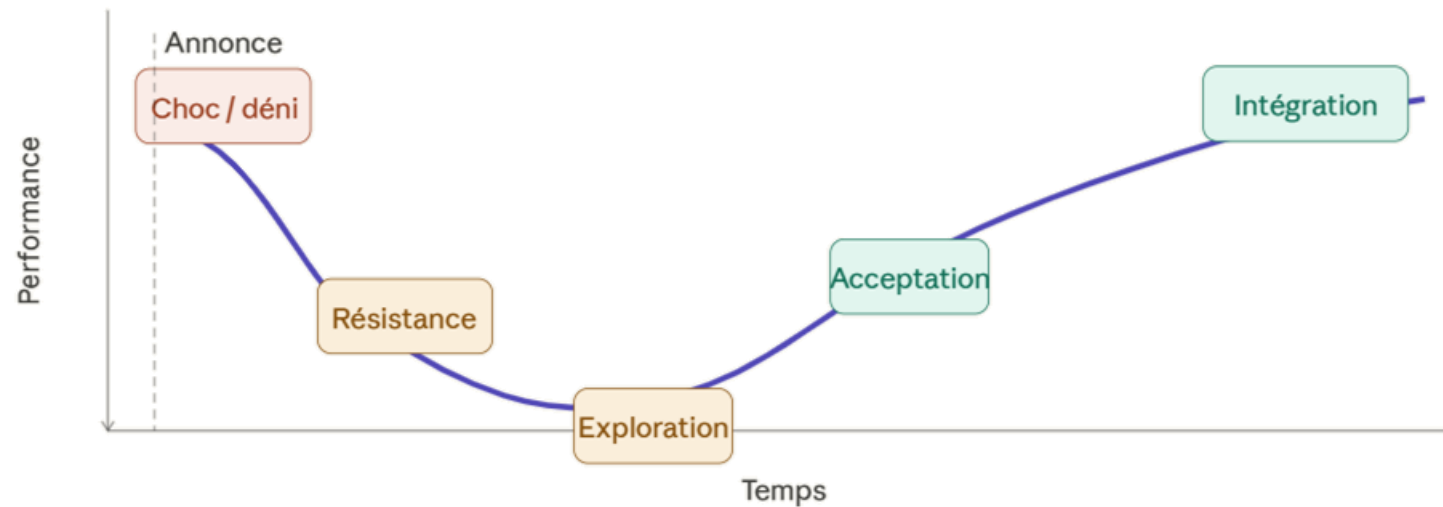
La résistance est naturelle. Elle peut avoir plusieurs origines :

- **Peur de l'inconnu** : incertitude sur le nouveau rôle, les nouvelles compétences requises.
- **Perte de repères** : les habitudes de travail sont bousculées.
- **Sentiment de dévalorisation** : l'ancien outil ou processus était maîtrisé, le nouveau remet à zéro l'expertise.
- **Manque d'information** : les rumeurs comblent le vide laissé par la communication officielle.
- **Enjeux de pouvoir** : certains acteurs craignent de perdre de l'influence.

Chapitre 6 : Organiser le changement

6.2. L'accompagnement au changement

6.2.2. Pourquoi les individus résistent-ils au changement ?



La courbe du deuil (Kübler-Ross adaptée au projet) illustre les étapes psychologiques traversées par les utilisateurs lors d'un changement : **choc initial, résistance, exploration progressive puis acceptation et intégration**. Le chef de projet doit adapter ses actions à chaque phase.

Chapitre 6 : Organiser le changement

6.2. L'accompagnement au changement

6.2.3. Les acteurs de l'accompagnement

Acteur	Rôle dans l'accompagnement
Sponsor / Direction	Légitimer le changement, donner le cap
Chef de projet	Coordonner les actions, arbitrer
Responsable RH	Accompagnement individuel, formation
Manager de proximité	Relais terrain, rassurer les équipes
Utilisateurs pilotes	Ambassadeurs du changement
Consultant change management	Expertise méthodologique externe

Chapitre 6 : Organiser le changement

6.2. L'accompagnement au changement

6.2.4. Les leviers de l'accompagnement

a) La communication

C'est le premier levier. Elle doit être **anticipée, régulière, honnête et adaptée au public**. Une bonne communication répond aux questions : **Pourquoi ce changement ? Qu'est-ce qui va changer concrètement ? Quand ? Qu'est-ce qui ne change pas ? Quel est mon rôle ?**

On distingue plusieurs types de messages :

- ☐ **Messages d'information** (quoi, quand, comment)
- ☐ **Messages de sens** (pourquoi, pour quoi faire)
- ☐ **Messages de reconnaissance** (votre contribution compte)

b) La formation

La formation réduit le **sentiment d'incompétence** et prépare les utilisateurs à **être opérationnels**. Elle peut prendre plusieurs formes : formations présentielle, e-learning, tutorat par des utilisateurs clés (key users), guides utilisateurs...

c) L'implication des utilisateurs

Plus les utilisateurs sont **impliqués** tôt (ateliers de co-conception, tests, retours), plus ils s'approprient le système. On parle de **stratégie participative**.

d) Le soutien managérial

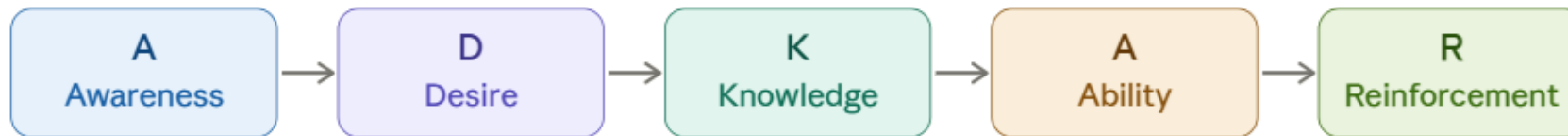
Les managers de proximité sont des relais essentiels. Un changement mal porté par les managers **échoue** même si le projet technique est **réussi**.

Chapitre 6 : Organiser le changement

6.2. L'accompagnement au changement

6.2.5. Mesurer l'état d'acceptation : le modèle ADKAR

Le modèle **ADKAR** (Prosci) est un outil de diagnostic individuel du changement :



- **Awareness** : l'individu comprend pourquoi le changement est nécessaire.
- **Desire** : il a envie de participer et de soutenir le changement.
- **Knowledge** : il sait comment changer (compétences, comportements).
- **Ability** : il est capable de mettre en œuvre les nouvelles compétences.
- **Reinforcement** : le changement est ancré et pérennisé.

Si un individu **bloque à une étape**, toutes les suivantes sont **inefficaces**. Par exemple, **former** quelqu'un qui n'a pas encore **envie de changer** (pas de Desire) est peine perdue.

Chapitre 6 : Organiser le changement

6.3. La conduite du changement

6.3.1. Définition

La conduite du changement est la **démarche structurée et planifiée** permettant de passer d'une **situation actuelle à une situation cible** en **minimisant les perturbations et en maximisant l'adoption**. Elle s'appuie sur des méthodes, des outils et un plan d'action formalisé.

Chapitre 6 : Organiser le changement

6.3. La conduite du changement

6.3.2. Les grandes étapes de la conduite du changement



Chapitre 6 : Organiser le changement

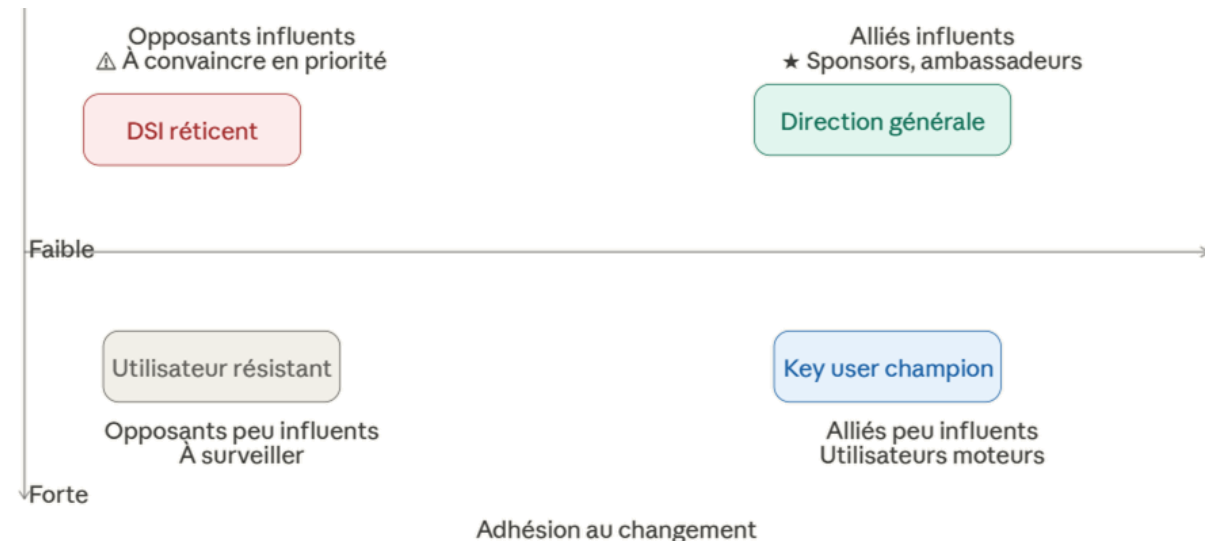
6.3. La conduite du changement

6.3.3. Étape 1 — Le diagnostic du changement

Avant d'agir, il faut comprendre le contexte. Le diagnostic porte sur :

- a) **L'analyse des impacts** : **Qu'est-ce qui va changer ? Pour qui ? Avec quelle ampleur ?** On établit une **matrice des impacts** croisant populations et domaines affectés (processus, outils, compétences, organisation).
- b) **La cartographie des acteurs** (sociogramme) : On identifie les parties prenantes et leur positionnement vis-à-vis du changement.

Cette matrice influence/adhésion permet de prioriser les efforts : les **opposants influents** nécessitent une **attention et une stratégie de conviction particulière**.



Chapitre 6 : Organiser le changement

6.3. La conduite du changement

6.3.3. Étape 1 — Le diagnostic du changement

● DSI réticent — *Influent + Opposant*

Dangereux. Il a le pouvoir de bloquer le projet ET il ne veut pas du changement.

→ **Priorité absolue** : le convaincre, l'impliquer, répondre à ses craintes.

● Direction générale — *Influente + Alliée*

Idéal. Elle soutient le projet ET elle a le pouvoir de le porter.

→ **En faire un sponsor**, un ambassadeur visible.

● Utilisateur résistant — *Peu influent + Opposant*

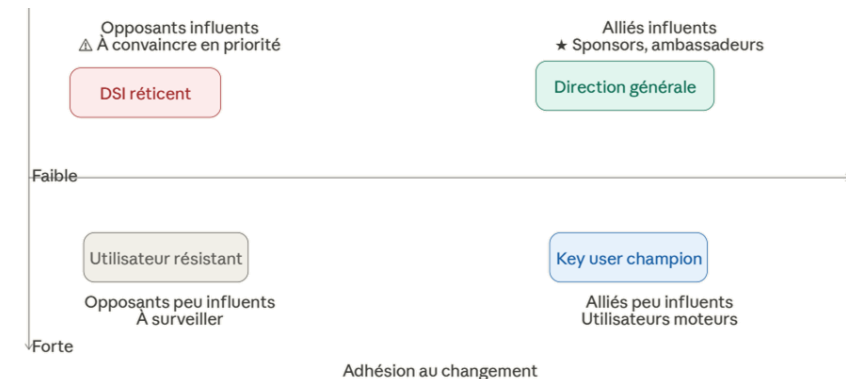
Il résiste mais n'a pas vraiment le pouvoir de bloquer.

→ **À surveiller** pour éviter que ça ne s'étende, mais pas prioritaire.

● Key user champion — *Peu influent + Allié*

Impliqué et moteur sur le terrain, même sans grand pouvoir hiérarchique.

→ **En faire un relais** pour convaincre ses collègues.



Chapitre 6 : Organiser le changement

6.3. La conduite du changement

6.3.4. Étape 2 — La stratégie de changement

Il existe plusieurs stratégies selon le contexte :

Stratégie	Description	Quand l'utiliser
Directive	Changement imposé par le haut, peu de participation	Urgence, crise, changement réglementaire
Participative	Co-construction avec les utilisateurs	Changement culturel, fort ancrage métier
Négociée	Compromis entre parties	Syndicalisation, enjeux RH sensibles
Incrémentale	Petits pas successifs	Fort risque de résistance, systèmes complexes
Expérimentale	Pilote puis généralisation	Innovation, incertitude sur les impacts

Chapitre 6 : Organiser le changement

6.3. La conduite du changement

6.3.5. Étape 3 — Le plan de conduite du changement

C'est le document central. Il comprend au minimum :

- ✓ **Le plan de communication** : Qui communique quoi, à qui, par quel canal, à quelle date ? Il répond aux questions des parties prenantes à chaque phase du projet.
- ✓ **Le plan de formation** : Recense les populations à former, les modalités (présentiel, distanciel, tutorat), le calendrier et les ressources nécessaires.
- ✓ **Le plan d'accompagnement** : Identifie les actions de soutien individuelles ou collectives : coaching, support utilisateur, sessions de questions-réponses...

Chapitre 6 : Organiser le changement

6.3. La conduite du changement

6.3.6. Étape 4 — Le déploiement et la gestion des résistances

Pendant le déploiement, les résistances se manifestent. Le chef de projet doit les identifier et y répondre de manière adaptée :

Type de résistance	Réponse adaptée
Résistance par manque d'info	Communication ciblée, FAQ, sessions d'information
Résistance par peur de l'échec	Formation renforcée, tutorat, droit à l'erreur
Résistance politique / intérêts	Négociation, implication dans les décisions
Résistance culturelle / valeurs	Accompagnement long terme, ambassadeurs internes

Chapitre 6 : Organiser le changement

6.3. La conduite du changement

6.3.7. Étape 5 — L'ancrage et la pérennisation

Un changement n'est réussi que lorsqu'il est ancré dans les pratiques quotidiennes. Les actions d'ancrage incluent :

- ☐ La mise à jour des procédures et des fiches de poste.
- ☐ L'intégration du changement dans les objectifs et indicateurs de performance.
- ☐ La célébration des succès pour renforcer la dynamique positive.
- ☐ La mise en place d'un support utilisateur durable (hotline, référents métier).
- ☐ Des audits post-déploiement pour mesurer l'adoption réelle.

Chapitre 6 : Organiser le changement

6.4. Les outils clés de la conduite du changement

1. La matrice des parties prenantes (Power/Interest Grid)

Elle classe les acteurs selon leur pouvoir et leur intérêt dans le projet, ce qui guide la stratégie de communication et d'implication.

2. Le plan de communication

Document formel avec : message, émetteur, cible, canal (réunion, email, intranet, affichage...), date et fréquence.

3. Le tableau de bord du changement

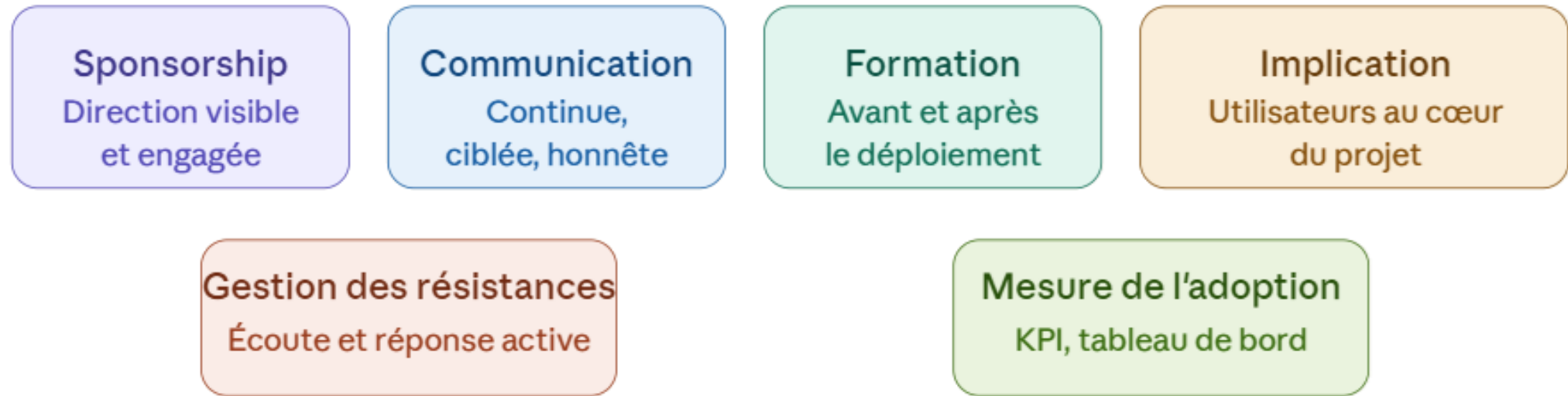
Il mesure le taux d'adoption, la satisfaction des utilisateurs, le nombre de tickets support, l'utilisation effective du nouveau système.

4. La matrice RACI appliquée au changement

Elle clarifie qui est **Responsable**, **Accountable** (décideur), **Consulté** et **Informé** pour chaque action du plan de changement.

Chapitre 6 : Organiser le changement

6.5. Les facteurs clés de succès



Chapitre 6 : Organiser le changement

6.6. Les erreurs fréquentes à éviter

- ❑ **Démarrer trop tard** : intégrer la conduite du changement dès le lancement du projet, pas à la veille du déploiement.
- ❑ **Sous-estimer la résistance** : même un bon outil peut être rejeté si le changement n'est pas accompagné.
- ❑ **Communiquer trop peu ou trop tard** : le vide d'information génère des rumeurs et amplifie les craintes.
- ❑ **Former sans accompagner** : une formation ponctuelle sans suivi est vite oubliée.
- ❑ **Négliger les managers intermédiaires** : ce sont les relais clés ; s'ils ne sont pas convaincus, les équipes ne le seront pas non plus.
- ❑ **Confondre déploiement et adoption** : l'outil peut être installé sans être réellement utilisé.

Chapitre 6 : Organiser le changement

6.7. Synthèse

La réussite d'un projet informatique dépend autant de la **qualité technique de la solution** que de la **capacité à faire adopter le changement**. L'accompagnement au changement agit sur les **individus** (**psychologie, motivation, compétences**), tandis que la **conduite du changement organise l'ensemble de la démarche** (**diagnostic, stratégie, plan, déploiement, ancrage**). Ces deux dimensions sont **indissociables** et doivent être **planifiées** dès le début du projet, avec le même sérieux que les **aspects techniques**.

À retenir : un projet informatique **réussit techniquement** mais **échoue humainement** s'il ne prend pas en compte **la dimension du changement**. La formule classique est :

Résultat = Qualité de la solution × Niveau d'adoption. Si l'adoption est nulle, le résultat est nul, quelle que soit la qualité technique.

Chapitre 7 : Planification d'un projet

- Le découpage du projet
- L'estimation des charges et des délais
- L'ordonnancement des tâches
- Le diagramme de Gantt
- La planification des ressources
- Le plan de projet (plan de management)
- La gestion des risques dans la planification
- La planification en méthodes agiles
- Les outils de planification

Chapitre 7 : Planification d'un projet

7.1. Introduction — Pourquoi planifier ?

Planifier un projet informatique, c'est **définir à l'avance ce qui doit être fait, par qui, quand, avec quels moyens et dans quel ordre**. Sans planification, un projet court trois risques majeurs : dépasser les délais, exploser le budget, et ne pas livrer ce qui était attendu. La planification est l'acte fondateur du **pilotage** ; elle transforme un objectif flou en **un plan d'action concret et contrôlable**.

La planification répond à six questions fondamentales :



Chapitre 7 : Planification d'un projet

7.2.Le découpage du projet — WBS (Work Breakdown Structure)

Avant de planifier des délais ou des coûts, il faut **décomposer le travail**. Le WBS (ou OTP — Organigramme des Tâches du Projet) est une **décomposition hiérarchique du projet** en livrables, puis en lots de travail, puis en **tâches élémentaires**. C'est le fondement de toute **planification sérieuse**.

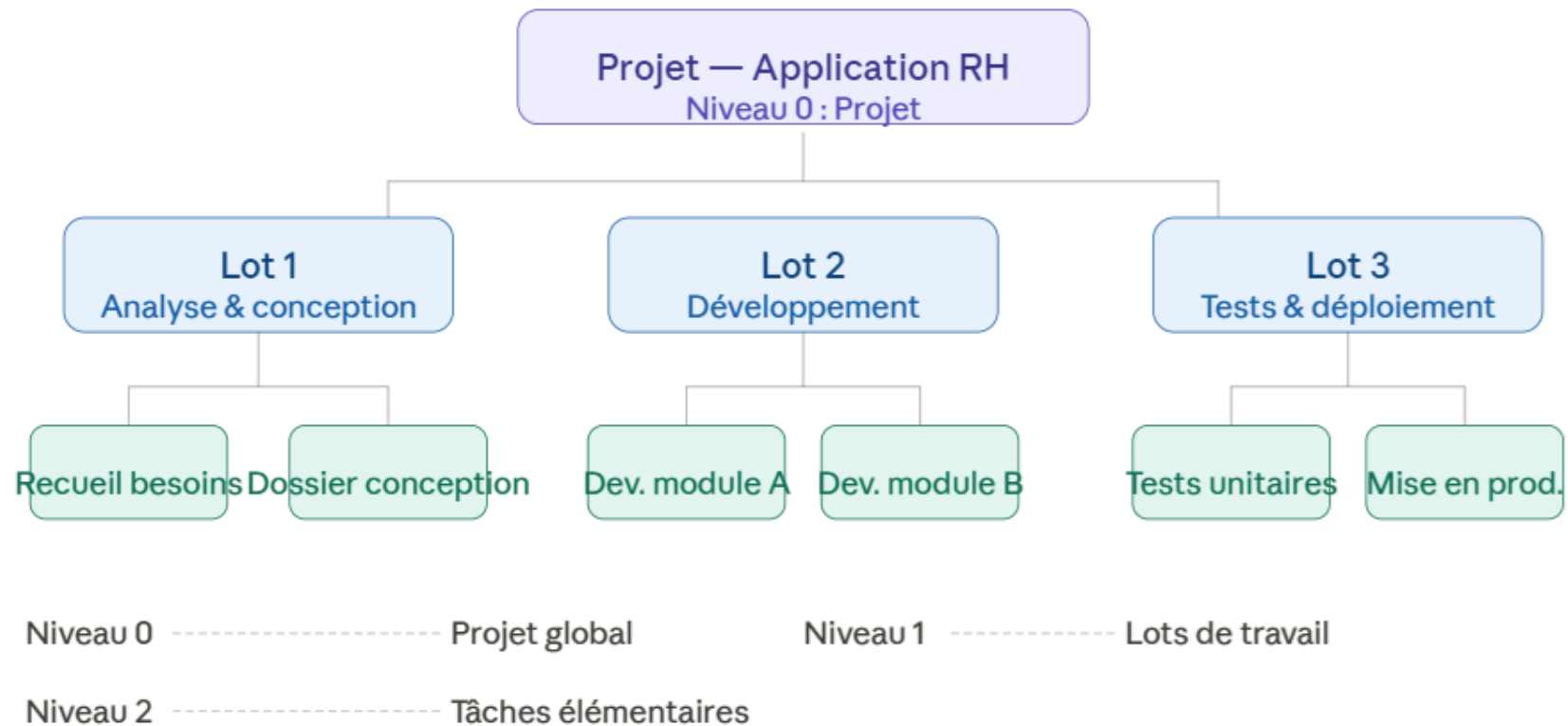
Le principe est simple :

Prendre le **résultat attendu comme point de départ**, puis le **découper progressivement** jusqu'à obtenir des **tâches concrètes**, dont on peut **évaluer le temps et les moyens nécessaires**.

Chapitre 7 : Planification d'un projet

7.2.Le découpage du projet — WBS (Work Breakdown Structure)

7.2.1. Structure du WBS



Chapitre 7 : Planification d'un projet

7.2.Le découpage du projet — WBS (Work Breakdown Structure)

7.2.2. Règles de décomposition du WBS

Une **tâche élémentaire** (au niveau le plus bas du WBS) doit respecter la **règle des 8/80** : sa durée doit être comprise entre 8 heures (1 jour) et 80 heures (2 semaines). En dessous, c'est trop micro-managé ; au-dessus, c'est trop difficile à suivre.

Chaque tâche doit avoir un responsable unique, un livrable identifiable et une durée estimable.

Chapitre 7 : Planification d'un projet

7.3. L'estimation des charges et des délais

7.3.1. Pourquoi estimer est difficile

Estimer dans un **projet informatique** est notoirement complexe : les besoins sont imprécis au départ, les technologies évoluent, et les équipes **sous-estiment systématiquement la complexité** (biais d'optimisme). La loi de Hofstadter formule cela ainsi : "Il faut toujours plus de temps que prévu, même en tenant compte de la loi de Hofstadter."

Chapitre 7 : Planification d'un projet

7.3. L'estimation des charges et des délais

7.3.2. Les méthodes d'estimation

- a) **L'estimation par analogie** : on s'appuie sur des projets similaires déjà réalisés. Rapide mais approximatif.
- b) **L'estimation par décomposition** : on décompose le projet en tâches élémentaires et on estime chacune. Plus précis mais chronophage.
- c) **Le planning poker (méthode agile)** : chaque membre de l'équipe attribue indépendamment une valeur à la tâche, puis on compare et on discute les divergences. Élimine les biais d'influence.
- d) **La méthode PERT (Program Evaluation and Review Technique)** : pour chaque tâche, on formule trois estimations.

Chapitre 7 : Planification d'un projet

7.4. L'ordonnancement des tâches

7.4.1. Les dépendances entre tâches

Les tâches ne sont pas indépendantes. On distingue quatre types de dépendances :

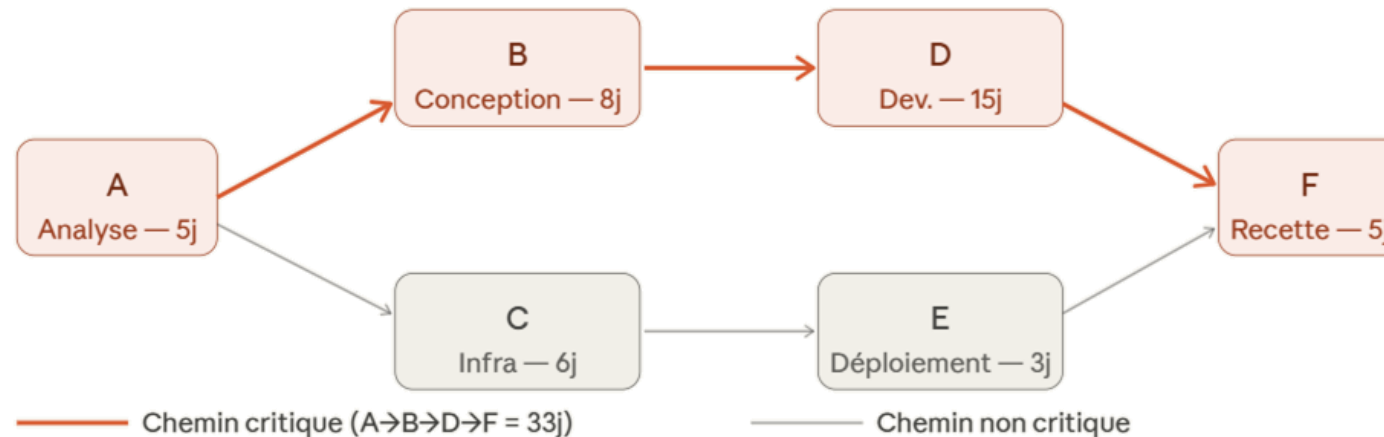
Type	Signification	Exemple
Fin-Début (FD)	B commence quand A se termine	Les tests commencent après le développement
Début-Début (DD)	B commence quand A commence	Rédaction doc et codage démarrent ensemble
Fin-Fin (FF)	B se termine quand A se termine	Recette et formation finissent ensemble
Début-Fin (DF)	Rare, B se termine quand A commence	Transition entre ancienne et nouvelle procédure

Chapitre 7 : Planification d'un projet

7.4. L'ordonnancement des tâches

7.4.2. Le diagramme de réseau (PERT/CPM)

Le réseau PERT représente les tâches et leurs dépendances sous forme de graphe orienté. Il permet de calculer le **chemin critique**.



Chapitre 7 : Planification d'un projet

7.4. L'ordonnancement des tâches

7.4.3. Le chemin critique

Le **chemin critique** est la séquence de tâches dont la somme des durées **est la plus longue**. Il détermine **la durée minimale du projet**. Toute tâche sur le chemin critique ne peut pas prendre de retard sans retarder l'ensemble du projet.

Les tâches hors du chemin critique ont une **marge totale** (flottement) : elles peuvent être retardées sans impact sur la date de fin du projet.

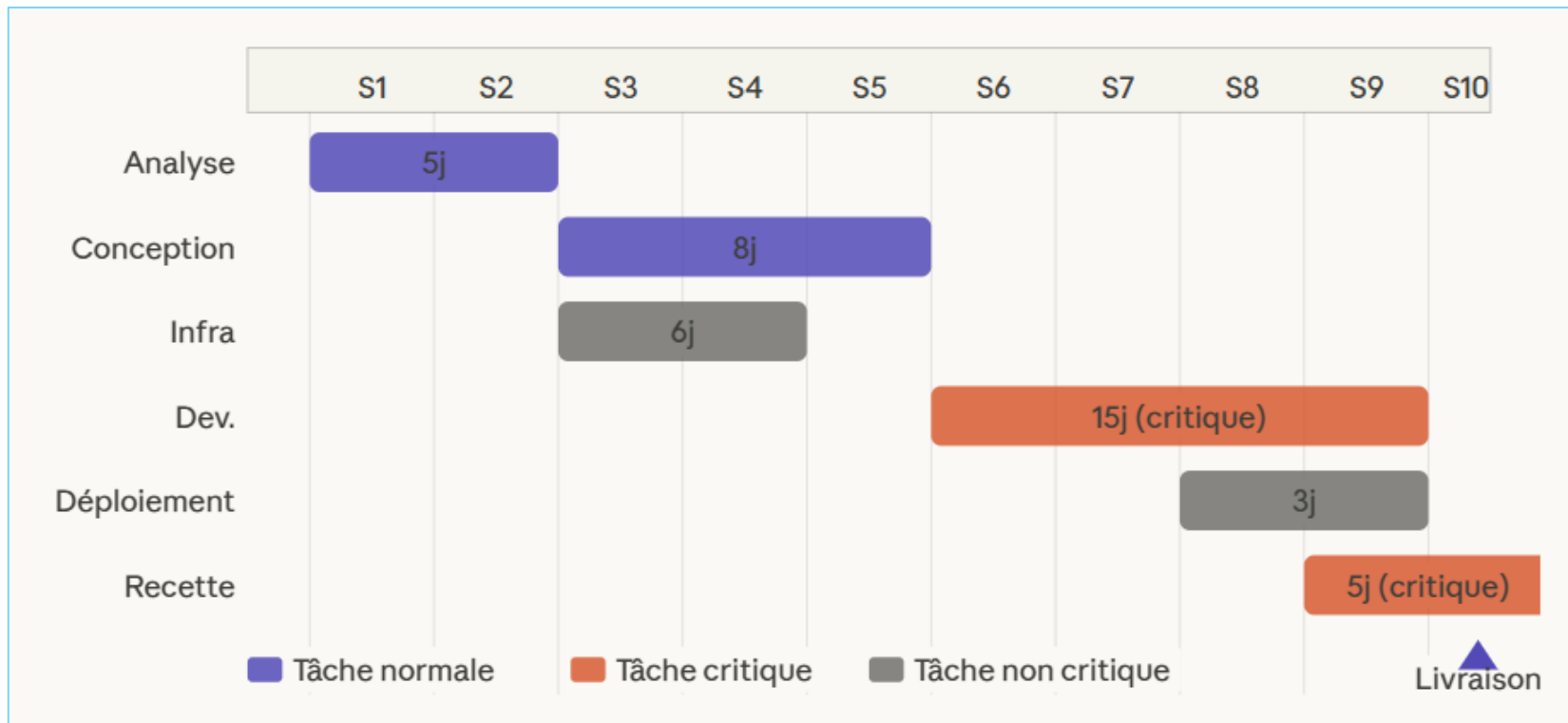
Calcul des marges :

- ❑ Date au plus tôt (Early Start / Early Finish) : calcul en allant de gauche à droite.
- ❑ Date au plus tard (Late Start / Late Finish) : calcul en allant de droite à gauche.
- ❑ Marge totale = Late Start – Early Start.
- ❑ Marge libre = Early Start de la tâche suivante – Early Finish de la tâche courante.

Chapitre 7 : Planification d'un projet

7.5. Le diagramme de Gantt

Le **diagramme de Gantt** est la représentation visuelle la plus utilisée en gestion de projet. Il représente les tâches sur un axe temporel, permettant de voir d'un coup d'œil qui fait quoi et quand.



Chapitre 7 : Planification d'un projet

7.5. Le diagramme de Gantt

7.5.1. Les éléments d'un Gantt

Un Gantt complet contient : la liste des **tâches** (colonne gauche), **l'axe temporel** (semaines, mois), les **barres de tâches avec leur durée**, les **dépendances** entre tâches, **les jalons** (losanges ou triangles marquant des événements clés), et souvent le **taux d'avancement** (barre interne plus foncée).

7.5.2. Les jalons (milestones)

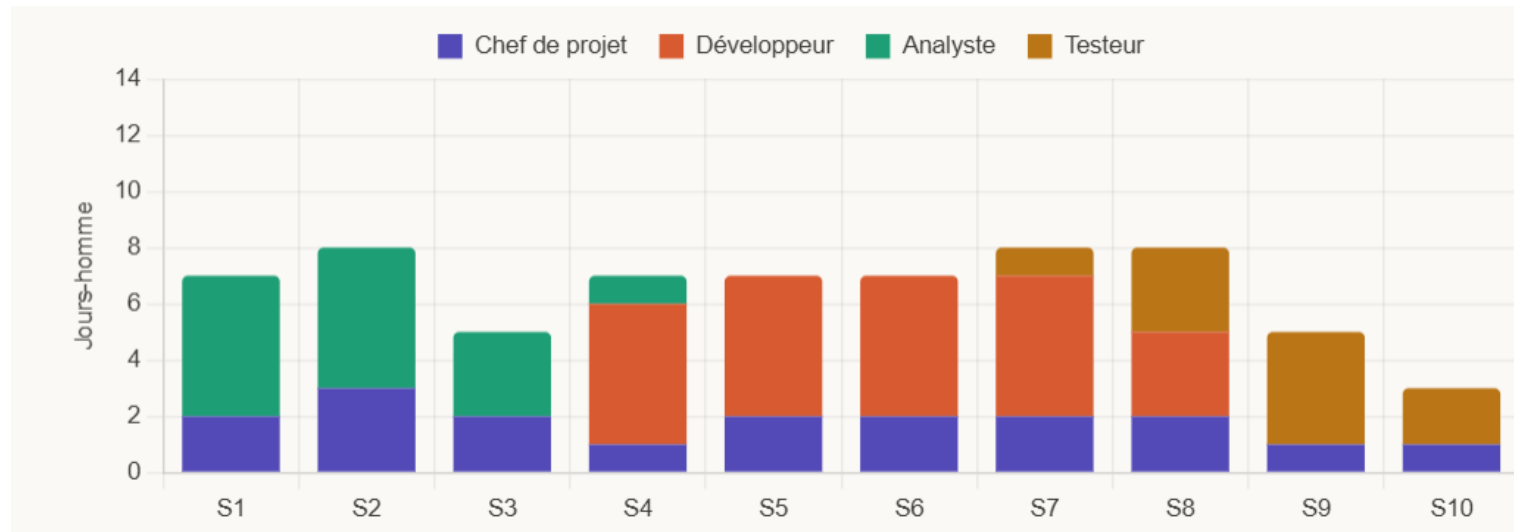
Un **jalon** est un événement de **durée nulle** qui marque une échéance importante : **fin d'une phase**, **livraison d'un document clé**, **validation** par le comité de pilotage, **mise en production**. Il sert de **point de contrôle** et de référence dans le suivi du projet.

Chapitre 7 : Planification d'un projet

7.6. La planification des ressources

7.6.1. L'affectation des ressources

Après avoir planifié les tâches dans le temps, on affecte les ressources humaines. Cela produit un **histogramme des charges** qui montre, pour chaque semaine, le nombre de jours-homme consommés par ressource.



Chapitre 7 : Planification d'un projet

7.6. La planification des ressources

7.6.2. Le nivellement et le lissage des ressources

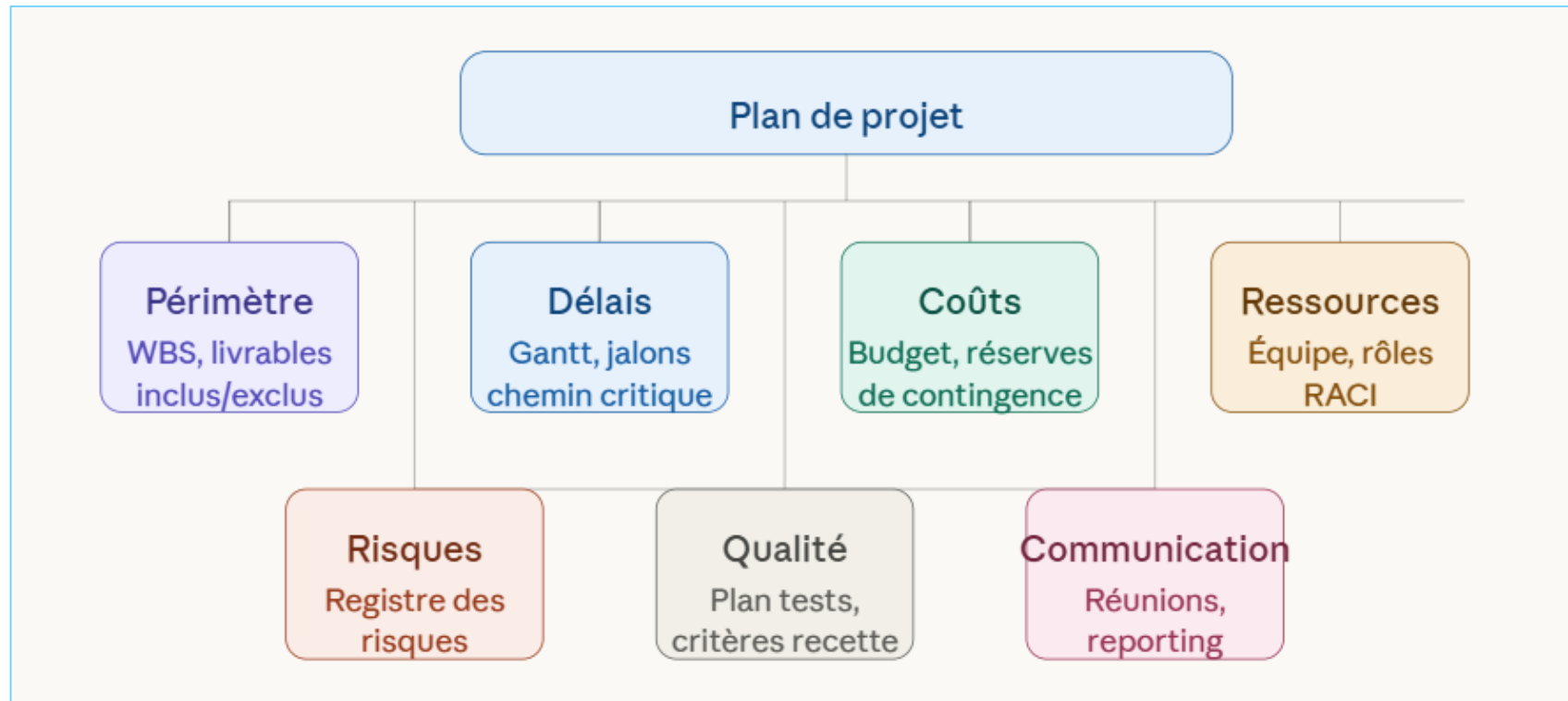
Quand une ressource est **sur-allouée** (plus de 100% de sa capacité), deux techniques existent :

- ❑ **Le lissage des ressources** : on déplace les tâches non critiques dans leurs marges pour réduire les pics de charge, sans modifier la date de fin du projet.
- ❑ **Le nivellement des ressources** : on accepte de décaler la date de fin du projet pour respecter les contraintes de ressources (ex : une seule personne disponible à la fois).

Chapitre 7 : Planification d'un projet

7.7. Le plan de projet (plan de management)

La planification se concrétise dans un **plan de projet** — document de référence qui regroupe toutes les décisions de planification.



Chapitre 7 : Planification d'un projet

7.9. La gestion des risques dans la planification

La planification intègre dès le départ la gestion des risques. Un risque est un événement incertain qui, s'il se réalise, affecte les objectifs du projet.

Le processus de gestion des risques :

Étape	Action	Outil
Identification	Lister les risques potentiels	Brainstorming, retour d'expérience
Analyse	Évaluer probabilité × impact	Matrice des risques
Planification	Définir les réponses	Plan de réponse aux risques
Suivi	Surveiller et actualiser	Registre des risques

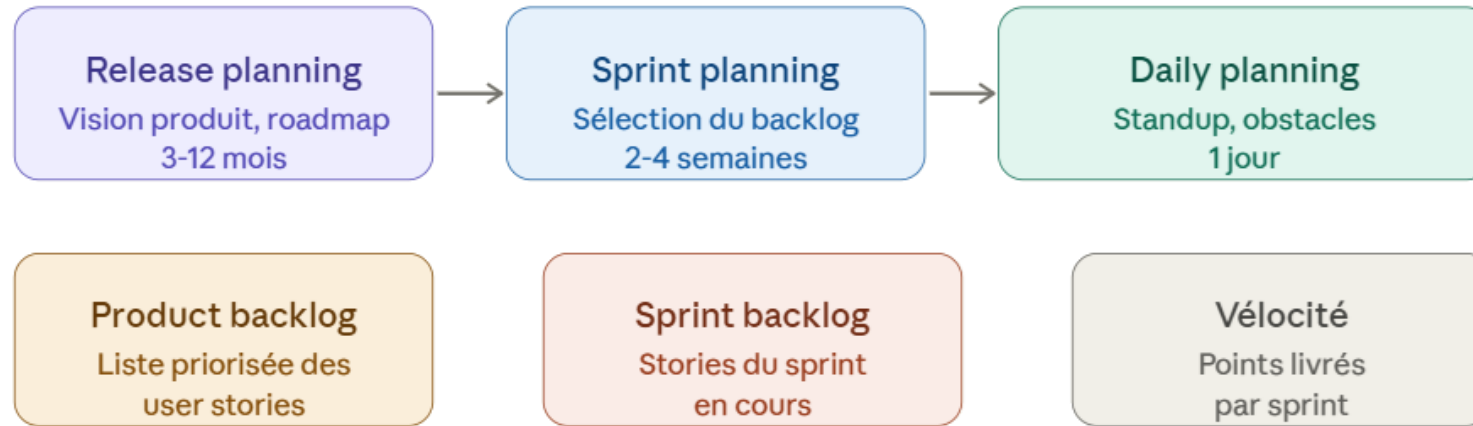
La matrice des risques classe chaque risque selon sa probabilité et son impact. Les risques en zone rouge (fort impact × forte probabilité) doivent être traités en priorité.

On distingue quatre stratégies de réponse : **éviter** le risque (changer le plan), **transférer** (assurance, sous-traitance), **atténuer** (réduire la probabilité ou l'impact), **accepter** (budget de réserve).

Chapitre 7 : Planification d'un projet

7.10. La planification en méthodes agiles

En méthodes agiles (Scrum, SAFe), la planification est **itérative et adaptative** plutôt que prédictive et figée. Elle se déroule à plusieurs niveaux :



En agile, on ne planifie pas tout en détail dès le début. Le **backlog** remplace le WBS complet : on y liste toutes les fonctionnalités souhaitées sous forme de **user stories**, et on les planifie au fur et à mesure dans les sprints, en fonction des priorités et de la **vélocité** de l'équipe.

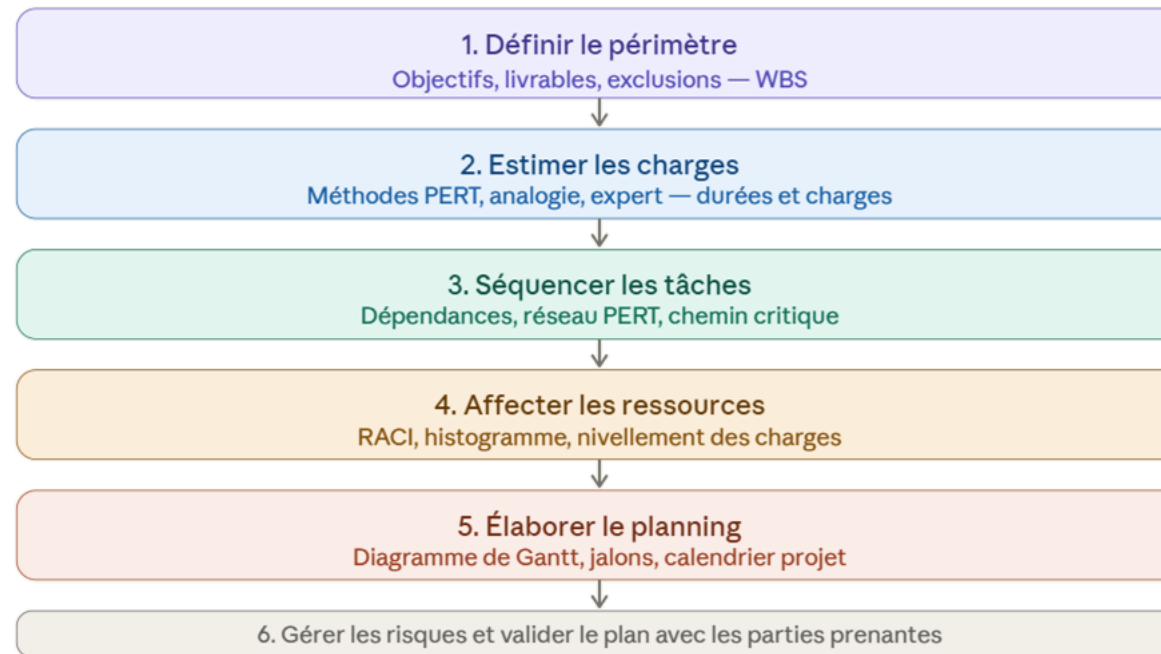
Chapitre 7 : Planification d'un projet

7.11. Les outils de planification

Outil	Usage	Exemples
Logiciels de planification	Gantt, chemin critique, ressources	MS Project, ProjectLibre (open source)
Outils agiles	Backlog, boards Kanban, burndown	Jira, Azure DevOps, Trello
Tableurs	Planification légère	Excel, Google Sheets
Outils collaboratifs	Partage, synchronisation	Confluence, Notion, Monday.com

Chapitre 7 : Planification d'un projet

7.12. Synthèse — Les étapes de la planification



À retenir : La planification n'est pas un acte unique réalisé une fois pour toutes au début du projet. C'est un processus continu : le plan est une **baseline** (référence de départ) qui sera mise à jour tout au long du projet en fonction **de l'avancement réel, des écarts constatés et des décisions prises**. **Un bon plan est un plan vivant, pas un document figé rangé dans un tiroir.**